

- Mobile Application Security
 - Start here
- Part 0: Foundations
 - Chapter 0.1: The vocabulary
 - Chapter 0.2: Cryptography you actually need
 - Chapter 0.3: Authentication, sessions and tokens
 - Chapter 0.4: What the platforms give you for free
 - Chapter 0.5: How to threat model a feature
 - Chapter 0.6: Classifying assets and ranking threats
- Part 1: Understanding the threat
 - Chapter 1: What an attacker can actually do
 - Chapter 2: The economics of the numbers
 - Chapter 3: The standard, and how its pieces fit
- Part 2: Data at rest
 - Chapter 4: How platform key storage actually works
 - Chapter 5: Key attestation — proving where a key lives
 - Chapter 6: Choosing storage in practice
- Part 3: Data in transit
 - Chapter 7: What TLS gives you, and what pinning adds
 - Chapter 8: The pinning debate, argued properly
- Part 4: Proving who is calling
 - Chapter 9: Play Integrity — what it proves and what it does not
 - Chapter 10: App Attest — Apple's model
 - Chapter 11: Biometrics, done so they cannot be bypassed
 - Chapter 12: The backend as the only real arbiter
- Part 5: Resilience
 - Chapter 13: How reverse engineering actually works
 - Chapter 14: Obfuscation and tamper detection — the honest chapter
- Part 6: The build and release pipeline
 - Chapter 15: Why your pipeline is now the target
- What actually shipped?
- Part 7: Platform surfaces and new frontiers
 - Chapter 16: WebViews — the most under-secured component in mobile
 - Chapter 17: IPC, deep links and app components
 - Chapter 18: Permissions and privacy
 - Chapter 19: Input validation and unsafe handling
 - Chapter 20: Securing AI features in mobile apps
 - Chapter 21: Kotlin Multiplatform — what can be shared
- Part 8: Practice
 - Chapter 22: Build your test lab
 - Chapter 23: Running an assessment and writing it up
 - Chapter 24: When something goes wrong
 - Chapter 25: Reference — testing method at a glance
- Part 9: The catalogues
 - Chapter 26: The 24 MASVS controls
 - Chapter 27: The 78 MASWE weaknesses
 - Chapter 28: The tests, techniques and practices you will use
- Part 10: Questions and answers
 - Foundations
 - Storage and keys
 - Key attestation
 - Network and pinning
 - Attestation and integrity
 - Biometrics
 - Backend
 - Pipeline
 - AI features

- Kotlin Multiplatform
 - Judgement
- Part 11: Making the case and running the programme
 - Chapter 29: The business case
 - Chapter 30: The regulatory landscape
 - Chapter 31: What comparable teams do
 - Chapter 32: The plan
- Part 12: Verification
 - Chapter 33: Audit log
 - Sources
 - Glossary

Mobile Application Security

A Study Book for Engineers

Edition 1 · September 2026 Hossam Atef — Software Engineer Android · iOS · Kotlin Multiplatform

Start here

Welcome. This book teaches you how to secure a mobile app, and more importantly, how to *reason* about mobile security so you can handle situations no book covers.

You don't need a security background. If you can build and ship a mobile app, you can read this. Part 0 covers the foundations: what encryption actually does, how authentication works, what the platforms give you for free. Every term gets defined the first time it appears, and there's a glossary at the back.

You do need to be willing to run things. Reading about a pinning bypass teaches you almost nothing. Running one against your own app teaches you a great deal. Part 8 walks you through building a test lab and using it.

What you'll learn

By the end of this book, you'll be able to:

- Describe what an attacker can do to your app, and explain why the usual instinct (hide the secret better) doesn't work.
- Choose where to store sensitive data on Android and iOS, and justify the choice.
- Explain how hardware-backed keys work well enough to answer "what happens when StrongBox isn't available?" without guessing.
- Decide whether to pin certificates, choose between static, dynamic and hybrid, and defend either answer.
- Implement device attestation and biometric authentication so they can't be bypassed by hooking a function, with working code for both platforms.
- Secure your build pipeline, which in 2026 is a more attractive target than your app.
- Threat-model a feature, including the new ones — AI assistants, agentic tooling.
- Test your own work and write findings someone can act on.
- Read the OWASP standard and speak its vocabulary.

How the book is organised

Part 0 — Foundations. Concepts everything else builds on: vocabulary, the cryptography you need, authentication, what the platforms give you free, how to threat model a feature, and how to classify what you are protecting. Skip it if you already know cryptography, authentication and the platform security models; come back to it when a later chapter uses a term you don't recognise.

Parts 1-3 — The core. Threat modelling, storing data safely, moving data safely.

Parts 4-5 — Trust and resilience. Proving who's calling; making reverse engineering expensive.

Part 6 — The pipeline. Your CI, your signing keys, your supply chain.

Part 7 — Platform surfaces and new frontiers. WebViews, IPC and deep links, permissions and privacy, input

validation, AI features, Kotlin Multiplatform. Most real findings live here.

Part 8 — Practice. Build a test lab, run an assessment on your own app, write it up, and handle the day something goes wrong.

Part 9 — The catalogues. The OWASP standard's own structure, so you can audit against it.

Part 10 — Questions and answers. Around a hundred questions with written-out answers. Use these to check yourself.

Part 11 — Making the case and running the programme. The business case, the regulatory position, what comparable teams do, and a phased plan with costs, owners and testable success criteria. Written to stand alone, so you can hand it to a manager.

Part 12 — Verification. What was checked, when, and what remains genuinely uncertain.

A few conventions

When a claim rests on a number or a specification, you'll find the source linked. When I'm summarising a standard rather than quoting it, I say so.

When the industry disagrees, you get both arguments rather than a quiet verdict. On some questions it disagrees sharply. You'll be the one in the design review, and you need to be able to argue it.

When something is a trap, it's flagged as one. There are more traps in this subject than you'd expect, and most of them look like working code.

On confidence. Not every claim in a book like this rests on equally solid ground, and hiding that would make the book less useful rather than more. So where a claim is weaker than it looks, you will see one of these markers:

- *(reported)* — a figure from vendor or practitioner write-ups rather than a primary source or forensic report. The mechanism is corroborated; the exact number may not be.
- *(estimate)* — a planning figure, not a measurement. Scale it to your own situation.
- *(reasoned)* — a conclusion this book draws by analogy or first principles, not something quoted from a standard. Sound, but yours to check.
- *(contested)* — the industry genuinely disagrees, and you get both sides.

Everything unmarked traces to a primary or authoritative source, listed at the back. Chapter 33 records what was verified and when.

On verification. Each implementation section ends with a **Verify it** block: the command to run and the pass/fail criterion. These are written from documented tool behaviour and from the OWASP MASTG test procedures. **They have not been executed against a live app** — expected outputs are described, never fabricated, and where behaviour varies by device or OS version the block says so. Treat them as a test plan, not a transcript.

On the code. Chapters covering a control end with a "How to implement it" section giving minimal, correct implementations for Android and iOS — and, where a mistake is common, the wrong version alongside it. Seeing `cbc` next to `GCM`, or `biometryAny` next to `biometryCurrentSet`, teaches faster than any amount of prose. The snippets are deliberately short: enough to adapt, not a substitute for the platform documentation, which is linked.

Part 0: Foundations

You need five things before the rest of the book makes sense: the vocabulary, enough cryptography to make decisions, how authentication actually works, what the platforms protect for free, and how to think about threats systematically. That's this part.

Chapter 0.1: The vocabulary

Security writing is dense with terms that sound interchangeable and aren't. Get these straight now and the rest of the book reads easily.

The three properties you're protecting

Security protects three things. Naming which one a control protects is the fastest way to tell whether it is the right control.

Confidentiality means only the right people can read the data. Encryption gives you this.

Integrity means nobody changed the data without you noticing. Hashes, message authentication codes and signatures give you this.

Availability means the service works when users need it. It's why a pinning mistake that disconnects every user is a *security* failure, not just an outage — you broke a security property.

You'll see these called the CIA triad. When you evaluate a control, ask which of the three it actually protects. A lot of confused design comes from assuming encryption gives you integrity, or that a signature keeps data secret. Neither is true.

Authentication versus authorization

These two words get used interchangeably in conversation. They are different jobs, and they fail separately.

Authentication answers "who are you?" Logging in, biometrics, a token that proves identity.

Authorization answers "are you allowed to do this?" Whether *this* authenticated user may read *that* record.

They fail differently and they fail independently. An app with perfect authentication and no authorization lets any logged-in user read every other user's data by changing an ID in a request. That bug class is extremely common and extremely serious, and it lives entirely on the server.

Threat, vulnerability, risk

A **threat** is something bad that could happen: someone steals a session token.

A **vulnerability** is the weakness that allows it: the token is stored in plaintext.

Risk combines how likely it is with how much it would hurt.

Why bother distinguishing them? Because you have limited time, and risk is what you prioritise on. A severe vulnerability nobody can reach matters less than a mild one on your login screen.

Attack surface, threat model, defence in depth

Your **attack surface** is every point where untrusted input or an untrusted actor meets your code: network responses, deep links, intents, WebView content, the clipboard, the camera, files a user picks, notifications, IPC from other apps.

A **threat model** is a structured answer to "what could go wrong here, and what do we do about it?" Chapter 0.5 shows you how to build one.

Defence in depth means layering controls so that one failure isn't total. The key word is *independent*: three controls that all fail when the device is rooted are one control wearing three hats.

Client, server, and why it matters here

Your **client** is the app on the user's device. Your **server**, often called the backend, is the code you run.

The single most important sentence in this book: **you control the server; you do not control the client**. Everything in Part 1 follows from that.

Terms you'll meet constantly

A quick pass through the words that appear on almost every page from here on.

Plaintext is data before encryption; **ciphertext** is after.

A **key** is the secret that makes encryption work. **Key material** means the actual bytes of a key.

A **nonce** is a "number used once" — a value you include in a request so an old copy of that request can't be reused.

Replay is what happens when you don't.

IPC is inter-process communication: mechanisms that let separate apps or processes talk. On Android that's intents, content providers, services and broadcasts.

Hooking is replacing a function's behaviour at runtime, without changing the file on disk. It's how most client-side security gets bypassed.

Rooting (Android) and **jailbreaking** (iOS) mean removing the OS restrictions that normally keep apps isolated from each other and from the system.

Attestation is a cryptographic statement from a trusted party about the state of something — usually "this really is a genuine app on a genuine device."

Chapter 0.2: Cryptography you actually need

You do not need to implement cryptography. You need to choose and use it correctly, and to recognise when someone else got it wrong. That's a much smaller topic, and this chapter is all of it.

The first rule: never write your own. Use the platform's implementation, or Google's Tink. Cryptographic code that looks right and is subtly wrong is the norm, not the exception, and the bugs are invisible in testing.

Symmetric encryption

One key encrypts and decrypts. Fast, and what you use for bulk data.

The algorithm you want is **AES**. But AES alone isn't a decision — you also choose a **mode**, and the mode is where things go wrong.

AES-GCM is what you should use. GCM is an **AEAD** mode, short for Authenticated Encryption with Associated Data. That means it gives you confidentiality *and* integrity together. If someone tampers with the ciphertext, decryption fails loudly instead of returning garbage you then trust.

AES-CBC encrypts but does **not** authenticate. Use it and an attacker can modify ciphertext in ways that change the plaintext predictably. If you must use CBC, you have to add a MAC yourself, correctly, in the right order — which is exactly the kind of thing people get wrong. Prefer GCM.

AES-ECB is broken for anything real. It encrypts identical blocks to identical ciphertext, so structure leaks straight through. If you find ECB in your codebase, that's a finding. `MASTG-TEST-0232` and `MASTG-DEMO-0058` exist for exactly this.

The IV/nonce rule that catches everyone. Modes need an **initialization vector** or nonce — a value that makes the same plaintext encrypt differently each time. It doesn't need to be secret, but with GCM it must **never repeat under the same key**. Reuse a GCM nonce and you don't just leak data, you can leak the authentication key itself. Always generate it randomly per operation and store it alongside the ciphertext. Never hardcode it, never use a counter you might reset, never reuse "just for this one case." `MASTG-TEST-0309` and `0310` test for this, and `MASWE-0007` covers it.

Asymmetric encryption

Two mathematically related keys: a **public key** you can share freely and a **private key** you protect. Anyone can encrypt to your public key; only your private key decrypts. Or: your private key signs, and anyone with your public key verifies.

Asymmetric crypto is slow, so it's used for small things — signing, key exchange, encrypting a symmetric key which then does the bulk work. That last pattern is called **hybrid encryption** and it's how TLS works.

The algorithms: **RSA** is the old standard, needs large keys (2048 bits minimum, 3072+ preferred). **Elliptic curve** (ECDSA for signing, ECDH for key agreement) gives equivalent strength with much smaller keys and better performance. Modern platforms prefer EC, and Apple's Secure Enclave supports **only** elliptic curve keys, which is a constraint you'll meet in Chapter 4.

Hashing

A **hash** turns input of any size into a fixed-size fingerprint, one way. You can't reverse it. Change one bit of input and the

output changes completely.

Use **SHA-256** or better. **MD5 and SHA-1 are broken** for security purposes — attackers can construct collisions, meaning two different inputs with the same hash. They're fine as non-security checksums; they are findings anywhere security depends on them ([MASWE-0008](#)).

Hashing is not encryption. Hashing is one-way and has no key. If you "hashed" something you need to read back later, you've lost it.

Password hashing is a different problem

Here's a distinction that trips up a lot of engineers: general-purpose hashes are designed to be **fast**, which is exactly wrong for passwords, because fast means an attacker can try billions of guesses.

For passwords, use a **deliberately slow** function: **Argon2** (preferred), **scrypt**, **bcrypt**, or **PBKDF2** with a high iteration count. These are called key derivation functions, and they take a **salt** — a random per-password value that stops an attacker from precomputing results for common passwords.

Note that this is almost always a *server* concern. Your app should send the password over TLS and let the backend hash it. An app that hashes a password locally and sends the hash has just made the hash the password.

[MASWE-0014](#) covers improper key derivation.

Message authentication codes and signatures

A **MAC** proves data wasn't modified *and* came from someone holding the shared key. **HMAC-SHA256** is the standard choice. Note that a MAC requires both parties to share a secret, so it can't prove *which* party — that's fine for client-server, useless for non-repudiation.

A **digital signature** uses asymmetric crypto: you sign with a private key, anyone verifies with the public key. This does prove origin, which is why app signing, JWT verification and attestation all use signatures.

The verification trap: generating a signature correctly and *verifying* it correctly are separate problems, and verification is where bugs live — accepting a token whose signature you didn't actually check, or accepting `alg: none`, or verifying against a key the attacker supplied. [MASWE-0011](#) is "improper verification of cryptographic signature" and it exists because this happens a lot.

Randomness

Cryptography needs unpredictable random numbers. Your language's default random number generator is usually **not** suitable — it's designed for speed and statistical distribution, not unpredictability, and its output can often be predicted from a few samples.

Use `SecureRandom` on Android and `SecRandomCopyBytes` or `CryptoKit` on iOS. Never seed a secure generator with something predictable like a timestamp — that throws away the entire property you wanted. [MASWE-0012](#), [MASTG-TEST-0204](#), [MASTG-BEST-0001](#).

Key lifecycle

Keys have a life, and each stage is a place to fail ([MASVS-CRYPTO-2](#)):

Generation — in secure hardware where possible, with a secure random source, at an adequate size.

Storage — in the platform keystore, never in your source code, never in your app package. A hardcoded key is not a key; it's a public constant with extra steps. [MASWE-0003](#), [MASWE-0004](#).

Use — restricted to the purpose you generated it for. A key that both signs and encrypts creates attack paths that neither would alone ([MASTG-TEST-0307](#)).

Rotation — a plan for replacing a key without losing access to data encrypted under the old one. Most apps have no plan, which is why [MASWE-0015](#) exists.

Destruction — actually removed when it's no longer needed, including on logout.

What to take away

Use AES-GCM for data, SHA-256 for hashes, Argon2 or bcrypt for passwords (server-side), elliptic curve for signing, a secure random generator always, and the platform keystore for keys. Never write your own primitives. Never reuse a GCM nonce. Never trust a signature you didn't verify properly.

That's enough cryptography to make good decisions for years.

Chapter 0.3: Authentication, sessions and tokens

Most mobile security work touches authentication, and most mobile authentication bugs come from a handful of misunderstandings. This chapter clears them.

What your app is holding

After a user logs in, your app holds something that proves the session. Usually two things:

An **access token** — short-lived (minutes to an hour), sent with each API request. Short life is the point: if it leaks, the window is small.

A **refresh token** — long-lived (days to months), used only to obtain new access tokens. This is the valuable one. It's why Chapter 6 tells you to keep access tokens in memory where you can, and give the refresh token real protection.

Many systems use **JWTs** (JSON Web Tokens) for access tokens: a base64-encoded JSON payload with a signature. Two things to know. First, **a JWT is signed, not encrypted** — anyone can read its contents, so never put anything secret in one. Second, **the signature only means something if you verify it**, and verification is a server job. An app that decodes a JWT to read the user ID and trusts it has trusted attacker-controlled data.

OAuth 2.0 on mobile, done correctly

If your app signs users in through an identity provider, you're using OAuth 2.0, and mobile has specific rules. They're in **RFC 8252, "OAuth 2.0 for Native Apps"**, and they exist because mobile breaks assumptions the base spec made.

Your app is a public client. This is the foundational point. RFC 8252 states it plainly: secrets embedded in an app binary are not secret, because they can be extracted from the app package. So any flow requiring a client secret in the app is broken by design. (This is Chapter 1's principle, arriving early.)

Use Authorization Code flow with PKCE. PKCE, short for Proof Key for Code Exchange and pronounced "pixie", solves a specific attack. Your app generates a random secret verifier, sends a *hash* of it with the authorization request, and presents the *unhashed* verifier when redeeming the code. An app that intercepted your authorization code wouldn't have the verifier, so the code is useless to it. RFC 8252 **requires** both clients and servers to use PKCE for public native app clients, and says authorization servers should reject native-app requests that don't use it.

Don't use the Implicit flow. RFC 8252 marks it NOT RECOMMENDED for native apps, for two reasons: it can't be protected by PKCE, and its access tokens can't be refreshed without user interaction.

Use the system browser, not a WebView. This one surprises people, because an embedded WebView feels more polished. RFC 8252 requires an external user-agent, meaning the browser, and explicitly does not support embedded WebViews. The reasoning: a WebView is *your* code, so users can't verify what site they're on, you can read what they type, and you don't get the single sign-on session the browser holds. Use Chrome Custom Tabs on Android and `ASWebAuthenticationSession` on iOS.

Don't build this yourself. The **AppAuth** libraries for Android and iOS implement the RFC 8252 pattern including PKCE. Use them.

Why the redirect matters: a private-use URI scheme (`myapp://callback`) can be registered by *multiple* apps, so it's indeterminate which one receives your authorization code — that's the code interception attack PKCE was created to mitigate. Prefer app-claimed `https` redirects (App Links on Android, Universal Links on iOS), which are verified against a domain you control. Chapter 17 covers that verification.

- <https://www.rfc-editor.org/rfc/rfc8252.html>
- <https://oauth.net/2/native-apps/>

Sessions, and ending them properly

A session begins at login and should genuinely end at logout. "Genuinely" is doing work in that sentence.

Logout must invalidate the token **server-side**. Deleting it from the client is not logout; it's hiding it. If the token still works when replayed, the user isn't logged out.

Logout must also clear **local data**: cached API responses, database rows, in-memory state, log entries, WebView storage and cookies. MASWE-0024, "sensitive data accessible after session termination", is common precisely because teams implement logout as an authentication concern rather than a data concern.

Set **timeouts** proportionate to risk: an inactivity timeout that requires re-authentication, and an absolute session lifetime. Shorter for financial flows.

Step-up authentication

Not all actions deserve the same proof. **Step-up** means requiring additional authentication for sensitive operations even within a valid session — biometrics before a transfer, a password before changing an email address.

`MASVS-AUTH-3` asks for this, and `MASWE-0023` names its absence. Chapter 11 covers implementing it so it can't be hooked away.

The bug class that isn't about tokens at all

Worth naming here because it's the most damaging authentication-adjacent bug and it isn't in the client: **missing authorization checks on the server**. Your app requests `/api/orders/12345`; the server returns it without checking whether it belongs to the calling user. Change the number, read someone else's order.

No client-side control fixes this. It's `MASVS-AUTH-1` and it's why Chapter 12 exists.

Chapter 0.4: What the platforms give you for free

Before you add anything, understand what Android and iOS already do. A surprising amount of "security work" duplicates a platform guarantee, and a surprising amount of real risk comes from switching one off.

The app sandbox

Both platforms isolate apps. Each app gets a private storage area other apps can't read, runs as its own user with its own process, and can only reach outside itself through defined channels.

This is the foundation of everything. It means your app's private files are protected from *other apps* by default — which is exactly why Chapter 6 argues that encrypting non-sensitive local data often buys nothing. The sandbox already handled the threat you were worrying about.

The sandbox does **not** protect you from the device's owner, or from an attacker who has rooted or jailbroken the device. Those cases remove the isolation.

Storage encryption

Both platforms encrypt storage at rest, tied to device unlock. Android has enforced **file-based encryption** since Android 10. On iOS it's **Data Protection**, with classes controlling when the decryption key is available (Chapter 4.4 covers those in detail, because choosing the class is a decision you make).

So data written by your app to private storage is encrypted at rest and isolated from other apps, without you doing anything. What that doesn't cover: a device that's unlocked and compromised, a backup that travels somewhere, or a class you loosened.

Code signing

Every app is cryptographically signed. The OS verifies the signature at install and refuses modified binaries. This is why repackaging your app requires re-signing it with a *different* key — and why your app can check its own signature to detect that (`MASTG-KNOW-0003`).

Signing also gates identity-based features: Android app links, keychain access groups on iOS, and the "restrict this API key to my app's signing certificate" protection in Chapter 15.

Permissions

Access to sensitive resources such as the camera, location, contacts and microphone requires a declared permission and, for dangerous ones, explicit runtime consent. Users can revoke consent later. Chapter 18 covers using permissions well, because over-requesting is both a privacy finding and a conversion problem.

Transport security defaults

Both platforms now require HTTPS by default. Android's **network security configuration** blocks cleartext unless you allow it; iOS's **App Transport Security** does the same. Modern Android also ignores **user-added certificate authorities** for app traffic by default, which is why proxying a modern app takes deliberate effort.

These defaults are strong. Most real network findings come from someone weakening them — a `cleartextTrafficPermitted="true"`, an ATS exception, a debug override that shipped.

Verified boot

The device checks at startup that it's running an unmodified OS, backed by hardware. You don't interact with this directly, but it matters: since May 2025, Play Integrity's device integrity verdict on Android 13+ **requires** a hardware-backed positive verified boot result. Chapter 9 explains what that means for your users.

Hardware key storage

Both platforms give you a place to put keys where your own process can't read them: the **Android Keystore** backed by a TEE or StrongBox, and the **iOS Keychain** backed by the Secure Enclave. This is the single most valuable thing the platform hands you, and Chapter 4 is devoted to how it works.

The takeaway

The platform handles isolation, storage encryption at rest, code integrity, permission gating, transport defaults and hardware key storage. Your job is to *use* those correctly, avoid switching them off, and add protection for the threats they don't cover — a compromised device, a leaked credential, an attacker who is the user.

Chapter 0.5: How to threat model a feature

Threat modelling sounds formal. In practice it's a forty-minute conversation with a whiteboard, and it's the highest-value security activity available to you because it's the only one that happens before the code exists.

Here's a method you can run on your next feature.

Step 1: Draw what you're building

Sketch the components and the data moving between them. App, backend, third-party services, local storage, any other app you talk to.

Draw **trust boundaries** — lines where data crosses from something you control to something you don't. Every boundary is where you need a check. The app-to-backend line is one. The user-to-app line is one. The "content from a WebView reaches native code" line is one, and it's the one people forget.

Step 2: Classify the data

For each piece of data, ask three questions. **How bad if it leaks? How bad if it's modified? How bad if it's unavailable?**

You'll find most data doesn't matter much, and one or two items matter enormously. That's the point: it tells you where to spend.

Step 3: Ask what could go wrong

Use prompts rather than imagination. STRIDE is the classic set — **S**poofing (pretending to be someone else), **T**ampering (changing data), **R**epudiation (denying an action), **I**nformation disclosure (leaking), **D**enial of service, **E**levation of privilege.

For mobile specifically, the MASWE catalogue in Chapter 27 is a better prompt list, because it's seventy-eight things that actually go wrong in mobile apps rather than six abstractions. Read down it and ask "does this apply?"

Four questions worth asking every time:

- What happens if the device is rooted or jailbroken?
- What happens if an attacker can see and modify all network traffic?
- What happens if a valid request is replayed from a different device?
- What happens if this input is hostile rather than the well-formed thing we tested with?

Step 4: Decide, and write it down

For each threat: **mitigate** it, **transfer** it (insurance, a third party's responsibility), **accept** it deliberately, or **eliminate** it by not building the risky thing.

Accepting is a legitimate choice, and writing down *why* is what makes it professional rather than negligent. "We accept that a rooted-device user can read their own cached order history, because it's their data and the value to an attacker is minimal" is a decision. Silence is not.

Step 5: Turn it into work

Each mitigation becomes a ticket with an owner. Each acceptance becomes a line in a document someone can revisit when circumstances change.

What good looks like

A one-page output: a diagram, a data classification, a short list of threats with decisions, and the accepted risks with reasons. Forty minutes. Revisited when the feature changes materially.

You'll notice this produces better results than any tool, because the hard part was never finding known vulnerability patterns — it was noticing that your new share-sheet feature lets another app hand you a file path you then read.

Chapter 0.6: Classifying assets and ranking threats

Chapter 0.5 gives you a method. This gives you two tables to fill in while you use it, because "what could go wrong" is easier to answer against a list than against a blank page.

Asset protection levels

Rate what you hold, because it tells you where to spend.

Asset	Typical classification	Impact if compromised
Authentication tokens	Critical	Full account takeover; access to everything the user can do
Payment credentials	Critical	Financial fraud and regulatory liability
User PII — email, phone, address	Critical	Identity theft; GDPR and similar liability
Restricted API keys	High	Unauthorised API use, metered cost, possible data access
Session state	High	Session hijacking and impersonation
App configuration and feature flags	Medium	Feature manipulation; possible service disruption
Analytics data	Low	Competitive intelligence leak
Public content	None	No impact

Threat likelihood and impact

Then rank the threats. The point of the exercise is the right-hand column: a threat with no mitigation named is a decision you have not made yet.

Threat	Likelihood	Impact	Mitigation, and where it is covered
API key extracted from the binary	Very high	High	Do not embed it. BFF pattern (Chapter 12.3); classification (Chapter 15.2)
Token theft from a compromised device	High	Critical	Hardware-backed storage and biometric binding (Chapters 6, 11)
Interception of traffic	High	Critical	TLS configuration, and pinning where justified (Chapters 7, 8)
App repackaged and redistributed	Medium	High	Attestation and signature verification (Chapters 9, 10)
Runtime hooking with Frida	Medium	High	Detection reported to a backend risk score (Chapters 12, 14)
Credential stuffing	High	High	Server-side rate limiting per account and device, plus attestation on login (Chapter 12)
Root or jailbreak exploitation	Medium	Medium	Detection and reporting; backend policy rather than a local block (Chapter 14.2)
Supply chain compromise	Medium	Critical	Dependency locking, SBOM, SDK audit, pipeline hardening (Chapter 15)

Both tables are starting points, not answers. Change the ratings to match your app — a messaging app and a parcel-tracking app should not produce the same numbers, and if yours match this table exactly, you have not done the exercise.

Part 1: Understanding the threat

Chapter 1: What an attacker can actually do

Every design decision in mobile security follows from an accurate picture of the adversary's capabilities. Most bad designs come from an inaccurate one — usually from imagining the attacker is a stranger on a network, when in fact the attacker owns the device your code is running on.

1.1 The device is not yours

When your app runs on a user's phone, you are executing code inside an environment that person fully controls. If that person is an attacker, or the device is rooted, jailbroken, emulated or simply instrumented, then every assumption you inherited from server-side development stops holding.

Concretely, assume an attacker can do all of the following, on both platforms, at will:

Read your code. A production APK decompiles to readable Java or Kotlin-like source in minutes with JADX. An IPA yields to radare2 and Hopper. Your class names, method flow, API endpoint structure, and any logic you wrote to make security decisions are all legible. Obfuscation renames things; it does not hide control flow from someone willing to spend an afternoon.

Read every constant in the binary. Run `strings` on your own release build. Whatever comes out, an attacker already has: API keys, endpoint URLs, encryption keys, debug flags, internal hostnames. This takes thirty seconds and it is the single most educational experiment in this book.

See all your traffic in cleartext. On a device they control, an attacker installs their own certificate authority and proxies your traffic through mitmproxy or Burp. TLS protects you from a stranger on the coffee shop network. It does not protect you from the person holding the phone.

Modify behaviour at runtime. Frida attaches to your process and lets an attacker replace any method's implementation while the app runs. A function that returns `true` when the device is rooted can be made to return `false`. A biometric callback can be invoked without any biometric. A certificate validator can be replaced with one that accepts everything.

Read process memory. Anything you decrypt "temporarily" exists in memory, and memory is readable. This is why "we decrypt it just before use" is not a mitigation.

Read local storage. On a rooted or jailbroken device, `/data/data/<package>/` and the app sandbox are open. SQLite databases, preference files, cached responses, and log files are all available.

Repackage and redistribute. Your app can be modified, re-signed with a different key, and distributed. Users who sideload will not notice.

1.2 What follows from this

Once you accept that list, one principle organises everything else:

Make secrets useless if found, not impossible to find.

Do not spend your budget trying to prevent extraction. You will lose, and the loss is not close. Spend it ensuring that what gets extracted has no value without a server-side check that the attacker does not control.

Three consequences deserve stating explicitly, because teams violate them constantly.

The client is never the trust boundary. Your server is. Any decision that matters — whether this user may transfer money, what this item costs, whether this account may be modified — happens server-side or does not happen securely at all.

Client-side checks produce signals, not verdicts. Root detection tells your backend "this session looks unusual." It does not tell your app "refuse this user." The distinction sounds pedantic and is load-bearing; Chapter 14 explains exactly why.

Defence in depth means genuinely independent layers. Each layer must reduce risk on its own, because you must assume any single layer will be defeated. Layers that all fail together are one layer wearing a costume.

1.3 Who is actually attacking you

"The attacker" is not one person, and treating them as one leads to spending money in the wrong places. In practice you

face four groups with very different economics.

The opportunist runs automated tooling against many apps looking for cheap wins: an exposed API key, an unauthenticated endpoint, a token in cleartext. They will not spend an hour on you. Almost every control in this book stops them, and this is where obfuscation and basic hygiene pay for themselves.

The cloner repackages your app to inject ads, steal credentials, or bypass payment. They need your binary to run modified. Signature verification, attestation, and integrity checks raise their cost materially.

The fraudster uses your app as intended, at scale, against your business logic — creating accounts, abusing promotions, laundering transactions, exhausting a metered resource. Client-side controls barely touch them. Backend risk scoring, rate limiting, and attestation-as-signal are what work.

The targeted adversary wants one specific thing badly and has time and money. They will defeat every client-side control you ship. What you can do is ensure that defeating them yields nothing without also compromising your backend, and that you detect them.

The reason to name these separately is triage. A news app worries about the opportunist. A banking app worries about all four. Knowing which one a control addresses stops you from arguing about obfuscation when your real problem is that your API trusts a client-supplied user ID.

Chapter 2: The economics of the numbers

You will be asked to justify security work in money. Use current figures and understand what they measure, because a stakeholder who catches you citing a stale number will discount everything else you say.

2.1 What breaches cost

IBM's *Cost of a Data Breach* report is the standard reference. The 2026 edition, released 29 July 2026, covers 602 organisations breached between March 2025 and February 2026.

The global average reached **\$4.99 million**, up 12% year over year and a record. The United States averaged **\$11.5 million**, more than double the global figure. Mean time to identify and contain rose to **247 days**, reversing five consecutive years of improvement.

Watch the trajectory, because people quote whichever year suits them: 2024 was \$4.88M, 2025 *fell* to \$4.44M — the first decline in five years — and 2026 rose to a record. If someone cites \$4.88M in 2026, they are two reports behind.

For the first time, IBM broke out AI-enabled breaches: **one in four** malicious breaches involved AI, and those averaged roughly **\$6 million**, about \$1 million above the mean. Deepfake and impersonation attacks drove the largest share of those incidents.

- <https://www.ibm.com/think/insights/cost-of-a-data-breach-industrial-sector>
- <https://databreachcost.com/report/2026>

2.2 What leaks look like

GitGuardian's *State of Secrets Sprawl 2026*, the fifth edition, published 17 March 2026, found **28.65 million** new hardcoded secrets added to public GitHub in 2025 — up 34%, the largest single-year jump recorded. Hardcoded secrets on public GitHub grew 152% between 2021 and 2025.

More usefully, it found that **64%** of secrets validated back in 2022 are still active. Leaking is common; remediating is rare. That asymmetry is the actual problem.

Two findings matter specifically to modern teams. **AI-assisted commits leak secrets at roughly double the rate of human-only commits, 3.2% versus a 1.5% baseline** — and eight of the ten fastest-growing leak categories are tied to AI services. Separately, **MCP configuration files alone exposed 24,008 unique secrets**, partly because official documentation encouraged hardcoding patterns.

And the location of leaks has shifted: **59%** of compromised machines in one analysed supply-chain attack were CI/CD runners rather than developer laptops, and about **28%** of secret incidents originate outside code repositories entirely, in Slack, Jira and Confluence. Those out-of-code leaks were 13 percentage points more likely to be rated critical.

- <https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/>
- <https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>

2.3 How to use these honestly

Two cautions, because misusing these numbers costs you credibility permanently.

First, these are enterprise averages. A mobile security incident at a mid-sized company usually costs far less than \$4.99 million. If you present the average as your exposure, someone will eventually notice, and every future request you make will be discounted. Say "the industry average for a full enterprise breach is X; our realistic exposure is Y, and here is how I derived Y."

Second, resist the temptation to invent ranges. Any "\$50,000 to \$500,000 per incident" figure you have seen is an estimate someone made up and everyone repeated. If you need a number for your own organisation, derive it: users affected × notification cost, plus support load, plus engineering time, plus whatever your regulator's penalty schedule says. That number will be defensible. The borrowed one will not.

Chapter 3: The standard, and how its pieces fit

There are four OWASP artifacts in this space and people conflate them constantly. Getting them straight is worth twenty minutes, because it is the vocabulary that security teams, auditors and interviewers use.

3.1 The four pieces

OWASP publishes four separate things for mobile security. Here is what each one is for.

The Mobile Top 10 is an awareness list of commonly observed risks, maintained by its own working group. It was updated in late 2024 — the first update in eight years. Its job is to make risk legible to people who are not specialists. It is not a standard and you cannot verify against it.

MASVS, the Mobile Application Security Verification Standard, is the standard: what must be true of a secure app. The current version is **v2.1.0**, released 18 January 2024, still current in 2026. It contains **eight categories holding twenty-four controls**, each with an ID like `MASVS-STORAGE-1` that a finding can point at.

MASWE, the Mobile App Security Weakness Enumeration, was introduced in July 2024 to fill the gap between high-level controls and low-level tests. It enumerates seventy-eight specific weaknesses — the things that actually go wrong.

MASTG, the Mobile Application Security Testing Guide, is how you verify all of it: test cases, techniques, background knowledge, best practices, and runnable demos, per platform. **v2.0.0** is the first stable non-beta release, completing a multi-year refactor that broke the old monolithic guide into individually referenceable, machine-readable components.

The chain runs:

MASVS control what must be true	→ MASWE weakness what goes wrong	→ MASTG test how to check it	→ MASTG demo a runnable example
MASVS-STORAGE-1 "securely stores sensitive data"	→ MASWE-0001 "Sensitive Data Stored Unencrypted in Private Storage"	→ MASTG-TEST-0001 "Testing Local Storage for Sensitive Data"	→ MASTG-DEMO-0059 "Using SharedPreferences to Write Sensitive Data Unencrypted"

Learn that chain. It is how you turn "make the app secure" into something a developer can fix and a tester can confirm.

One thing to watch when you look tests up. The v2 refactor is still in progress. Low-numbered tests (roughly `MASTG-TEST-0001` to `0100`) are the original v1 tests, and the project is progressively splitting them into smaller "atomic" v2 tests with higher numbers. Some v1 tests are already marked deprecated on their own page, with links to the v2 tests that replace them — `MASTG-TEST-0044` is one example.

So when you open a test page, **check for a deprecation banner first**. If you see one, follow it to the current test. Throughout this book, where a v1 ID is still the clearest way to point at a topic, it is used — but the page is the authority, not the book.

3.2 The levels that are not levels any more

This is the single most common way to date yourself in a security conversation.

Before MASVS v2.0.0 (April 2023), the standard contained verification levels: L1 as a baseline, L2 adding defence in depth for apps handling sensitive data, and R for resilience against reverse engineering.

Those levels **no longer live in MASVS**. They moved into the MASTG as *testing profiles*: **MAS-L1** for apps handling sensitive data at a basic level, **MAS-L2** for highly sensitive data requiring a higher level, and **MAS-R** for apps needing resilience against reverse engineering and tampering, independent of security level.

So the correct phrasing is "the MAS-L2 profile," not "MASVS L2." Likewise, if a blog post says "MSTG" rather than "MASTG," or cites L1/L2/R as MASVS levels, it predates the refactor and its API-level detail is probably stale too.

3.3 What OWASP will not do for you

OWASP states plainly that it **cannot certify** mobile applications, and does not accredit third parties to do so. Anyone selling "OWASP certification" is selling something that does not exist. What you can do is publish a *certification statement* — your own claim, verifiable by anyone with the MASTG, about which controls you meet and how you tested. That is more credible than a badge anyway, because it is falsifiable.

The MASVS is licensed under Creative Commons Attribution-ShareAlike 4.0, with no fee.

3.4 The tension inside the standard

Worth knowing early, because it will confuse you otherwise: **MASVS-NETWORK-2 asks for identity pinning on all remote endpoints under the developer's control, while OWASP's own Pinning Cheat Sheet argues that most apps should probably never pin.**

Both are OWASP. Both are current. They are not actually contradictory. The scoping phrase "under the developer's control" does the work, and Chapter 8 unpacks it. But discovering the apparent conflict mid-audit is disorienting. Now you know.

Primary sources:

- MASVS — <https://mas.owasp.org/MASVS/>
 - MASWE — <https://mas.owasp.org/MASWE/>
 - MASTG — <https://mas.owasp.org/>
 - Release history — <https://github.com/OWASP/mastg/releases>
-

Part 2: Data at rest

Chapter 4: How platform key storage actually works

Most developers use the Keystore and the Keychain without knowing what happens underneath, and then cannot answer basic questions about what their protection is worth. This chapter is the mechanism.

4.1 The Android Keystore, from the top down

The `AndroidKeyStore` you call from Kotlin is a Java Cryptography Architecture provider running **inside your app's own process**. It does not hold your keys. It forwards requests to a system daemon.

That daemon is **keystore2** on modern Android — rewritten in Rust, which matters because key handling is exactly the kind of code where memory-safety bugs are catastrophic. **keystore2** stores **keyblobs**: your key material, encrypted such that the daemon can store it but **cannot use or reveal it**. That property is the foundation of the whole design. Even the system service holding your key cannot read it.

Below the daemon sits a hardware abstraction layer implementing `IKeyMintDevice`. **KeyMint** is the current HAL, replacing the older **Keymaster**; it added Curve25519 support among other things. Behind the HAL runs the **KeyMint trusted application** — software executing in a secure context, most often in ARM TrustZone. That trusted app has access to raw key material, performs every cryptographic operation, and validates all access-control conditions on a key before permitting its use.

So when your code "encrypts with a Keystore key," what physically happens is: your process sends the plaintext and a key handle to the daemon, the daemon passes the encrypted keyblob and the request to the trusted app, the trusted app decrypts the keyblob inside secure hardware, checks the key's authorizations, performs the operation, and returns only the result. **The key never enters your process, and never enters the Android OS.**

One more component matters: **Gatekeeper**, the system component responsible for user authentication by password and fingerprint. It is not part of the Keystore, but the Keystore supports **authentication-bound keys**, meaning keys usable only after the user has authenticated. Gatekeeper is what vouches for that authentication. This is the machinery behind Chapter 12.

- <https://source.android.com/docs/security/features/keystore>
- <https://developer.android.com/privacy-and-security/keystore>

4.2 TEE versus StrongBox, and why the difference is real

A key's protection has a **security level**, and you can query it. `KeyInfo.getSecurityLevel()` returns one of three values (on API 28 and below, use the boolean `KeyInfo.isInsideSecurityHardware()`):

SOFTWARE means the OS is the only thing protecting the key. On a compromised or rooted device, treat that key as extractable. It is better than a key in your APK, and it is not hardware protection.

TRUSTED_ENVIRONMENT means the key lives in the TEE — a secure area of the *main* processor, isolated from the OS by hardware. If Android itself is fully compromised, the TEE remains intact: an attacker who can read internal storage might be able to *use* your Keystore keys on that device, but cannot *extract* them from it. Google's own open-source TEE is **Trusty**, provided free to OEMs; other TEE implementations exist and Android supports them.

STRONGBOX means the key lives in a dedicated, purpose-built secure processor — an embedded Secure Element or an integrated Secure Enclave, physically separate from the application processor. Titan M in Pixel devices is the well-known example. StrongBox arrived in Android 9 (API 28) and offers stronger isolation and tamper resistance than the TEE.

Three practical consequences follow, and they are where designs go wrong.

StrongBox is not universal. It became "strongly recommended" for devices launching with Android 12 and is expected to become a requirement eventually, but you cannot assume it. Request it, catch the failure, and decide your fallback deliberately.

StrongBox is slower and supports fewer algorithms. Low-power secure elements support a deliberately reduced subset of algorithms and key sizes. If you demand StrongBox with an exotic configuration, key generation will fail on devices that would otherwise have served you fine.

Silent fallback is a finding, not a fallback. If your payment flow requests StrongBox, fails, and quietly falls back to a software-backed key without telling anyone, you have a control that reports success while providing nothing. Either fall back to TEE explicitly and record that you did, or require server-side step-up authentication for that flow on that device. Write down which. One question separates people who have shipped hardware-backed crypto from people who have only

read about it: what does your app do when StrongBox is unavailable?

4.3 The iOS Keychain, from the top down

Apple's design differs in structure but rhymes in principle.

The Keychain is a **single SQLite database** on the file system, shared across all apps, managed by the **securityd** daemon. There is one database, not one per app. When your code calls a Keychain API, that becomes a call to `securityd`, which decides what your process may see by inspecting your `keychain-access-groups`, `application-identifier`, and `application-group` entitlements. Sharing between apps is possible only for apps signed by the same developer, enforced through code signing and provisioning profiles.

Encryption uses **two distinct AES-256-GCM keys per item**. A **metadata key** encrypts every attribute other than the secret value, which lets `securityd` search the database quickly; that key is protected by the Secure Enclave but cached in the Application Processor for speed. A **per-row secret key** encrypts `kSecValueData`, the actual secret, and using it **always requires a round trip through the Secure Enclave**.

That split is elegant and worth understanding: search stays fast because metadata decryption is cheap, while the thing that matters cannot be read without hardware involvement.

The **Secure Enclave** itself is a separate coprocessor, present on A7 and later. Apple's own description is the useful one: it provides all cryptographic operations for Data Protection key management and maintains the integrity of Data Protection **even if the kernel has been compromised**. Keys generated in it never enter RAM and are never processed by the OS or by user-space code; a `SecKey` in your Swift is a *handle*, not the key.

Two constraints on the Secure Enclave surprise people. It supports **elliptic-curve keys only** — no RSA. And it is for **signing and key agreement**, not direct encryption and decryption. To encrypt data with Secure Enclave protection you do ECDH key agreement or encrypt a symmetric key, rather than calling an encrypt function on an SE-resident key.

Access control lists on Keychain items are **evaluated inside the Secure Enclave** and released to the kernel only when their constraints are satisfied. That is why biometric-gated Keychain items are meaningfully protected and not just UI.

- <https://support.apple.com/guide/security/keychain-data-protection-secb0694df1a/web>

4.4 Data Protection classes: the decision you keep getting wrong

Every Keychain item carries an accessibility class via `kSecAttrAccessible`, and every file carries an analogous Data Protection class. They control *when* the decryption key is available.

`kSecAttrAccessibleWhenUnlocked` — readable only while the device is unlocked. Maps to `NSFileProtectionComplete` for files. The strongest ordinary choice.

`kSecAttrAccessibleAfterFirstUnlock` — readable after the user has unlocked once since boot, and thereafter even while locked. Maps to `NSFileProtectionCompleteUntilFirstUserAuthentication`. This is the class for items that background refresh needs, and Apple names that use case explicitly.

`kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly` — behaves like "when unlocked" but exists only if the device has a passcode set, and never leaves the device.

`kSecAttrAccessibleAlways` — readable at any time, mapping to `NSFileProtectionNone`. Apple discouraged this and it should not appear in new code.

Add `ThisDeviceOnly` to any of these and the item is excluded from backups and will not migrate to another device.

The failure mode is depressingly consistent. Background refresh breaks because an item is `WhenUnlocked`. Someone loosens the class to make the bug go away. Now a token that should require an unlocked device is readable on a locked one, and it is in the iCloud backup as well. **The right fix is almost always to restructure when the work happens, not to weaken the class**. If background refresh genuinely needs a credential, `AfterFirstUnlock` is the considered answer — not `Always`.

One more thing to know honestly: physical-access attacks have historically defeated these boundaries. Checkm8-class exploits and jailbreak tools such as checkra1n have enabled Keychain dumps without the user's passcode on affected hardware. Data Protection raises the bar substantially; it is not absolute against an attacker with the device and the right exploit.

4.5 A note on sharing

On Android, keys are per-app by default and the Keystore's isolation is strong. On iOS, adding another app to your `keychain-access-groups` entitlement means **trusting that app completely** — anything in the group can call `SecItemCopyMatching` on your items, including a compromised app from your own Team ID. Access groups are a genuine feature with a genuine cost; enumerate what is in yours.

Chapter 5: Key attestation — proving where a key lives

The Keystore protects your key. Attestation lets your *server* verify that protection independently, which is a different and more valuable thing.

5.1 The problem it solves

Your app tells your backend "I generated a key in hardware." Why would the backend believe it? A rooted device, an emulator, or a modified build can claim anything. Key attestation is the cryptographic answer to that question.

Key attestation arrived in Android 7.0 with Keymaster 2. ID attestation, which can include hardware identifiers, followed in Android 8.0 with Keymaster 3.

5.2 How it works

The flow is worth memorising because it is the shape of every attestation protocol you will meet:

1. Your **server** generates a random nonce and sends it to the app.
2. The **app** generates a key pair in the `AndroidKeyStore`, passing that nonce as the attestation challenge.
3. **Android builds an X.509 certificate chain** rooted in a Google hardware attestation root. The leaf certificate carries a `KeyDescription` extension containing the security level, boot state, the key's authorizations, and your nonce.
4. The app sends the **chain** to your server.
5. Your **server** verifies the chain against the Google attestation root, confirms the nonce matches what it issued, and reads the security level and key properties.

The crucial detail is where the information comes from. The authorization list in that certificate is **collected or generated by code inside the secure hardware and is not controlled by the platform** — sourced from the bootloader or over a secure channel that does not require trusting Android. That is why the claim is worth something: the OS cannot forge it, because the OS was never asked.

The `SecurityLevel` field tells you how resistant the key and the attestation itself are to attack: `SOFTWARE`, `TRUSTED_ENVIRONMENT`, or `STRONGBOX`. Reading that field is the point of the exercise.

- <https://source.android.com/docs/security/features/keystore/attestation>
- <https://developer.android.com/identity/digital-credentials/credential-issuer/keystore-attestation>

5.3 The thing that will break your production app in 2026

Attestation certificates used to be provisioned into devices at manufacture. Google moved to **Remote Key Provisioning (RKP)**, which improves privacy substantially: each application receives a different attestation key, keys rotate regularly, and Google's backend is segmented so that the server verifying a device's public key does not see the attestation keys — meaning Google cannot correlate attestation keys back to a device.

The operational consequence: **Google activated a new RKP root certificate on 1 February 2026, and all RKP-enabled devices must use it by 10 April 2026. Applications that verify key attestation and do not trust the new root will start failing.**

If your backend pins or hardcodes the attestation root, this is a live outage waiting for you. Many implementations do exactly that, following older tutorials. Two further notes from Google's own guidance: the chain is **longer** under online provisioning than it used to be and **is subject to change**, so do not hardcode a chain length; and the root is moving from RSA to ECDSA, so do not hardcode an algorithm either.

- <https://www.comviva.com/blog/safeguarding-cryptographic-keys-implementing-tee-and-strongbox-in-android-applications/>

5.4 What attestation does not prove

Two honest limits.

It proves properties of a **key**, not the trustworthiness of a **user**. An attested key on a genuine device in the hands of a fraudster is still a fraudster.

And the mechanism is a target. Tools exist that defeat hardware-backed key attestation by running a software KeyMint inside the real keystore daemon for selected apps, embedding AOSP's own reference trusted application and signing attestations with a supplied keybox — producing certificates generated the same way real hardware generates them, and

therefore internally consistent. Attestation raises the bar considerably. It is not a wall, and a design that treats a single attestation result as dispositive is brittle. Feed it into a risk score alongside other signals; Chapter 13 covers how.

Chapter 6: Choosing storage in practice

Now the applied chapter. This is where the most widely-repeated advice in Android development recently became wrong.

6.1 EncryptedSharedPreferences is dead

For years the recommended way to store a token on Android was `EncryptedSharedPreferences` from Jetpack Security Crypto (`androidx.security:security-crypto`), which wrapped `SharedPreferences` with Tink encryption and Keystore-held keys.

Google deprecated the entire library in April 2025, at version 1.1.0-alpha07, with no further releases planned.

Google's own reference now states the replacements bluntly: `EncryptedSharedPreferences` → use `SharedPreferences`; `EncryptedFile` → use `java.io.File`; `MasterKey` and `MasterKeys` → use `javax.crypto.KeyGenerator` with the `AndroidKeyStore` instance.

Google said little about why, but the reasons are visible in the bug reports. The library had to paper over Android Keystore inconsistencies across manufacturer devices and OS versions. It performed synchronous cryptography on the calling thread, producing `StrictMode` violations. And "keyset corruption" exceptions plagued Crashlytics logs on specific OEM devices. Maintaining two parallel storage stacks, legacy encrypted preferences alongside the modern `DataStore` path, was not sustainable.

The execution left developers without clear guidance for a long stretch, and Google still does not publish a single authoritative document combining `DataStore` with Tink for secure local storage. That gap is why this chapter exists.

- <https://developer.android.com/reference/androidx/security/crypto/package-summary>
- <https://blog.includesecurity.com/2026/08/encryptedsharedpreferences-is-dead-heres-what-you-should-use-instead/>

6.2 Superseded advice, kept findable

If you arrived here searching for one of these, you are in the right place and the advice has moved.

EncryptedSharedPreferences, EncryptedFile, MasterKey — deprecated April 2025. The whole Jetpack Security Crypto library was deprecated at 1.1.0-alpha07 with no further releases. Replacement in §6.3; why the original advice was misleading even when current in §6.4.

SafetyNet Attestation — retired. Play Integrity is the only supported path. Chapter 9.

UIWebView — deprecated. Use `WKWebView`, which runs content out of process. §16.6.

WHOIS-based domain email validation — discontinued 15 July 2025. §8.5.

"MASVS L1 / L2 / R". The verification levels left MASVS at v2.0.0 and became MASTG testing profiles: MAS-L1, MAS-L2, MAS-R. §3.2.

MASTG-TEST-0044 and MASTG-TEST-0087 ("Make Sure That Free Security Features Are Activated") — deprecated v1 tests. Superseded by the atomic v2 tests listed in §19.6.

Stubs exist because readers arrive from three-year-old blog posts searching the old term, and because a book that silently rewrites its own advice is harder to trust than one that shows the change.

6.3 What to use instead

Three layers, each doing one job:

Jetpack DataStore for persistence. Asynchronous I/O through coroutines, type-safe with Proto `DataStore`, no main-thread blocking. Note carefully that **DataStore is not encrypted**. It is a persistence mechanism, not a security one.

Google Tink for encryption. `StreamingAead` encrypts at file level rather than per value, which performs better and avoids a class of problems that per-value encryption created.

Android Keystore for key protection, via `KeyGenerator` with the `AndroidKeyStore` provider, so that key material never enters your process.

Migration uses `DataStore`'s `SharedPreferencesMigration` API to move existing values across. Two practical notes: the migration does **not** delete the old XML file, deliberately, so that a failed migration does not lose data — clear it yourself with `context.deleteSharedPreferences(...)` once you are confident. And if you cannot migrate now, a community fork

maintains `EncryptedSharedPreferences` and `EncryptedFile` against current Tink versions. Treat that as a bridge with a deadline, not a destination. Its own documentation notes that Tink Android 1.18.0 dropped SDK 21 in favour of a minimum of SDK 23.

6.4 The uncomfortable question nobody asked

Here is the part that got lost in a decade of "use `EncryptedSharedPreferences`" advice, and it is stated most clearly by the maintainer of that community fork: **the existence of the library misled developers into believing plain**

`SharedPreferences` was inherently insecure, which was never true.

Since Android 10, file-based encryption is enforced on device, and the app sandbox isolates your data from other applications. An attack that reads another app's `SharedPreferences` generally requires physical access to the device or an already-compromised device. If your threat model does not include those, encrypting your feature flags accomplished nothing except latency and a class of keyset-corruption crashes.

So decide by threat model rather than by reflex:

Long-lived refresh tokens and credentials deserve Keystore-backed keys, encrypted payloads, and `setUserAuthenticationRequired` where the user flow tolerates it. These are worth real protection because they are worth stealing and they last.

Short-lived access tokens are best held in memory for the session where your architecture allows. What is never written cannot be read off disk. Persist only what you must survive a process death.

Feature flags, UI state, non-sensitive preferences belong in plain `DataStore`. Encrypting them buys nothing.

Anything you can avoid storing should not be stored. This is the cheapest control in the entire book and the one most often skipped, because it requires a product conversation rather than a code change.

6.5 The leaks that are not about encryption

MASVS-STORAGE-2 exists because sensitive data escapes through channels that have nothing to do with your storage choice. Each of these is a real finding class:

Logs. A token in a debug log is a token on disk, and on Android any app with log access on older versions could read it. Strip logging from release builds and verify by inspecting the artifact, not by trusting a build flag.

Backups. Android Auto Backup and iCloud backup will happily copy your encrypted files somewhere the key does not exist, producing runtime crashes on restore — and will happily copy your *unencrypted* files somewhere an attacker can read them. Exclude sensitive data explicitly with backup rules; `isExcludedFromBackupKey` on iOS.

Screenshots. The OS captures your screen when the app backgrounds, for the recents UI. If a balance or a card number is on screen, it is now in a file. On Android use `FLAG_SECURE` and `setRecentsScreenshotEnabled`, plus `SecureFlagPolicy.SecureOn` for Compose dialogs, which people miss. On iOS use a covering view.

The keyboard cache. Predictive text remembers what users type into ordinary fields, including into fields that should never have accepted a secret. Use non-caching input types.

Notifications. A locked-screen notification showing a one-time code defeats the purpose of the one-time code.

The clipboard. On iOS the general pasteboard is shared and, historically, syncs across devices. Copying a password there is copying it to somewhere you do not control. Restrict to the local device, set an expiry, and clear after use.

Process memory. Nothing you can fully solve, but you can reduce the window: avoid holding secrets in long-lived `String` objects, prefer `CharArray` you can zero, and do not keep a decrypted blob resident for the app's lifetime.

6.6 How to implement it

Android: `DataStore` + Tink + Keystore

Three layers, per §6.2. The Keystore holds the key, Tink does the encryption, `DataStore` persists the result.

```
// 1. One-time: create a hardware-backed AES key that never leaves secure hardware.
private fun getOrCreateKey(alias: String): SecretKey {
    val keyStore = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }
    (keyStore.getEntry(alias, null) as? KeyStore.SecretKeyEntry)?.let { return it.secretKey }

    val spec = KeyGenParameterSpec.Builder(
        alias,
        KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT
    )
        .setBlockModes(KeyProperties.BLOCK_MODE_GCM) // AEAD, per Chapter 0.2
        .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
        .setKeySize(256)
        // For a refresh token, require the user to have authenticated:
        // .setUserAuthenticationRequired(true)
        // .setInvalidatedByBiometricEnrollment(true) // see Chapter 11.3
        .setIsStrongBoxBacked(true) // may throw; see below
        .build()

    return KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
        .apply { init(spec) }
        .generateKey()
}
```

Handle the StrongBox failure explicitly, because this is §4.2's decision in code:

```
val key = try {
    getOrCreateKey(ALIAS) // with setIsStrongBoxBacked(true)
} catch (e: StrongBoxUnavailableException) {
    // Deliberate, recorded fallback to TEE – not a silent downgrade.
    analytics.log("keystore_strongbox_unavailable")
    getOrCreateKeyWithoutStrongBox(ALIAS)
}

// Confirm what you actually got, rather than assuming:
val info = SecretKeyFactory.getInstance(key.algorithm, "AndroidKeyStore")
    .getKeySpec(key, KeyInfo::class.java) as KeyInfo
val level = info.securityLevel // SECURITY_LEVEL_STRONGBOX / TRUSTED_ENVIRONMENT / SOFTWARE
```

Encrypting, with a fresh IV every time — §0.2's rule that a GCM nonce must never repeat:

```
fun encrypt(plaintext: ByteArray, key: SecretKey): ByteArray {
    val cipher = Cipher.getInstance("AES/GCM/NoPadding")
    cipher.init(Cipher.ENCRYPT_MODE, key) // IV generated for us – do not supply one
    val ciphertext = cipher.doFinal(plaintext)
    return cipher.iv + ciphertext // store the IV alongside; it is not secret
}

fun decrypt(stored: ByteArray, key: SecretKey): ByteArray {
    val iv = stored.copyOfRange(0, 12)
    val ciphertext = stored.copyOfRange(12, stored.size)
    val cipher = Cipher.getInstance("AES/GCM/NoPadding")
    cipher.init(Cipher.DECRYPT_MODE, key, GCMParameterSpec(128, iv))
    return cipher.doFinal(ciphertext) // throws if tampered – that is the point
}
```

The wrong version, for contrast:

```
// DON'T: CBC gives you no integrity, and a fixed IV leaks structure.
val cipher = Cipher.getInstance("AES/CBC/PKCS5Padding")
cipher.init(Cipher.ENCRYPT_MODE, key, IvParameterSpec(ByteArray(16))) // fixed IV – broken
```

Then persist the ciphertext with DataStore. Remember DataStore itself is not encrypted; you encrypt, then persist.

iOS: Keychain with the right accessibility class

```
func storeToken(_ token: Data, account: String) throws {
    // Remove any existing item first; SecItemAdd fails on duplicates.
    let deleteQuery: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: account
    ]
    SecItemDelete(deleteQuery as CFDictionary)

    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: account,
        kSecValueData as String: token,
        // WhenUnlocked: needs an unlocked device. ThisDeviceOnly: not in backups,
        // does not migrate to a new device. See Chapter 4.4.
        kSecAttrAccessible as String: kSecAttrAccessibleWhenUnlockedThisDeviceOnly
    ]

    let status = SecItemAdd(query as CFDictionary, nil)
    guard status == errSecSuccess else { throw KeychainError.unhandled(status) }
}
```

If background refresh needs the token while the device is locked, the considered change is

`kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly` — **not** `kSecAttrAccessibleAlways`, which maps to no protection at all.

For a key that must never leave hardware, create it in the Secure Enclave — remembering from §4.3 that it is elliptic-curve only and does signing and key agreement rather than direct encryption:

```
let access = SecAccessControlCreateWithFlags(
    nil,
    kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly,
    [.privateKeyUsage, .biometryCurrentSet], // "CurrentSet", not "biometryAny" — Chapter 11.3
    nil
)!

let attributes: [String: Any] = [
    kSecAttrKeyType as String: kSecAttrKeyTypeECSECPPrimeRandom,
    kSecAttrKeySizeInBits as String: 256,
    kSecAttrTokenID as String: kSecAttrTokenIDSecureEnclave,
    kSecPrivateKeyAttrs as String: [
        kSecAttrIsPermanent as String: true,
        kSecAttrApplicationTag as String: "com.example.signing".data(using: .utf8)!,
        kSecAttrAccessControl as String: access
    ]
]

var error: Unmanaged<CFError>?
let privateKey = SecKeyCreateRandomKey(attributes as CFDictionary, &error)
```

Verify it

Code review does not tell you whether encryption is in the path you think it is. Check the artifact.

Android. Pull the file the app actually wrote and read it:

```
adb shell run-as com.example.app cat /data/data/com.example.app/shared_prefs/*.xml
adb shell run-as com.example.app ls -la /data/data/com.example.app/files/
```

Pass: base64 ciphertext, or binary that is not your token. **Fail:** readable JSON or a recognisable token string — which means your encryption layer is not on the write path, a common outcome when a migration half-succeeded.

Confirm the key is where you think:

```
val info = SecretKeyFactory.getInstance(key.algorithm, "AndroidKeyStore")
    .getKeySpec(key, KeyInfo::class.java) as KeyInfo
Log.d("keycheck", "securityLevel=${info.securityLevel}") // STRONGBOX / TRUSTED_ENVIRONMENT / SOFTWARE
```

Fail: `SOFTWARE` on a flow you designed for hardware backing. That is §4.2's silent-fallback finding, showing up in a log line.

iOS. On the simulator, inspect the app container for plaintext:

```
xcrun simctl get_app_container booted com.example.app data
grep -rI "eyJ" <that path> # a JWT begins "eyJ"
```

Pass: nothing. **Fail:** a token in a plist, a cache, or a log file.

6.7 Encrypting a local database

Chapter 6 covers tokens and preferences. Databases are the other half, and they are where most personal data actually lives.

The default position: on both platforms, app-private database files are already inside the sandbox and covered by platform storage encryption at rest (Chapter 0.4). For a great deal of app data, that is genuinely sufficient.

When to add database encryption on top: when your threat model includes a compromised or rooted device, when a regulator requires encryption of data at rest as a distinct control, or when the database holds something whose exposure would be individually serious — health records, financial history, message content.

Android. The standard route is **SQLCipher**, which provides a drop-in encrypted SQLite replacement and integrates with Room through a `SupportFactory`. Hold the passphrase in a Keystore-backed key rather than a constant — a hardcoded passphrase converts database encryption into an obfuscation exercise. Note the cost: SQLCipher adds binary size and a measurable performance overhead on large queries, so measure before committing.

iOS. Two options. SQLCipher works the same way. Or use **file-level Data Protection** by setting an appropriate protection class on the database file, which leans on the platform rather than adding a dependency — the trade-off being that your data is readable whenever the device is unlocked, which may be exactly what your app needs anyway.

What database encryption does not solve. The key has to be available for the app to read its own data, so on a device where an attacker can run code as your app, they can read the database through your app's own key. This raises cost against offline extraction and physical access. It does not defeat a live compromise. Be clear which of those you are buying.

Part 3: Data in transit

Chapter 7: What TLS gives you, and what pinning adds

To reason about pinning you first need a clear picture of what standard TLS already does, because most pinning arguments are really arguments about a gap that people cannot describe precisely.

7.1 The baseline

When your app connects over HTTPS, the server presents a certificate chain: a leaf certificate for your domain, one or more intermediate CA certificates, and a root. Your device validates that chain against a **trust store** of root certificates shipped and maintained by the platform. It checks that the chain is cryptographically valid, that the leaf covers the hostname you asked for, and that nothing has expired or been revoked.

This is genuinely strong, and it is free. It defeats the coffee-shop attacker completely.

Before you consider anything more exotic, get the free things right, because most apps still do not:

Require TLS 1.2 minimum, prefer 1.3. Configure it explicitly rather than inheriting a default that an old library overrides.

Forbid cleartext. On Android that is `cleartextTrafficPermitted="false"` in `network_security_config.xml`; on iOS it is App Transport Security. Then check for the hardcoded `http://` URL that some SDK is using anyway.

Do not trust user-added certificate authorities for your own domains. This is the single most valuable line in an Android network security configuration. By default, apps targeting modern Android already ignore user-added CAs, which is why proxying a modern app requires more effort — but if you have loosened this for debugging, make sure that configuration cannot ship.

Ship no debug network configuration. A `debug-overrides` block that reaches production is a hole with an audit trail.

Handle TLS errors correctly, especially in WebViews. `onReceivedSslError` with a call to `proceed()` is a complete bypass of everything above, and it appears in real apps because it makes a certificate warning go away during development.

7.2 So what is left for pinning to do?

Standard validation trusts **any** root in the device trust store. That means roughly a hundred organisations, plus anything an administrator or user has added, can issue a certificate for your domain that your app will accept.

Pinning narrows that set. Instead of "any trusted CA," your app accepts only a specific cryptographic identity you nominated. The threat it addresses is a **rogue or compromised CA**, or a certificate a device administrator has installed — including the enterprise MDM case and the "user tapped through a profile install" case.

That is a real threat, and it is a narrower threat than it was ten years ago. Which brings us to the argument.

Chapter 8: The pinning debate, argued properly

This is the most contested question in mobile security. You should be able to argue either side, because your interviewer might hold either position and both are defensible.

8.1 The case against pinning

Four arguments, and they are stronger than most pinning advocates admit.

OWASP's own Pinning Cheat Sheet is blunt about it. Its position is that for most applications the answer is probably never, because the risk of outages almost always outweighs the security benefit given advances elsewhere.

Google's Android documentation cautions against it, on the grounds that future server configuration changes carry a high risk of breaking the app.

The environment improved underneath the threat. Certificate Transparency means misissued certificates for your domain are publicly logged and detectable. Certificate lifetimes are collapsing, limiting the value of any single compromise. Automated issuance via ACME removed most of the human error that used to produce mistakes. Together these narrowed pinning's marginal benefit while leaving its operational risk untouched.

The failure mode is catastrophic and asymmetric. A pinning mistake does not degrade your app; it disconnects every user on that build simultaneously, and the only fix is an app store update — which takes hours at best, and which

users must then install.

8.2 The case for pinning

Four arguments the other way, and they are stronger than most pinning critics admit.

MASTG treats it as recommended practice, especially for MAS-L2 apps handling financial or health data, and MASVS-NETWORK-2 asks for identity pinning on remote endpoints under the developer's control.

MASTG explicitly addresses the Google warning, noting that it can be mistakenly interpreted as saying Google does not recommend certificate pinning, when the actual recommendation is to take the necessary precautions — backup pins and a rotation plan.

Mobile is not the web. The strongest technical argument. A web client cannot be updated on your schedule; a mobile client can be force-updated through the stores. The operational objection that kills pinning for browsers is materially weaker for apps.

Regulated industries do it anyway. Banking apps pin, frequently because a regulator or an auditor requires defence in depth rather than because the engineering team chose it. If you work in that world, "OWASP says don't" will not end the conversation.

8.3 The honest synthesis (*contested*)

The debate is not "pinning good" versus "pinning bad." It is about operational maturity.

Pinning done badly is worse than not pinning at all — one pin, no backup, no rotation plan, no monitoring —, because you have added a self-inflicted outage risk without meaningfully raising an attacker's cost.

Pinning done properly adds a real layer, and for a payment or health application that layer is worth the operational weight.

The question to ask is therefore not "should we pin?" but "**can this team operate a pinset for the next three years without locking users out?**" That is a question about runbooks, ownership, and monitoring, not about cryptography. If the answer is no, the correct decision is not to pin — and to say so plainly, rather than pinning badly to satisfy a checklist.

8.4 What to pin, and why SPKI

First: three different things people all call "pinning"

"SSL pinning" in a job description or a blog post can mean any of three things. You will meet all three names, so learn them now.

Type	What the app stores	What it compares against
Certificate pinning	The whole certificate, or a hash of it	The certificate the server presents, in full
Public key pinning , also called SPKI pinning	A SHA-256 hash of the Subject Public Key Info	The public key taken from a certificate in the chain
CA pinning	The identity of an issuing authority	Whether the presented chain passes through that authority

Certificate pinning is not just weaker — it is differently broken. Because it matches the whole certificate, it also pins the expiry date and the serial number. So it breaks on **every** renewal, even when the server reuses the same key pair. There is no situation where this is the right choice for a mobile app.

Public key pinning is the one you want, and the rest of this chapter assumes it.

CA pinning is really public key pinning applied further up the chain. It appears below as the "intermediate" and "root" options.

Now the rule

Always pin the hash of the **SPKI**, or Subject Public Key Info. Never pin the full certificate.

The reason is renewal. A certificate expires and is reissued regularly. If the new certificate carries **the same public key**, then its SPKI hash is unchanged and an SPKI pin still validates. A full-certificate pin breaks on every renewal without exception.

That gives you a choice of pin target, and the trade-off is trust breadth against operational resilience:

Pin target	Survives renewal?	What you are trusting
Leaf SPKI	Only if the key pair is reused	Exactly one key. Tightest, most fragile
Intermediate CA SPKI	Yes, across leaf renewals	Anything that CA issues for you
Root CA SPKI	Yes, for years	That CA's entire issuance surface. Weakest protection

Three questions this raises

Which hash algorithm? SHA-256. On Android's network security configuration it is the only value `pin digest` accepts, and it is why you see `sha256/` prefixes in OkHttp and in iOS configuration.

Does a more expensive certificate help? No. Domain-validated, organisation-validated and extended-validation certificates differ in how much the CA checked about *your organisation*, not in cryptographic strength. You are pinning a public key, so the validation level buys you nothing here. People assume otherwise, and it is worth correcting in a design review.

What about a self-signed certificate or your own private CA? For an internal or enterprise app where you control both ends, pinning your own self-signed certificate or private CA is legitimate and common. It shifts the calculus in both directions. There is no public CA that could misissue for you, which removes the very threat pinning exists to stop. But there are also no Certificate Transparency logs to fall back on, because CT only covers publicly trusted certificates. If you go this route, pinning is carrying more of the load, so the rotation runbook in §8.7 matters more rather than less.

Ship at least one backup pin. Always. A pinset with a single pin is one certificate incident away from a total outage that only an app store release can repair. The backup should be a pin you can actually switch to — a key you hold, or an intermediate you have verified.

8.5 The 2026 development that changes the arithmetic

Two clocks are shrinking, not one. Before the table makes sense, you need the concept behind the second column.

What domain validation is

Before a certificate authority gives you a certificate for `api.example.com`, it has to check that you actually control that domain. Otherwise anyone could request a certificate for your site. That check is **Domain Control Validation**, usually shortened to **DCV**.

You prove control in one of a few ways. You publish a specific DNS record the CA asks for (`DNS-01`). You place a specific file at a URL on the domain (`HTTP-01`). Historically you could also reply to an email at an address on the domain, though **WHOIS-based email validation was discontinued on 15 July 2025**.

Now the part that matters. Once a CA has validated your control, it does not re-check every single time it issues you a certificate. It can **reuse** that proof for a period. That period is the second column in the table, and it runs on a completely separate clock from the certificate's own lifetime.

So, two clocks doing two different jobs:

- **Certificate lifetime** — how often you must **replace the certificate**.
- **DCV reuse window** — how often you must **re-prove that you own the domain**.

The schedule

Both are shortening under CA/Browser Forum **Ballot SC-081v3**, approved in April 2025. Apple originally proposed it, and it was adopted with no votes against.

From	Maximum certificate lifetime <i>how often you replace it</i>	DCV reuse window <i>how often you re-prove ownership</i>
Until 15 March 2026	398 days	398 days
15 March 2026 (in force now)	200 days	200 days
15 March 2027	100 days	100 days
15 March 2029	47 days	10 days

What that means for your workload

Take the 2029 row and work it through.

Certificates. A 47-day maximum means replacing each one every six to seven weeks, about eight times a year. Most

teams do this once a year today. That is roughly an eightfold increase in renewal events. For an estate of 1,000 certificates, about 1,000 renewals a year becomes more than 8,000.

Validation. Here is the part that catches people, and it is why the second column is not a footnote. The 10-day reuse window means your proof of domain ownership **expires every 10 days, whether or not you are renewing a certificate**. You are not doing one validation per certificate. You are re-proving ownership on a rolling 10-day cycle, which works out at roughly **35 to 37 validations per year, per domain** — even though you only replace the certificate eight times.

That asymmetry is the whole point. A team that automates certificate renewal but not validation still fails, because the CA challenge step will hit an expired authorization.

The consequence. Manual validation stops working. Email replies and one-off file placement cannot run at a 10-day cadence. Automated validation through ACME becomes the only sustainable route.

The mechanism that makes it bearable

One newer option is worth knowing, because it exists precisely to solve this: **persistent DCV**, the `DNS-PERSIST-01` method introduced by Ballot SC-088v3 and permitted since November 2025.

Rather than changing a DNS record for every validation, you publish a single TXT record once, at `_validation-persist` on your domain. The CA then re-validates against that standing record with no per-renewal DNS change. If you own your DNS, ask your CA whether they support it. It turns 35 DNS changes a year into one.

One caveat on ACME

Since this section pushes you toward automation, here is the honest counterweight. Research presented at the ACM Web Conference in 2025 showed that **stolen ACME account credentials can produce fraudulent certificates without the attacker ever controlling the domain**, because of validation caching. Automating validation moves your risk from "we forgot to renew" to "our ACME account credentials are now a high-value target." Protect them the way you protect a signing key (Chapter 15.6).

Here is why this decides your pinning design. Automated renewal usually generates a **fresh key pair** each time. If it does, your leaf SPKI changes on every renewal — which was survivable at 398 days and is untenable at 47. Two ways out:

Pin the intermediate CA SPKI, which survives leaf rotation entirely. You widen your trust to that CA's issuance for your domain, and you accept that.

Or reuse the key pair across renewals, so the SPKI stays stable while the certificate rotates.

Either is defensible. What is not defensible is not knowing which one your infrastructure does. Go and ask whoever owns your certificates, today, before you ship a pin.

And to restate the trap in one line, because teams keep walking into it: **plan against both columns**. A team that reads only the certificate-lifetime column builds something that works in 2027 and falls over in 2029, when validation starts expiring four times faster than the certificate it supports.

- <https://www.digicert.com/blog/tls-certificate-lifetimes-will-officially-reduce-to-47-days>
- <https://shop.sslinsights.com/blog/ca-browser-forum-47-day-certificate-roadmap/>
- DCV explained, with the workload arithmetic — <https://www.encryptionconsulting.com/understanding-10-day-domain-validation/>
- Persistent DCV and `DNS-PERSIST-01` — <https://www.encryptionconsulting.com/persistent-dcv-dns-validation/>
- SC-081v3 in full, including the ACME research — <https://www.captaindns.com/en/blog/tls-certificate-validity-reduction-47-days>

8.6 Where the pins live: static, dynamic and hybrid

Everything above tells you *what* to pin. This section is about *where the pin lives*, and it is the decision that determines whether pinning is operationally survivable.

Static pinning

The pins are baked into the app at build time — an Android network security configuration file, or a trust-evaluation callback on iOS.

It is easy to audit and easy to reason about. You can read the file and know exactly what your app trusts. And it **welds your network operations to your app release cycle**: when a certificate changes, you ship a release. That clause is the whole problem, and §8.5's shrinking certificate lifetimes make it sharper every year.

The failure story is concrete. A certificate rotates, the old pin no longer matches, and every user on that build loses connectivity until they install an update. Some never will.

Dynamic pinning

The app ships with **bootstrap pins**, then fetches a **signed pin manifest** from a management endpoint at runtime.

Here is the essential detail, and if you take one thing from this section, take this: **the manifest is signed with a key that has nothing to do with the web PKI**. Think about why. If you validated the connection that delivers your pins using the pins it delivers, you would have circular trust and no security at all. So the manifest carries its own signature, verified against a public key compiled into the app, independent of TLS. TLS gets you a connection; the signature gets you authenticity.

What this buys you: rotation without a release. Your certificate changes, you publish a new manifest, and apps in the field pick it up. The problem static pinning has with 47-day certificates disappears.

Open-source implementations exist, such as Wultra's `ssl-pinning-ios` and its Android counterpart, and several commercial platforms sell it as a managed service. The motivation they all state is the same: certificate expiry forces app updates, some percentage of users never update, and those users lose access.

The honest cost of going dynamic

Vendor material tends to present dynamic pinning as strictly better. It isn't, and you should know the trade before you choose it.

You have moved your trust root, not removed it. That manifest signing key now needs protecting for the lifetime of your app, in an HSM or equivalent, with a rotation story of its own. You have swapped a certificate-rotation problem for a signing-key-custody problem. For some teams that is a clear win; for others it is the same problem wearing a different hat.

You have added a runtime dependency. If your management endpoint is down or unreachable, what does your app do? Fail closed and you have invented a new outage. Fail open and you have invented a bypass. You need an answer, and it needs a cached manifest with a defined staleness window.

You still need backup pins. Dynamic delivery does not remove that requirement; it just moves where the backups live.

You have stepped outside the standard. OWASP's guidance and the MASTG's tests are written around static pinning. MASTG-KNOW-0015 describes the platform mechanisms; there is no equivalent normative treatment of signed manifests. An auditor may not recognise your design, and you will be explaining it rather than pointing at a control ID.

My view, stated plainly (*reasoned*): for a small team, static pinning on an intermediate CA SPKI with a documented runbook and a real backup pin is usually the better engineering decision. Dynamic pinning earns its complexity when you have many endpoints, frequent rotation you do not control, a install base that updates slowly, or a security team that can actually own a signing key.

How this differs on Android and iOS

Dynamic pinning is not equally easy on the two platforms, and the reason is the same on both: **the declarative mechanism cannot be updated at runtime**. So on each platform, choosing dynamic means giving up the declarative option and its advantages.

	Android	iOS
Declarative mechanism	<code>network_security_config.xml</code> , API 24+	<code>NSPinnedDomains</code> in <code>Info.plist</code> , iOS 14+
Can it be updated at runtime?	No — compiled into the app	No — Apple documents no way to change it
Programmatic mechanism	<code>OkHttp CertificatePinner</code> , or a custom <code>X509TrustManager</code>	<code>URLSessionDelegate</code> trust evaluation
So dynamic pinning means	Rebuilding the pinner, or a custom trust manager	Implementing the delegate yourself
Covers WebView traffic?	Declarative: yes. Programmatic: no	Neither

Three consequences follow, and the third is the one that catches teams out.

On Android, going dynamic costs you WebView coverage. This is the important one. Android applies network security configuration rules to WebView traffic in the same app automatically, so static pinning protects your WebView for free. `OkHttp's CertificatePinner` does not — it only governs traffic through that `OkHttp` client. Move to dynamic pinning and your WebView silently stops being pinned. If you have a WebView loading anything sensitive, you either keep a static configuration alongside the dynamic path, or you intercept WebView requests yourself with `shouldInterceptRequest` and a custom trust manager. Neither is free, and the choice needs making deliberately rather than discovered in an audit.

On iOS, WebView was never covered anyway. `NSPinnedDomains` does not apply to `WKWebView` or `SFSafariViewController`, and neither does your `URLSessionDelegate`, because `WKWebView` does not route through your session. So iOS gives you no supported way to pin `WKWebView` traffic. That is a real limitation, not an oversight on your part, and the mitigation is architectural: do not load sensitive content in a `WebView` (Chapter 16.5).

CertificatePinner is immutable once built. You cannot add a pin to a live instance. Dynamic pinning on Android therefore means rebuilding the pinner and the client when a new manifest arrives, or wrapping the whole thing in a custom `X509TrustManager` that reads its pin set from a mutable source. The rebuild approach is simpler and usually correct; the trust manager approach is more code and more ways to be wrong.

One security trade-off worth naming. Declarative pinning is easy to audit, which is genuinely valuable — a reviewer reads one file. It is also easy to strip. Researchers have demonstrated removing `NSPinnedDomains` from an `Info.plist`, re-signing the app, and installing it with pinning simply gone; the same applies to editing a network security configuration in a repackaged APK. Pinning implemented in code, especially native code, costs an attacker more to remove. That is not an argument against declarative pinning, since an attacker on their own device wins either way (Chapter 1). But if your threat model includes redistributing a modified build to other users, it is a point in favour of the programmatic route.

Hybrid, which is what most mature apps do

Ship a **static backup pin** in the binary, and rely primarily on **dynamically fetched pins**.

You get rotation flexibility from the dynamic path and a floor under you if the management endpoint fails. This is increasingly the default in practice, for the obvious reason that it has the smaller failure surface of the two.

The safety valve people forget: expiration

Android's `pin-set` takes an optional `expiration` attribute, and it is worth understanding exactly what it does, because it is counter-intuitive.

After the expiration date, pinning stops being enforced. The connection still validates normally against the platform trust store, but your pins are ignored. It is a deliberate fail-open: it trades security for availability, so that a forgotten pin does not brick your app forever.

That is a genuine safety valve and it is also a silent downgrade. Set a date, and put a calendar reminder well before it — otherwise your app will quietly stop pinning one morning and nothing will tell you. OWASP's own guidance on this is to set an appropriate expiration date *and* ensure timely updates, precisely so the app does not end up bypassing pinning after the date has passed.

Side by side

The prose above gives you the reasoning. This gives you the comparison in one place.

	Static	Dynamic	Hybrid
Pros	Easy to audit — a reviewer reads one file. No runtime dependency. Recognised by the standard and by auditors. On Android, covers <code>WebView</code> traffic	Rotation without an app release. Survives 47-day certificates. Users on old builds keep working	Rotation flexibility with a floor under you. Smallest failure surface of the three
Cons	Rotation needs a release, and users who do not update lose access. Trivially stripped from a repackaged build	A signing key you must now protect for years. New runtime dependency and failure mode. Outside the standard, so you explain rather than cite. On Android, loses <code>WebView</code> coverage	Most moving parts. Two mechanisms to test and keep correct
Best for	One or two endpoints you control, and you can ship releases	Many endpoints, rotation you do not control, or a slow-updating install base	Most mature apps at scale
Worst for	Estates with frequent uncontrolled rotation	Small teams with no key-custody capability	Teams that will only maintain one path properly

Choosing

Situation	Approach
One or two endpoints, certificates you control, you can ship releases	Static , intermediate SPKI, backup pin, expiration set, runbook written
Frequent rotation you don't control, or slow-updating install base	Hybrid : static backup plus signed dynamic manifest
Many endpoints, a security team that can own a signing key	Dynamic , with a cached manifest and a defined staleness policy
You can't securely update the pinset, or don't control both ends	Don't pin . CT plus HSTS (§8.9)

8.7 What pinning costs you operationally

If you pin, you owe your team all of the following. This is the list that determines whether pinning helps or hurts.

A **documented rotation runbook** with a named owner, describing exactly what happens when a certificate changes. **Backup pins in every release.** A **monitoring alert on pin validation failures**, so you learn from telemetry rather than from app store reviews. **Certificate Transparency monitoring** for your domains, so you detect misissuance — which is the threat you pinned against in the first place. A **remote kill switch** to disable pinning without a release, because the alternative is waiting for store review during an outage. And an **explicit agreement with whoever manages certificates** that changes require a coordinated app release.

Missing any one of these is how pinning causes the outage it was supposed to prevent.

8.8 The decision, by application type

Reason from the threat, not the table — but the table encodes the reasoning.

Banking, fintech, trading: SPKI pinning on an intermediate with a backup, plus CT monitoring. Frequently a compliance requirement, and the value of the transactions justifies the operational weight.

Health and medical: the same. Patient data is high-value and long-lived, and the regulatory environment rewards defence in depth.

E-commerce: depends on where payment happens. Handle cards in-app and you are in the first category. Delegate to a gateway and CT plus HSTS is a reasonable position.

Ride-sharing, delivery, marketplaces: CT plus HSTS plus backend fraud detection. Payment is usually tokenised elsewhere, and the operational overhead of pinning buys little.

Social, messaging, content: CT plus HSTS. Platform defaults with a strict configuration are sufficient. Pinning a news feed adds risk and no security.

Enterprise and internal: pin only if you control both client and server. If a customer's MDM is legitimately inspecting traffic, pinning breaks your own deployment.

The rule that decides it, from OWASP's cheat sheet: if you do not control both ends of the connection, or you cannot securely update the pinset, do not pin.

8.9 The alternatives, properly described

"Use CT and HSTS instead" is easy to say and worth unpacking, because they do different jobs.

Certificate Transparency is a set of public, append-only logs of issued certificates. Every publicly trusted certificate must be logged. You monitor the logs for your own domains and get alerted when a certificate you did not request appears. Note what this is: **detection, not prevention**. It does not stop a misissued certificate from working; it tells you one exists, quickly, so you can revoke.

How to actually monitor Certificate Transparency

"Monitor CT" is easy to say. Concretely, you want to know when a certificate exists for your domain that you did not request.

The quickest manual check is a CT search service — `crt.sh` lets you query every logged certificate for a domain. Run it once now for your own domains; teams are routinely surprised by forgotten subdomains and certificates issued by providers they no longer use.

For ongoing monitoring you want alerts rather than a manual check: most commercial CT monitoring services and several CAs offer this, and you can build it by polling a CT log API for your domains and diffing against a known-good inventory. The requirement is not sophistication, it is **someone receiving the alert and knowing what to do with it** — which is a runbook question, not a tooling question.

Worth pairing with the inventory problem: you cannot detect an unexpected certificate for your domain if you do not know which certificates are expected.

HSTS tells clients to use HTTPS only for your domain, preventing protocol downgrade. It closes the "attacker strips TLS" path. It does nothing about a fraudulently issued certificate.

Backend anomaly detection catches the traffic pattern of an interception attempt — a client whose behaviour changes, requests arriving from unexpected paths, sudden shifts in device signals.

Together these give you fast detection and a narrowed attack surface without the outage risk. For most applications that is the correct trade. For the ones handling money or health records, layering pinning on top is worth it — if, and only if, \$8.6 is genuinely in place.

8.10 How to implement it

Minimal, correct implementations. Both platforms, static first, then the dynamic manifest shape.

Android: network security configuration (declarative, API 24+)

This is the first choice if your networking is not centred on OkHttp. One useful property: **Android applies these rules to WebView traffic in the same app automatically**, so your pins cover WebView requests without extra work.

```
<!-- res/xml/network_security_config.xml -->
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <domain-config cleartextTrafficPermitted="false">
    <domain includeSubdomains="true">api.example.com</domain>
    <!-- SHA-256 hashes of the SPKI, not of the whole certificate -->
    <pin-set expiration="2027-12-31">
      <!-- Primary: intermediate CA SPKI -->
      <pin digest="SHA-256">YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=</pin>
      <!-- Backup: never ship without one -->
      <pin digest="SHA-256">Vjs8r4z+80wjNcr1YKepWQboSIRi63WswXhIMN+eWys=</pin>
    </pin-set>
  </domain-config>
</network-security-config>
```

Reference it from the manifest:

```
<application android:networkSecurityConfig="@xml/network_security_config" ... >
```

Three things to notice. The `expiration` attribute is the fail-open valve from §8.6 — after that date pinning is silently off. `includeSubdomains` is convenient and widens what you are pinning, so be deliberate. And this mechanism needs API 24 or higher; below that you need the programmatic route.

Android: OkHttp CertificatePinner (programmatic, all API levels)

Choose this if your stack is already OkHttp-centric. It works on every API level, and under the hood it installs a custom `TrustManager`.

```
val certificatePinner = CertificatePinner.Builder()
    .add("api.example.com", "sha256/YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=") // primary
    .add("api.example.com", "sha256/Vjs8r4z+80wjNcr1YKepWQboSIRi63WswXhIMN+eWys=") // backup
    .build()

val client = OkHttpClient.Builder()
    .certificatePinner(certificatePinner)
    .build()
```

A trick worth knowing for getting the values: give `CertificatePinner` a deliberately wrong pin, make one request, and read the correct hashes out of the exception message. Faster than the OpenSSL incantation and less error-prone than copying from a blog.

Choosing an Android approach

	Network Security Config	OkHttp CertificatePinner	Both together
Pros	Declarative and auditable. Covers all app traffic including WebView. No code	Works on every API level. Programmatic, so it can be made dynamic. Clear failure exceptions	Defence in depth; if one is misconfigured the other still holds
Cons	API 24+ only. Cannot change at runtime. Stripped by editing one file in a repackaged APK	Only governs traffic through that client — WebView is not covered . Immutable once built	Two places to update; a mismatch between them is confusing to debug
Use when	Most apps, and any app with a WebView reaching a pinned host	Your stack is OkHttp-centric, or you need dynamic pins	Financial and health apps where the extra check is worth the maintenance

Android: making the programmatic path dynamic

`CertificatePinner` is **immutable once built**, so "update the pins" means build a new one and a new client:

```

class PinnedClientProvider(private val manifestStore: ManifestStore) {

    @Volatile private var cached: Pair<Int, OkHttpClient>? = null // manifest version -> client

    fun client(): OkHttpClient {
        val manifest = manifestStore.current() // verified signature, checked version
        cached?.let { (version, c) -> if (version == manifest.version) return c }

        val pinner = CertificatePinner.Builder().apply {
            manifest.entries.forEach { entry ->
                entry.pins.forEach { pin -> add(entry.host, pin) } // "sha256/..."
            }
            // Always keep the bootstrap pins as a floor.
            BOOTSTRAP_PINS.forEach { (host, pin) -> add(host, pin) }
        }.build()

        val built = OkHttpClient.Builder().certificatePinner(pinner).build()
        cached = manifest.version to built
        return built
    }
}

```

Two things this deliberately does. It keeps the **bootstrap pins** in the set rather than replacing them, so a bad manifest cannot lock you out of your own backend. And it keys the cache on manifest **version**, so the client is rebuilt only when the pin set actually changes — rebuilding per request would throw away your connection pool.

Remember from §8.6 that this path does **not** cover WebView traffic. If you have a WebView reaching a pinned host, keep a static `network_security_config.xml` entry for it as well.

iOS: NSPinnedDomains (declarative, iOS 14+)

Apple calls this **Identity Pinning**, and it is the iOS counterpart to Android's network security configuration. It arrived in iOS 14 and macOS 11, and a surprising number of teams do not know it exists.

```

<!-- Info.plist -->
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <false/>
    <key>NSPinnedDomains</key>
    <dict>
        <key>api.example.com</key>
        <dict>
            <key>NSIncludesSubdomains</key>
            <true/>
            <!-- Pin the intermediate or root CA. Use NSPinnedLeafIdentities
                 instead to pin the leaf. -->
            <key>NSPinnedCAIdentities</key>
            <array>
                <dict>
                    <key>SPKI-SHA256-BASE64</key>
                    <string>Ylh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=</string>
                </dict>
                <dict>
                    <!-- Backup pin -->
                    <key>SPKI-SHA256-BASE64</key>
                    <string>Vjs8r4z+80wjNcr1YKepWQboSIRi63WsWXhIMN+eWys=</string>
                </dict>
            </array>
        </dict>
    </dict>
</dict>

```

Note the value format, because it is a useful cross-check. `SPKI-SHA256-BASE64` is the base64-encoded SHA-256 digest of the certificate's DER-encoded ASN.1 Subject Public Key Info structure — **exactly the same value you put in Android's pin digest**. If your Android and iOS pins differ for the same endpoint, one of them is wrong.

Choose the right key. `NSPinnedCAIdentities` pins an intermediate or root authority; `NSPinnedLeafIdentities` pins the leaf certificate. Everything in §8.4 about chain position applies here, so `NSPinnedCAIdentities` on an intermediate is usually the right choice.

Five documented limitations, all of which have bitten someone:

`NSIncludesSubdomains` only covers **one** subdomain level. Deeper hosts need their own entries.

Values must be **duplicated in every Info.plist** you ship, and again for every distinct host. There is no shared configuration.

You **cannot use User Defined Settings variables** in a localized `Info.plist`, so you cannot parameterise pins per build configuration the way you might expect.

It does **not apply to WKWebView or SFSafariViewController**, despite both sitting on CFNetwork. Your WebView traffic is unpinned.

During development, changing or removing an entry may require **reinstalling the app** before ATS invalidates the

cached trust settings and stops trusting the old identity. Otherwise you will think your change did nothing.

And the honest caveat. Its declarative nature is its strength and its weakness. A security auditor can read one file, which is a real advantage. But an attacker can edit that file, re-sign the app and install it with pinning gone — Guardsquare published a walkthrough doing exactly that, and the same result is achievable in memory with Frida. Pinning in code costs more to remove. See §8.6 for when that distinction matters.

iOS: URLSessionDelegate, done in the right order

The order of operations here is the part people get wrong, so read the comments.

```
import Foundation
import Security
import CryptoKit

final class PinningDelegate: NSObject, URLSessionDelegate {

    // SHA-256 hashes of pinned SPKI – primary and backup
    private let pinnedKeyHashes: Set<String> = [
        "YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg=",
        "Vjs8r4z+80wjNcr1YKepWQbo5IRi63WswXhIMN+eWys="
    ]

    func urlSession(
        _ session: URLSession,
        didReceive challenge: URLAuthenticationChallenge,
        completionHandler: @escaping (URLSession.AuthChallengeDisposition, URLCredential?) -> Void
    ) {
        guard challenge.protectionSpace.authenticationMethod == NSURLAuthenticationMethodServerTrust,
            let serverTrust = challenge.protectionSpace.serverTrust else {
            completionHandler(.cancelAuthenticationChallenge, nil)
            return
        }

        // 1. Set the SSL policy for this host, so hostname validation happens.
        let policies = [SecPolicyCreateSSL(true, challenge.protectionSpace.host as CFString)]
        SecTrustSetPolicies(serverTrust, policies as CFArray)

        // 2. Let the system validate the chain FIRST. Skipping this is the classic
        // mistake: you would accept an expired or untrusted certificate that
        // happens to match your pin.
        var error: CFError?
        guard SecTrustEvaluateWithError(serverTrust, &error) else {
            completionHandler(.cancelAuthenticationChallenge, nil)
            return
        }

        // 3. Only now compare pins, against the whole chain so an intermediate pin works.
        guard let chain = SecTrustCopyCertificateChain(serverTrust) as? [SecCertificate] else {
            completionHandler(.cancelAuthenticationChallenge, nil)
            return
        }

        for certificate in chain {
            guard let publicKey = SecCertificateCopyKey(certificate),
                let keyData = SecKeyCopyExternalRepresentation(publicKey, nil) as Data? else {
                continue
            }
            let hash = Data(SHA256.hash(data: keyData)).base64EncodedString()
            if pinnedKeyHashes.contains(hash) {
                completionHandler(.useCredential, URLCredential(trust: serverTrust))
                return
            }
        }

        // 4. No pin matched. Refuse – never fall through to default handling here.
        completionHandler(.cancelAuthenticationChallenge, nil)
    }
}
```

Four notes. Use `SecTrustCopyCertificateChain`; `SecTrustGetCertificateAtIndex` and `SecTrustCopyPublicKey` are deprecated on current SDKs. **Hash the SPKI, not the certificate** — that is what `SecKeyCopyExternalRepresentation` gives you. **Iterate the whole chain** so that pinning an intermediate works. And **never call** `completionHandler(.performDefaultHandling, nil)` in your failure path — that is a pinning implementation that does not pin, and it appears in production apps.

For the equivalent value-extraction on the server side, note that your pin is the base64 of the SHA-256 of the SPKI — the same value in both platforms' configuration, which is a useful sanity check when they disagree.

Choosing an iOS approach

	NSPinnedDomains	URLSessionDelegate	TrustKit
What it is	Declarative, in <code>Info.plist</code> , iOS 14+	Your own trust evaluation in code	A long-standing third-party pinning library
Pros	Simplest to add. Auditable — one file. No code to get wrong	Full control of chain position and pin logic. Can be made dynamic. Harder to strip	Handles the fiddly parts for you, with reporting built in. Good first step for a team new to pinning
Cons	Cannot change at runtime. <code>NSIncludesSubdomains</code> covers one level only. Values duplicated per <code>Info.plist</code> . Does not cover <code>WKWebView</code> . Trivially stripped by editing the plist and re-signing	Easy to implement wrongly — see the four traps in this section. More code to maintain	A dependency in your trust path, which is itself a supply-chain consideration (Chapter 15.4)
Use when	You want pinning today with minimal risk of implementation error	You need intermediate pinning, dynamic pins, or resistance to stripping	Your team wants pinning working correctly before it wants it perfect

Kotlin Multiplatform: pinning with Ktor

If you share networking through Ktor, there is one thing to know before anything else: **Ktor does not handle pinning for you**. Pinning is enforced by the underlying engine, and each engine needs configuring separately. A project that configures Android and forgets iOS is pinned on one platform and open on the other, which is worse than it sounds because it will pass a test suite.

The shape that works: **pin values in `commonMain`, enforcement in each platform's `actual`**. The hashes are the same value on both platforms (§8.10), so sharing them removes the most common source of drift.

```
// commonMain – the pins themselves are shared
object CertificatePins {
    const val DOMAIN = "api.example.com"
    const val CURRENT_PIN = "sha256/YLh1dUR9y6Kja30RrAn7JKnbQG/uEtLMkBgFF2Fuihg="
    const val BACKUP_PIN = "sha256/Vjs8r4z+80wjNcr1YKepWQboSIRi63WsWxhIMN+eWys="
}

expect fun createPinnedHttpClient(): HttpClient
```

```
// androidMain – OkHttp engine, so CertificatePinner applies
actual fun createPinnedHttpClient(): HttpClient = HttpClient(OkHttp) {
    engine {
        preconfigured = OkHttpClient.Builder()
            .certificatePinner(
                CertificatePinner.Builder()
                    .add(CertificatePins.DOMAIN, CertificatePins.CURRENT_PIN, CertificatePins.BACKUP_PIN)
                    .build()
            )
            .build()
    }
}
```

```
// iosMain – Darwin engine, so you implement the challenge handler,
// which is the URLSessionDelegate logic from earlier in this section
// ported to Kotlin/Native: evaluate the chain first, then compare SPKI hashes.
actual fun createPinnedHttpClient(): HttpClient = HttpClient(Darwin) {
    engine {
        handleChallenge { _, _, challenge, completionHandler ->
            // 1. SecTrustSetPolicies + SecTrustEvaluateWithError (chain first)
            // 2. SecTrustCopyCertificateChain → SecCertificateCopyKey
            //    → SecKeyCopyExternalRepresentation → SHA-256 → base64
            // 3. Compare against CertificatePins; useCredential or cancel
        }
    }
}
```

Two mistakes to avoid, and they are the ones that actually happen. **Configuring one platform and forgetting the other** — put both `actual` implementations in the same pull request, and test both. And **assuming Ktor pins automatically** because the shared code looks like it is doing something; it is not, until an engine enforces it.

Note also that this only covers traffic through your Ktor client. Per §8.6, `WebView` traffic on Android needs a network security configuration entry as well, and on iOS cannot be pinned at all.

The dynamic manifest, in shape

Not a library, just the structure, so you can evaluate one or build one:

```
{
  "manifest": {
    "version": 42,
    "issuedAt": "2026-09-10T00:00:00Z",
    "expiresAt": "2026-10-10T00:00:00Z",
    "entries": [
      { "host": "api.example.com",
        "pins": ["sha256/YLh1dUR9y...=", "sha256/Vjs8r4z+8...="] }
    ]
  },
  "signature": "base64(ECDSA-P256-SHA256 over canonical bytes of `manifest`)"
}
```

The client logic that makes it safe:

1. Fetch over TLS with the **bootstrap pins** still enforced.
2. **Verify signature** against a public key compiled into the app. This is the trust anchor, and it is not a web PKI key.
3. Reject if **version** is **lower** than the cached version (*reasoned*) — otherwise an attacker replays an old manifest containing a pin they have since compromised. This is the same monotonic-counter idea as App Attest's assertion counter in Chapter 10.
4. Reject if past `expiresAt`, and define what happens then: fall back to bootstrap pins, not to no pinning.
5. Cache the manifest, and define a **staleness window** — how long you will run on a cached manifest before you require a fresh one.

Step 3 is the one that gets missed, and it is the difference between dynamic pinning and a downgrade attack.

Testing that any of this works

Do not trust the code review. Put mitmproxy in front of the app with its CA installed and confirm the connection **fails**. Then use `objection's android sslpinning disable` and confirm it now succeeds — which tells you your pinning was real and also how easily a determined attacker removes it on a device they control. MASTG-TECH-0012 is the technique; MASTG-TEST-0022, 0242, 0243 and 0244 are the tests.

Verify it

Pinning is the control most often believed-in and least often tested, because a working app proves nothing — an unpinned app also works.

Step 1 — confirm pinning is actually on. Put a proxy in front of the app with its CA installed on the device. **Pass:** the connection fails. **Fail:** you can read the traffic, which means pinning is absent, misconfigured, or not applied to that client.

Step 2 — confirm it can be bypassed, and note how easily.

```
objection -g com.example.app explore
android sslpinning disable
```

Expected: traffic now intercepts. That is not a failure of your implementation — it is §14.2's point demonstrated on your own app. What you are measuring is the *cost* to an attacker: whether `objection`'s generic bypass was enough, or whether they needed a custom Frida script for your specific implementation.

Step 3 — confirm the backup pin works, which nobody tests until the outage. Temporarily make your primary pin invalid and confirm the app still connects on the backup. If it does not, you have one pin, not two, and §8.4's mandatory backup is decorative.

Step 4 — confirm both platforms pin the same thing. The `SPKI-SHA256-BASE64` value on iOS and the `pin digest` value on Android are the same string. If they differ for one endpoint, one is wrong.

Relevant tests: MASTG-TEST-0022, 0242, 0243, 0244; technique MASTG-TECH-0012.

8.11 When pinning breaks: troubleshooting

Pinning failures are opaque by design — the connection simply refuses. Work down this table.

Symptom	Likely cause	What to do
App cannot connect after a certificate renewal	The new certificate's SPKI does not match any pin you shipped	Confirm whether your renewal reuses the key pair (§8.5). If not, your backup pin should have matched — check why it did not
<code>SSLPeerUnverifiedException</code> from <code>OkHttp</code>	Pin hash mismatch	Re-extract the SPKI hash from the live server and compare it byte for byte with the embedded value. The exception message contains the hash the server actually presented
<code>NSURLErrorServerCertificateUntrusted</code> on iOS	Pin validation failed in your delegate	Check which certificate in the chain you are extracting from. Pinning an intermediate while reading index 0 is the usual cause
Works on Android, fails on iOS (or the reverse)	Different certificate chains served per platform, usually via SNI or a CDN	Verify with <code>openssl s_client -servername .</code> Make sure the same chain is served to all clients
Pinning blocks every request, including in development	No debug bypass configured	Android: a <code>debug-overrides</code> block in the network security configuration. iOS: skip pinning under <code>#if DEBUG</code> . Make certain neither reaches release
Users locked out after an emergency rotation	No backup pin shipped, and no force-update mechanism	This is why backup pins are mandatory (§8.4) and why <code>MASVS-CODE-2</code> asks for enforced updates. Ship a hotfix and force the update
Works on Wi-Fi, fails on cellular	A carrier transparent proxy. Rare, but it happens	Test across carriers. Note that if this is real, pinning is doing exactly the job you added it for
A cloud provider's rotation broke pinning	Your provider rotated the intermediate CA	Pin the intermediate rather than the leaf, or move to dynamic pins (§8.6). Cloud-hosted endpoints rotate on someone else's schedule
Works in staging, fails in production	The two environments serve different chains	Extract hashes from both and make sure your pinset covers both, or pin per-environment by build variant
Broke immediately after a store release	A build-variant mix-up — debug trust configuration leaked into release	Inspect the release artifact, not the source (Chapter 15.7)

The pattern across most of these: the failure is almost never in the pinning code. It is in the assumption that the certificate you tested against is the certificate your users receive.

Part 4: Proving who is calling

Chapter 9: Play Integrity — what it proves and what it does not

Attestation is the highest-value control most teams skip, because it moves the trust decision to your server and backs it with a cryptographic statement from the platform vendor rather than a claim from a client you do not control.

SafetyNet Attestation is fully retired. Play Integrity is the only supported path on Android.

9.1 The shape of the thing

Your app requests an **integrity token** from Google Play services. That token is encrypted and signed by Google. Your app sends it to your backend, your backend sends it to a Google server for decryption and verification, and Google returns a verdict payload. Your backend decides what to do with it, then tells the app.

Note where the decryption happens. The token is not something your app can read or forge, and the verdict arrives at your server from Google rather than from the device. That is what makes it worth more than any client-side root check.

The payload is plain-text JSON with a fixed structure:

```
{
  "requestDetails": { ... },
  "appIntegrity": { ... },
  "deviceIntegrity": { ... },
  "accountDetails": { ... },
  "environmentDetails": { ... }
}
```

Check `requestDetails` first, before you look at anything else. It contains the package name and the nonce or request hash. If those do not match the request you issued, nothing else in the payload means anything — you may be looking at a replayed token from a different request or a different app.

9.2 Standard or classic — choosing properly

There are two request types and the choice is not cosmetic.

Standard requests suit any app and are what most developers should use. Latency is a few hundred milliseconds on average, reliability is high, and importantly, Google Play takes on some of the protection against replay and token exfiltration. They need a two-step flow: **prepare the token provider well in advance** (at app launch or in the background), then **request a token on demand** when you make the server call you want to verify.

Classic requests are more expensive to make, and **you** are responsible for correctly implementing protection against exfiltration and certain attack classes. Your server issues the nonce. They are intended as an occasional one-off check on a high-value action, not as a per-request mechanism.

The practical guidance: standard for nearly everything, classic for infrequent high-confidence checks where you want a server-issued nonce and are prepared to own the replay protection.

Operational limits to design around. Practitioner reporting puts classic requests around 5 per minute per app instance, standard warm-ups similarly, with a default daily ceiling near 10,000 token requests and 10,000 decodes (*reported*). These figures are not a stable published table — verify yours in Play Console and request an increase early if you need one, rather than discovering the ceiling on launch day.

One trap that costs people hours: never decrypt the same token twice. Repeated decryption returns **cleared verdicts** — the device recognition verdict comes back empty, and the app recognition and licensing verdicts come back **UNEVALUATED**. If you are seeing mysteriously empty verdicts in production, check whether a retry path is decrypting a token that was already consumed.

9.3 Reading the verdicts like an engineer, not a bouncer

The device recognition verdict differentiates three labels, and Google strengthened all of them in May 2025:

MEETS_STRONG_INTEGRITY — on Android 13 and later, this now additionally requires a **security update installed within the last 12 months**.

MEETS_DEVICE_INTEGRITY — on Android 13 and later, this now **requires a hardware-backed, positive verified boot verdict**.

MEETS_BASIC_INTEGRITY — the weakest label, indicating a device that passes minimal checks.

The same May 2025 change reduced the device signals needed for the default verdict by roughly **90%** and improved worst-case latency by up to **80%**, while explicitly increasing the differentiation between the three labels. Verified boot became the anchor.

Here is the consequence almost everyone gets wrong. `MEETS_STRONG_INTEGRITY` failing does **not** mean the device is compromised. On Android 13+, it most often means a legitimate user has not received a security update in a year — which is extremely common on older or regionally-supported hardware, and entirely outside that user's control.

So treat the labels as a gradient, not a gate:

Strong integrity is a bonus signal for your very highest-risk actions. Never a baseline requirement, or you will lock out real customers on older devices.

Device integrity is a reasonable expectation for sensitive flows on modern hardware. When it fails, route the session into a higher-risk bucket rather than blocking: allow browsing, require step-up authentication for anything sensitive, cap limits, monitor.

Basic integrity only means treat with suspicion and escalate scrutiny — not refuse service.

9.4 How to roll it out without breaking your users

Google's own documentation recommends the sequence, and it is right: **implement without enforcement first.**

Ship the integration, collect verdicts from your real install base, and look at the actual distribution. Only then estimate the impact of each enforcement option and adjust your anti-abuse strategy accordingly. Enforcing on day one against an unknown distribution is how a team blocks a measurable slice of paying customers and finds out from app store reviews.

Then enforce incrementally, starting with your highest-value flows and leaving general use alone.

When you do refuse or restrict something, give the user a route out. Library version 1.5.0 (August 2025) added remediation dialogs, `GET_INTEGRITY` and `GET_STRONG_INTEGRITY`, precisely so that a user can fix their own problem rather than hitting a dead end that reads "device not supported."

- <https://developer.android.com/google/play/integrity/overview>
- <https://developer.android.com/google/play/integrity/standard>
- <https://developer.android.com/google/play/integrity/improvements>
- <https://developer.android.com/google/play/integrity/verdict>

9.5 The honest limits

Play Integrity proves that a request comes from your genuine, Play-installed, unmodified app binary, on a device meeting a stated integrity level, associated with a licensed account.

It does **not** prove that the user is legitimate. It is not a jailbreak oracle. It depends on Google Play services being present and functional, which excludes some legitimate devices and markets entirely. And independent commentary reasonably notes that quota and threshold behaviour is a real constraint at scale, which is part of why commercial alternatives exist.

Use it as one strong input to a backend risk decision. Not as the decision.

Chapter 10: App Attest — Apple's model

Apple solves the same problem with a different structure, and the difference is worth understanding because it shapes your integration.

10.1 Attest once, assert often

The flow has two distinct phases, and conflating them is the most common implementation error.

Registration, once per key. Your app generates a key in the Secure Enclave and gets a key ID. It asks your server for a challenge. It calls `attestKey(keyId, clientDataHash:)`, which **contacts Apple's servers**. App Attest fetches the key pair along with attestation data derived from the Secure Enclave, which is a snapshot of the device's hardware properties taken at boot and cannot be modified. Apple validates the device data and returns an attestation. Your server validates that attestation and **stores the public key** against that user.

Ongoing use, per protected request. Your app calls `generateAssertion(keyId, clientDataHash:)`. Assertions are **generated locally with no round trip to Apple**. Your app embeds the assertion in the request payload, and your server verifies it.

That asymmetry dictates the design: attestation contacts Apple, assertions do not. Attest rarely. Assert as needed, on the requests that matter.

Assertions still perform real cryptographic work, so do not generate them in tight loops or on hot lifecycle paths. Use them for sensitive payloads: authentication-related requests, premium content access, transactions.

10.2 What your server must do

Three checks, and the third is the one people skip.

Verify the assertion signature using the authenticator data, the server challenge, and the public key you stored during attestation.

Confirm the authenticator data contains the hash of your app ID. This binds the assertion to your application.

Track the assertion counter per key and require it to be strictly increasing. Without this, assertions are replayable — and you have built an elaborate signature check that does not stop the attack it exists to prevent. This is the single most common App Attest implementation bug.

On the nonce: the challenge can be a server-issued random value, or predictably reconstructible data such as a request digest or timestamp. What matters is that your server can independently reconstruct it for that specific request, which is what eliminates replay.

10.3 What changed in 2026

WWDC26 Session 201 introduced material additions:

A **fraud metric** you can incorporate into your risk pipeline, rather than treating attestation as a binary.

New signals in iOS 27 to strengthen validation.

App Attest on macOS 27, extending the model beyond iOS.

And one piece of operational guidance that deserves quoting because it prevents a real customer-harming bug: **do not reject every new key for an existing user.** Reinstalls, device migrations and restores legitimately produce new keys. A backend that treats "this user presented a key I have not seen" as fraud will punish honest customers for changing phones. Handle it as a re-registration event with appropriate risk scoring.

- <https://developer.apple.com/videos/play/wwdc2026/201/>

10.4 Errors you will actually meet

`NSError.invalidKey` and `serverUnavailable` are the two you will see, and they want different handling. For `invalidKey`, discard the key and generate a fresh one before retrying. For `serverUnavailable`, retry with the **same** key and the same client data hash, per Apple's own guidance. Cap your retries and back off.

Two known rough edges, from developer forum reports rather than documentation: a small subset of devices fail `attestKey` **persistently** with `invalidKey (reported)`, surviving reinstall, reboot and app update; and Apple has not confirmed whether throttling can surface as `invalidKey` rather than as a distinct error. Apple does not publish rate limits, and there is no documented process to request accommodation for a large-scale rollout.

The practical response: **stage your rollout.** Do not trigger simultaneous attestation across your entire install base. Use exponential backoff, honour `retryAfter`, and design a grace mode that lets a user continue with elevated scrutiny rather than being locked out because attestation failed on their particular handset.

10.5 Limits, and when to actually check

Apple publishes no thresholds for App Attest, which makes planning harder than it should be. Here is what is known, with the confidence level attached.

Limit	Value	Confidence
Daily quota	Not publicly disclosed. No official SLA or quota documentation	Apple
Rate limit	Roughly 10-20 requests per second	<i>(reported)</i> — community observation, no Apple guarantee
<code>attestKey()</code> calls	Once per key. In practice that means once per app installation, since a fresh install generates a new key	Apple
Quota increase	Not available. Apple offers no mechanism to request one	Apple
Rollout	Onboard gradually, a percentage of users at a time, to avoid hitting undocumented limits	Apple guidance and practice

Two notes on the `attestKey()` row, because the precise wording matters. The rule is **once per key**, and "once per

installation" is the operational consequence rather than the rule itself. A reinstall, a device migration or a restore legitimately produces a new key — and per Chapter 10.3, Apple's own 2026 guidance is explicit that you should **not** treat a new key from an existing user as fraud.

When to check integrity

Because quotas are undocumented on iOS and metered on Android, you cannot attest everything. This table is the discriminator, and it applies to both platforms.

User action	Check integrity?	Why
Payment or checkout	Yes	Financial transaction; highest risk
Add a payment method	Yes	Financial data at risk
Change password or email	Yes	The classic account-takeover vector
Delete account	Yes	Irreversible
Initial login	Yes	This is where the session is established
Product browsing	No	Low risk, high volume, would burn quota for nothing
Search	No	No sensitive data involved
View cart	No	No state change
View profile	No	Already behind authentication

That last row is the most useful discriminator in the table, and it generalises: **if an action is already gated by something you trust, attesting it again buys little**. Attestation earns its cost at the boundary where trust is established or where state changes irreversibly, not on every screen behind the login.

10.6 The shared limit

Both platforms' attestation proves something about the **app and device**, not about the **user**. Apple states directly that the mechanism cannot guarantee your app is running on a system that is not jailbroken, which could enable an attacker to circumvent your checks.

Attestation belongs in a risk score, alongside authentication, account history, behaviour and velocity. Chapter 13 is about building that.

Chapter 11: Biometrics, done so they cannot be bypassed

Biometric authentication is the control where the gap between a correct and an incorrect implementation is largest, and where the incorrect implementation looks identical to the user.

11.1 Decorative versus real

There are two ways to write biometric authentication. They look identical to the user. One of them stops an attacker and one does not.

The decorative implementation. You call the biometric prompt. It calls back with success. You branch on that boolean and unlock the feature.

An attacker with Frida hooks the callback and returns success without any biometric ever occurring. Total bypass, achievable in minutes, and MASWE-0020 exists precisely to name it.

The real implementation. You tie the biometric to a **cryptographic operation**. The prompt unlocks a Keystore or Keychain key that requires user authentication, and you use that key to sign or decrypt something your server verifies.

Now hooking the callback accomplishes nothing. The attacker can make your UI say "authenticated," but they cannot produce the signature, because the key is in hardware and hardware will not release it without a genuine authentication that Gatekeeper (or the Secure Enclave) vouches for.

That is the whole chapter, mechanically. Everything else is detail.

On Android, that means `BiometricPrompt` with a `CryptoObject` backed by a key created with `setUserAuthenticationRequired(true)`. MASTG-BEST-0036 names this directly: use cryptographic binding for biometric authentication.

On iOS, it means `LocalAuthentication` with a Secure Enclave key whose `SecAccessControl` requires biometry — and

remember that access control lists are evaluated **inside** the Secure Enclave and released only when their constraints are met.

11.2 Biometric classes, and why Class 3 matters

Android grades biometric modalities by strength. **Class 3 (BIOMETRIC_STRONG)** is the tier strong enough to gate cryptographic operations — meaning it is the only tier that can back a `CryptoObject`. **Class 2 (BIOMETRIC_WEAK)** accepts weaker modalities and cannot.

For anything financial, require Class 3. MASTG-BEST-0031 says the same. If you accept Class 2 for a payment confirmation, you have accepted a modality the platform itself considers insufficient to protect a key.

11.3 The enrollment problem nobody thinks about

Here is a scenario worth sitting with. An attacker steals an unlocked phone, or a phone whose passcode they have observed. They go into settings and **add their own fingerprint**. Your app's biometric prompt now accepts them.

The defence is to invalidate biometric-bound keys when the enrolled set changes. On Android that is `setInvalidatedByBiometricEnrollment(true)`. On iOS it is `kSecAccessControlBiometryCurrentSet` rather than `biometryAny` — the "current set" wording is the whole point, and Apple documents that access can be limited by specifying that Face ID or Touch ID enrolment has not changed since the item was added, precisely to prevent an attacker adding their own fingerprint.

MASWE-0022 covers this and it is one of the most commonly missed weaknesses in the catalogue. MASTG-BEST-0037 is the corresponding practice.

The cost is real: a user who legitimately adds a fingerprint has to re-enrol in your app. For a banking app that is the correct trade. Decide it deliberately rather than by omission.

11.4 Fallbacks, and the tension inside them

Users lose fingerprints to bandages, cuts and cold weather; faces to sunglasses, masks and bad light. An app with no fallback locks people out of their own accounts.

But MASWE-0021 names the opposite failure: **fallback to non-biometric credentials allowed for sensitive transactions**. If your "use passcode instead" path bypasses the cryptographic binding, you have reintroduced the bypass through the back door.

Resolve it by making the fallback *equivalent in strength*, not weaker. Device credential fallback that still unlocks a hardware key is fine. Server-side step-up authentication is fine. A fallback that just sets `authenticated = true` is not a fallback; it is the vulnerability with a friendlier label.

Also require **explicit user confirmation** for high-value actions (MASTG-BEST-0038). Passive face recognition that authorises a transfer the moment the user glances at their phone is a usability decision with a security consequence.

11.5 Where to spend the friction

Require biometrics for: payments and transfers, adding a payee, changing a password or email address, viewing full card details, disabling security features, and high-value account changes.

Do **not** require them for: app launch on every cold start, browsing, reading, or anything a user does twenty times a day.

Friction spent where it does not buy security is friction users route around — by disabling the feature, by choosing a weaker option, or by abandoning the flow. A security control that users switch off protects nothing. This is not a UX concession; it is a security argument.

11.6 How to implement it

The whole point of §11.1: bind the prompt to a cryptographic operation so hooking the callback gains an attacker nothing.

Android: BiometricPrompt with a CryptoObject

```

// 1. A key that only exists while the user is authenticated, and dies if
//    the enrolled biometric set changes (Chapter 11.3).
val spec = KeyGenParameterSpec.Builder(
    "biometric_key",
    KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT
)
    .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
    .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
    .setUserAuthenticationRequired(true)
    .setInvalidatedByBiometricEnrollment(true)    // the stolen-phone defence
    .build()

// 2. Require Class 3. Class 2 cannot back a CryptoObject at all.
val promptInfo = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Confirm transfer")
    .setSubtitle("Authenticate to authorise 5,000 EGP to Ahmed")
    .setAllowedAuthenticators(BiometricManager.Authenticators.BIOMETRIC_STRONG)
    .setNegativeButtonText("Cancel")
    .build()

// 3. Hand the Cipher to the prompt. This is the binding.
val cipher = Cipher.getInstance("AES/GCM/NoPadding").apply {
    init(Cipher.ENCRYPT_MODE, getBiometricKey())
}

biometricPrompt.authenticate(promptInfo, BiometricPrompt.CryptoObject(cipher))

```

In the callback, **use the crypto object** — do not merely observe that authentication succeeded:

```

override fun onAuthenticationSucceeded(result: BiometricPrompt.AuthenticationResult) {
    // The cipher is now unlocked. Produce something the server verifies.
    val payload = result.cryptoObject?.cipher?.doFinal(transferRequest.toByteArray())
    api.authoriseTransfer(payload)    // server checks it; the client decides nothing
}

```

The wrong version, which is extremely common:

```

// DON'T: no CryptoObject, no server verification. One Frida hook and this returns success.
override fun onAuthenticationSucceeded(result: BiometricPrompt.AuthenticationResult) {
    isAuthenticated = true
    performTransfer()
}

```

Check availability before you offer it, and handle the invalidation case, because a user who re-enrolled will hit `KeyPermanentlyInvalidatedException` on first use — that is the control working, and your code must treat it as "re-register", not as a crash:

```

when (BiometricManager.from(context)
    .canAuthenticate(BiometricManager.Authenticators.BIOMETRIC_STRONG)) {
    BiometricManager.BIOMETRIC_SUCCESS -> showBiometricPrompt()
    BiometricManager.BIOMETRIC_ERROR_NONE_ENROLLED -> offerEnrollment()
    else -> fallBackToServerSideStepUp()    // equivalent strength – Chapter 11.4
}

```

iOS: LocalAuthentication bound to a Secure Enclave key

Reuse the `SecAccessControl` from §6.5 with `.biometryCurrentSet`, then sign with it. Because the access control is evaluated **inside** the Secure Enclave, the biometric prompt appears as part of using the key — there is no boolean for an attacker to intercept:

```

// Using the key triggers the biometric prompt; the signature is the proof.
var error: Unmanaged<CFError>?
guard let signature = SecKeyCreateSignature(
    privateKey,                                // Secure Enclave, biometryCurrentSet
    .ecdsaSignatureMessageX962SHA256,
    transferRequestData as CFData,
    &error
) as Data? else {
    // User cancelled, or biometrics changed and the key is gone.
    return fallBackToServerSideStepUp()
}
try await api.authoriseTransfer(request: transferRequestData, signature: signature)

```

That is the shape to aim for on both platforms: **the server verifies a signature it could only receive if a genuine authentication happened.**

Verify it

The whole point of §11.1 is that a hooked callback should gain an attacker nothing. Test exactly that.

Hook the success callback and see what happens:

```
// frida -U -f com.example.app -l bypass.js
Java.perform(function () {
  var Prompt = Java.use("androidx.biometric.BiometricPrompt$AuthenticationCallback");
  Prompt.onAuthenticationSucceeded.implementation = function (result) {
    console.log("[*] forcing success");
    return this.onAuthenticationSucceeded(result);
  };
});
```

Pass: the UI may report success, but **the protected action still fails**, because no signature was produced and your server rejected the request. **Fail:** the transfer completes — which means you branched on a boolean and §11.1's decorative implementation is what you shipped.

Then test enrollment invalidation. Add a new fingerprint in device settings and use the feature again. **Pass:** `KeyPermanentlyInvalidatedException`, handled as a re-registration prompt. **Fail:** it still works, meaning `setInvalidatedByBiometricEnrollment` is not set and MASWE-0022 applies.

Relevant tests: MASTG-TEST-0018, 0326, 0328.

Chapter 12: The backend as the only real arbiter

Every control so far degrades gracefully when it fails. This one does not. Get it wrong and nothing else in the book matters.

12.1 The non-negotiables

Six rules. Unlike most of this book, these do not have a "it depends on your threat model" caveat.

Validate everything server-side. Prices, quantities, entitlements, balances, permissions, discounts, limits. The client proposes; the server decides. If your API accepts a price from the client, you do not have a pricing bug, you have a free store.

Never trust a client-supplied identity claim. Derive the acting user from the session token, never from a field in the request body. `{"userId": 12345}` is a suggestion.

Bind tokens so they are not portable. A bearer token that works from anywhere is a token worth stealing. Bind it to a device key, an attestation key, or a session, so that a stolen token is useless off the device it was issued to. This single measure devalues a whole category of attack.

Rate limit per account and per device, not only per IP. IP-based limiting is defeated by any residential proxy pool, which costs an attacker very little.

Log security decisions with enough context to investigate later, and never log the secrets themselves. "Denied step-up for user X on device Y at time Z because attestation returned basic integrity" is useful. The token that was presented is a liability.

12.2 Attestation as a signal, not a switch

The design that scales is a **risk score**. Attestation contributes to it; so do account age, device history, behavioural velocity, geography, transaction size, and time since last authentication.

The reason to build it this way is not elegance. A single binary signal is a single point of failure: when it is defeated, and Chapter 5 showed that attestation can be, you have nothing. A score degrades. Lose one input and the others still discriminate.

It also gives you proportionate responses rather than a binary. Instead of allow-or-block, you get: allow silently; allow with step-up authentication; allow with a lower limit; allow but flag for review; queue for manual review; refuse. Most fraud is better handled by the middle options, because they impose cost on the attacker without punishing the false positive.

12.3 The Backend-for-Frontend pattern

There is an architectural decision that makes most of this chapter easier, and it deserves naming because teams often reach it late.

A **Backend-for-Frontend**, or **BFF**, is a thin backend layer sitting between your mobile app and your core services. It owns every sensitive secret and makes every trust decision.

Use it when your app talks to third-party APIs, processes payments, or handles authentication. In practice that is most apps.

How it works:

1. The app authenticates to the BFF using short-lived tokens held in hardware-backed storage (Chapter 6).
2. The **BFF holds all third-party API keys**, payment credentials and service secrets. None of them ship in the app.
3. The BFF validates every request: authentication, authorization, input validation, rate limiting, integrity verification.
4. The BFF proxies to upstream services, attaching the real credentials **server-side**.

What that buys you, and each of these solves a problem that appears elsewhere in this book:

Benefit	The problem it solves
Third-party keys never leave the server	Chapter 15.2's "restricted" class has nowhere safe to live on a client. The BFF is that place
Secrets rotate without an app release	Otherwise rotating a key means a store review and users who never update
Trust decisions happen where an attacker cannot reach them	Chapter 1's principle, expressed as architecture rather than discipline
One place for logging, auditing and rate limiting	Chapter 12.1's rate limiting and Chapter 24's incident logging get a single home

The honest cost. A BFF is another service to build, deploy, monitor and secure. It adds a network hop and a latency budget. For a small app talking only to your own well-designed API, it is over-engineering. The question to ask is whether you currently have any third-party key in the client that you wish you did not — if the answer is yes, you already need a BFF and are paying for its absence in a different currency.

12.4 One test for any design

Ask this of every authenticated endpoint you own:

| If an attacker replays a valid request from a different device, what stops them?

If the answer is "nothing," you have found your next piece of work, and it is more important than anything on the client. Token binding, attestation assertions, and nonce or counter checks are the three mechanisms that answer that question properly.

12.5 Risk scoring, in practice

Chapter 12.2 argues for a risk score rather than a binary. Here is what one actually looks like, so the argument is concrete.

```

data class SecuritySignals(
    val attestationPassed: Boolean,
    val deviceIntegrityLevel: String,    // STRONG / DEVICE / BASIC / NONE
    val rootDetected: Boolean,
    val hookingDetected: Boolean,
    val debuggerDetected: Boolean,
    val emulatorDetected: Boolean
)

data class UserHistory(
    val accountAgeDays: Int,
    val knownDevice: Boolean,
    val failedAuthLast24h: Int,
    val transactionsLast1h: Int
)

/** Higher is riskier. Tune the weights against your own fraud data, not mine. */
fun calculateRiskScore(
    signals: SecuritySignals,
    history: UserHistory,
    amountMinorUnits: Long
): Int {
    var score = 0

    // Client-reported signals: inputs, never verdicts (Chapter 14.2)
    if (!signals.attestationPassed) score += 40
    when (signals.deviceIntegrityLevel) {
        "NONE" -> score += 30
        "BASIC" -> score += 20
        "DEVICE" -> score += 0
        "STRONG" -> score -= 5 // a small bonus, not a requirement
    }
    if (signals.hookingDetected) score += 35
    if (signals.debuggerDetected) score += 25
    if (signals.rootDetected) score += 10 // deliberately low: often legitimate
    if (signals.emulatorDetected) score += 15

    // Account and behavioural signals: these are yours, so trust them more
    if (history.accountAgeDays < 7) score += 20
    if (!history.knownDevice) score += 15
    score += history.failedAuthLast24h * 5
    if (history.transactionsLast1h > 5) score += 15
    if (amountMinorUnits > HIGH_VALUE) score += 20

    return score.coerceIn(0, 100)
}

```

Then map the score to a **graduated response**, which is the entire point:

```

when (calculateRiskScore(signals, history, amount)) {
    in 0..24 -> allow()
    in 25..49 -> allowWithStepUp() // biometric confirmation (Chapter 11)
    in 50..74 -> allowWithReducedLimit() // or queue for review
    else -> denyAndFlagForReview()
}

```

Three design points worth taking from this.

Root detection carries a low weight on purpose. Rooted devices are common and frequently legitimate. Weighting root heavily is how you build a system that punishes power users and misses actual fraud.

Server-derived signals outweigh client-reported ones. Account age and failed-authentication counts come from your own database, so an attacker cannot forge them. Everything in `SecuritySignals` arrives from a client you do not control, which is why no single one of them is decisive.

Tune the weights against real outcomes. The numbers above are a starting shape, not a recommendation (*reasoned*). A score that has never been compared against confirmed fraud is a guess with arithmetic on top.

Impossible travel

One cheap, high-signal backend check: did this account just move faster than physically possible?

```

data class GeoLocation(val latitude: Double, val longitude: Double, val timestamp: Long)

fun detectImpossibleTravel(current: GeoLocation, previous: GeoLocation?): Boolean {
    if (previous == null) return false

    val distanceKm = haversineDistance(
        current.latitude, current.longitude,
        previous.latitude, previous.longitude
    )
    val hours = (current.timestamp - previous.timestamp) / 3_600_000.0

    // 900 km/h ≈ commercial flight. Anything faster is not travel.
    return distanceKm > hours * 900
}

private fun haversineDistance(lat1: Double, lon1: Double, lat2: Double, lon2: Double): Double {
    val r = 6371.0 // Earth radius, km
    val dLat = Math.toRadians(lat2 - lat1)
    val dLon = Math.toRadians(lon2 - lon1)
    val a = sin(dLat / 2).pow(2) +
        cos(Math.toRadians(lat1)) * cos(Math.toRadians(lat2)) * sin(dLon / 2).pow(2)
    return r * 2 * atan2(sqrt(a), sqrt(1 - a))
}

```

Prefer backend-side IP geolocation over device GPS for this. It needs no permission, it is harder for a user to spoof casually, and per Chapter 18.2 it keeps you out of a location-data declaration you would otherwise have to make and justify. Use coarse location if you must use the device at all.

Part 5: Resilience

Chapter 13: How reverse engineering actually works

You cannot reason about anti-tampering without knowing what you are defending against, at the level of the actual commands. This chapter is the attacker's workflow — which is also your testing workflow, because Chapter 22 asks you to run it against your own app.

13.1 Static analysis

Static analysis reads the app without running it.

Obtain the artifact. Pull the APK from a device or the store; extract the IPA. MASTG covers this as MASTG-TECH-0003.

Explore the package. The manifest tells an attacker your components, permissions, exported activities, deep link schemes, and whether you are debuggable. This is a map of your attack surface, and it is the first thing anyone reads.

Retrieve strings (MASTG-TECH-0071). This is where hardcoded keys, internal hostnames, and debug flags fall out. Thirty seconds of work.

Decompile. JADX turns Dalvik bytecode into readable Java. For Android you can also disassemble to smali, which is what you edit if you intend to repack. On iOS, radare2 and similar tools disassemble the Mach-O binary; Swift and Objective-C metadata gives up class and method names generously.

Read the security logic. Root detection, pinning configuration, license checks — all of it is now visible, and its structure tells the attacker exactly what to hook.

13.2 Dynamic analysis

Dynamic analysis runs the app and interferes.

Get a shell and reach the data directory (MASTG-TECH-0001, MASTG-TECH-0008). `/data/data/<package>/` holds your preferences, databases and caches. This is where "we encrypt sensitive data" gets verified or falsified in about a minute.

Monitor logs (MASTG-TECH-0009). Release builds that still log are found here.

Set up an interception proxy (MASTG-TECH-0011). Route traffic through mitmproxy or Burp with the attacker's CA installed.

Bypass certificate pinning (MASTG-TECH-0012). This is the step that tells you whether your pinning is real. `objection` will do it with one command for common implementations; a Frida script handles the rest. **Every engineer who has shipped pinning should have done this to their own app**, because until you have, you do not know whether your pinning survives contact.

Hook methods with Frida (MASTG-TECH-0051). Replace any function's behaviour at runtime. Return `false` from your root check. Return `true` from your biometric callback. Log the arguments to `Cipher.doFinal` and watch your own plaintext go past.

Repackage (MASTG-TECH-0004). Modify smali, rebuild, re-sign with the attacker's key, install. This is the cloning attack, and it is why signature verification exists.

The tools an attacker uses, and what each one gives them

You will see these names in every discussion of mobile security. Knowing what each actually does makes the rest of this chapter concrete.

Tool	What it does	What it gives an attacker
JADX	Decompiles an APK to readable Java-like source	Your logic, endpoints, and where your security decisions live
Frida	Runtime instrumentation — hooks and replaces any method in a live process	The ability to change what your code does without changing the file
objection	Frida-powered toolkit with prebuilt commands	Storage inspection and pinning bypass without writing a script
Xposed / LSPosed	System-wide hooking framework on rooted Android	Persistent modification across apps
Cydia Substrate / Cycript	The iOS equivalents of runtime hooking and inspection	The same capability on jailbroken iOS
radare2 / Hopper	Disassemblers for native and Mach-O binaries	Your native code, including logic you moved there for protection
apktool	Decodes and rebuilds an APK	Repackaging: change something, re-sign, redistribute
mitmproxy / Burp Suite	Intercepting proxies	All your traffic, in cleartext, on a device they control
strings	Prints printable strings in a binary	Every constant you embedded, in about thirty seconds

Two observations worth carrying. Every one of these is **free and documented**, so none of your protection can assume scarcity of tooling. And you should be running all of them against your own app (Chapter 22) — an attacker's toolkit and your test toolkit are the same toolkit.

13.3 The three demos that make the argument

The MASTG ships runnable demos, and one sequence is worth more than any amount of prose on this topic. Read them in order:

MASTG-DEMO-0106 — extracting sensitive data from `Cipher.doFinal` via Frida hooking. The attack works.

MASTG-DEMO-0107 — detecting Frida hooks and terminating the application in response. The defence works.

MASTG-DEMO-0108 — bypassing that Frida detection in `/proc/self/maps` and extracting the data anyway. The defence is defeated.

Protect, detect, bypass. That progression, published by the standard itself, is the strongest available argument for the position in the next chapter. Two more demos make the same point about obfuscation: MASTG-DEMO-0132 and 0133 show root detection defeated when it is protected only by identifier renaming, in the Java and native layers respectively.

- <https://mas.owasp.org/MASTG/techniques/>

Chapter 14: Obfuscation and tamper detection — the honest chapter

This is where security writing usually oversells, so let me be direct: **everything in this chapter can be bypassed by a competent attacker with device access**. You are buying time and raising cost, not achieving prevention.

That is not an argument against doing it. Time and cost are real currency against three of the four adversaries in Chapter 1. It is an argument against *believing your own marketing*, and against designing as though these controls hold.

14.1 The cheap things, which you should simply do

Start here. Every item below costs close to nothing and removes the easiest tier of attacker.

Enable R8 with obfuscation and resource shrinking on Android; **strip symbols** on iOS. Free, automatic, and it removes the trivial-effort tier of attacker.

Remove all logging from release builds, and verify by inspecting the artifact rather than trusting the build flag. MASTG-BEST-0002 and 0022.

Ensure `debuggable` is false and no debug configuration shipped. MASTG-BEST-0007. A debuggable production build is not a hardening gap, it is an open door.

Disable WebView debugging in release. MASTG-BEST-0008.

Prevent screenshots on sensitive screens. `FLAG_SECURE`, `setRecentsScreenshotEnabled`, `setSecure` for SurfaceViews, and

`SecureFlagPolicy.SecureOn` for Compose dialogs — that last one catches people who did everything else right.

Use up-to-date signing schemes (MASTG-BEST-0006) and a current `minSdkVersion` (MASTG-BEST-0010). Old signature schemes and old API levels both reopen closed attack classes.

Set sensible session timeouts, shorter for financial flows.

None of this is clever. All of it is worth more than most clever things.

14.2 Detection: report, never block

Detect what you can — root and jailbreak indicators, hooking frameworks (Frida, Xposed, Substrate, Cypript), debuggers, emulators and virtual devices, repackaging via your own signature check, and storage or code integrity failures.

Then **send those findings to your backend as risk inputs, and let the session continue**.

Three reasons, and they compound.

Local blocks are trivially removed. The attacker is already running your code under a debugger with the ability to hook any method. Your `if (isRooted()) exit()` is one hook away from `if (false) exit()`. You have built a speed bump and paid for a wall. MASTG-DEMO-0108 is this argument, demonstrated.

You generate false positives against real people. Developers, power users, users of custom ROMs in regions where that is normal, users of legitimate accessibility tooling. A hard local block converts these into support tickets and one-star reviews, for no security gain.

You destroy your own intelligence. A backend that receives "hooking framework detected on this session" can require step-up authentication, cap transaction limits, flag the account, and correlate across sessions to find the actual attacker. An app that shows a "device not supported" dialog has taught the attacker exactly which check to remove next, and told you nothing.

MASTG-BEST-0029 frames this correctly as implementing resilience and **RASP signals** — signals being the operative word.

14.3 Where obfuscation genuinely helps

Two places, and it is worth knowing them so you spend effort well.

Raising the floor. Against automated tooling and the opportunist, obfuscation plus resource shrinking is often enough to make your app not worth the trouble relative to the next one.

Protecting detection logic itself. If your root detection is legible, it is removable. MASTG-DEMO-0132 shows detection logic in the Java layer defeated when protected only by identifier renaming; MASTG-DEMO-0133 shows the same in native code with insufficient obfuscation. Moving security-critical logic to native code with real obfuscation raises the cost of removing it — MASTG-TEST-0368 and 0369 test exactly this.

What obfuscation does not do is protect a secret. An obfuscated key is a key. Chapter 1's principle still governs.

14.4 Detection, in code

Chapter 14.2 argues for detecting and reporting rather than blocking. This section is what the detecting part looks like, so the advice is actionable rather than abstract.

The shape to aim for on both platforms (*reasoned*): collect named signals, return them as data, send them to your backend. No branching on the result locally.

Android: root indicators

```

object RootDetector {

    fun signals(): List<String> = buildList {
        if (checkSuBinaries()) add("SU_BINARY")
        if (checkRootPackages()) add("ROOT_PACKAGE")
        if (checkBuildTags()) add("TEST_KEYS")
        if (checkRwPaths()) add("SYSTEM_WRITABLE")
    }

    private fun checkSuBinaries() = listOf(
        "/system/bin/su", "/system/xbin/su", "/sbin/su",
        "/system/app/Superuser.apk", "/data/local/bin/su"
    ).any { java.io.File(it).exists() }

    private fun checkRootPackages() = listOf(
        "com.topjohnwu.magisk", "eu.chainfire.supersu", "com.koushikdutta.superuser"
    ).any { pkg -> runCatching { context.packageManager.getPackageInfo(pkg, 0) }.isSuccess }

    private fun checkBuildTags() =
        android.os.Build.TAGS?.contains("test-keys") == true

    private fun checkRwPaths() = listOf("/system", "/system/bin", "/vendor")
        .any { java.io.File(it).canWrite() }
}

```

Android: debugger and emulator

```

object DebugDetector {

    data class Result(val isDebugged: Boolean, val isEmulator: Boolean, val signals: List<String>)

    fun detect(): Result {
        val signals = mutableListOf<String>()

        if (android.os.Debug.isDebuggerConnected()) signals += "DEBUGGER_CONNECTED"
        if (tracerAttached()) signals += "TRACER_ATTACHED"
        if (debuggableBuild()) signals += "DEBUGGABLE_BUILD"
        if (emulatorFingerprint()) signals += "EMULATOR_FINGERPRINT"

        return Result(
            isDebugged = signals.any { it.startsWith("DEBUGGER") || it.startsWith("TRACER") },
            isEmulator = signals.any { it.startsWith("EMULATOR") },
            signals = signals
        )
    }

    /** A non-zero TracerPid means something is attached to this process. */
    private fun tracerAttached(): Boolean = runCatching {
        java.io.File("/proc/self/status").readLines()
            .firstOrNull { it.startsWith("TracerPid:") }
            ?.substringAfter(":")?.trim()?.toIntOrNull() ?: 0
    }.getOrDefault(0) != 0

    private fun debuggableBuild(): Boolean =
        (appInfo.flags and android.content.pm.ApplicationInfo.FLAG_DEBUGGABLE) != 0

    private fun emulatorFingerprint(): Boolean = with(android.os.Build) {
        FINGERPRINT.startsWith("generic") || FINGERPRINT.contains("vbox") ||
        MODEL.contains("Emulator") || MODEL.contains("Android SDK built for") ||
        HARDWARE.contains("goldfish") || HARDWARE.contains("ranchu") ||
        PRODUCT == "sdk" || PRODUCT == "google_sdk"
    }
}

```

TracerPid is the most useful of these. A debugger or Frida attaching shows up there, and it needs no permission to read.

iOS: jailbreak and hooking indicators

```
enum JailbreakDetector {

    static func signals() -> [String] {
        var out: [String] = []
        if suspiciousPathsExist() { out.append("SUSPICIOUS_PATH") }
        if canOpenCydiaScheme() { out.append("URL_SCHEME") }
        if canWriteOutsideSandbox() { out.append("SANDBOX_WRITABLE") }
        if canFork() { out.append("FORK_ALLOWED") }
        if suspiciousDylibLoaded() { out.append("HOOKING_DYLIB") }
        return out
    }

    private static func suspiciousPathsExist() -> Bool {
        ["/Applications/Cydia.app", "/bin/bash", "/usr/sbin/sshd",
         "/etc/apt", "/private/var/lib/apt/", "/usr/lib/libsubstrate.dylib"]
        .contains { FileManager.default.fileExists(atPath: $0) }
    }

    private static func canOpenCydiaScheme() -> Bool {
        guard let url = URL(string: "cydia://package/com.example") else { return false }
        return UIApplication.shared.canOpenURL(url)
    }

    /// A sandboxed app cannot write outside its container.
    private static func canWriteOutsideSandbox() -> Bool {
        let path = "/private/\(UUID().uuidString)"
        guard (try? "test".write(toFile: path, atomically: true, encoding: .utf8)) != nil
        else { return false }
        try? FileManager.default.removeItem(atPath: path)
        return true
    }

    /// fork() succeeding indicates the sandbox is not being enforced.
    private static func canFork() -> Bool {
        let pid = fork()
        if pid >= 0 { if pid > 0 { kill(pid, SIGTERM) }; return true }
        return false
    }

    /// Substrate, Frida's gadget and similar arrive as loaded images.
    private static func suspiciousDylibLoaded() -> Bool {
        for i in 0..

```

Reporting, which is the whole point

```
suspend fun reportSecuritySignals() {
    val payload = SecurityReport(
        rootSignals = RootDetector.signals(),
        debugSignals = DebugDetector.detect().signals,
        appSignature = currentSignatureHash(),
        timestamp = System.currentTimeMillis()
    )
    runCatching { api.reportSignals(payload) } // never block the user on this call
}
```

Three rules for this code. **Send, never branch:** the backend decides (Chapter 14.2). **Fail silently:** a detection report that crashes the app or blocks a flow has converted a security feature into an outage. And **do not name your detection classes** `RootDetector` in a release build: §15.8 shows the R8 configuration that stops you handing an attacker a search term.

Verify it

Detection code has two failure modes, and you should test for both.

False negatives. Run the app on a rooted device or emulator with Magisk and confirm your signals actually fire. Reaching your backend with an empty signal list from an obviously rooted device means the detection is not running, not that the device is clean.

False positives, which matter more. Run on a clean, stock, unrooted device. **Pass:** no signals. **Fail:** any signal at all — you are about to apply risk scoring to ordinary users. Test on at least one non-Google-Play device and one custom ROM if your market includes them.

Then confirm you are not blocking. Grep your own codebase:

```
grep -rn "isRooted\\|isJailbroken\\|isDebugged" --include=*.kt --include=*.swift .
```

Every hit should feed a report, not a branch that exits or refuses. A hit inside an `if` that ends a flow is §14.2's mistake.

And confirm the names are gone from release. Decompile your release build and search for `RootDetector`. Finding it means your R8 configuration is keeping what §15.8 tells you to let it obfuscate.

14.5 The hygiene controls, in code

Five small controls that appear in every real assessment. Each is a few lines and each maps to a MASWE weakness.

Screenshot and recents protection

MASWE-0038. On Android, apply the flag per screen rather than app-wide, so you are not degrading the recents experience everywhere:

```
class PaymentActivity : AppCompatActivity() {
    override fun onResume() {
        super.onResume()
        window.setFlags(
            WindowManager.LayoutParams.FLAG_SECURE,
            WindowManager.LayoutParams.FLAG_SECURE
        )
    }
    override fun onPause() {
        super.onPause()
        window.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
    }
}
```

For Compose dialogs, remember `SecureFlagPolicy.SecureOn` — `FLAG_SECURE` on the activity does not cover a dialog in its own window.

On iOS there is no direct equivalent, so there are two practical approaches. Detect that a capture happened and react:

```
NotificationCenter.default.addObserver(
    forName: UIScreen.capturedDidChangeNotification, object: nil, queue: .main
) { _ in
    if UIScreen.main.isCaptured { self.blurSensitiveContent() }
}
```

Or use the long-standing `isSecureTextEntry` trick: place your sensitive view inside the layer of a secure `UITextField`, which the OS excludes from screenshots. It works, and it is a workaround rather than an API, so test it after each iOS release.

Secure logging

MASWE-0005. The robust pattern is a logging tree that drops everything below error in release:

```
// Application.onCreate
if (BuildConfig.DEBUG) {
    Timber.plant(Timber.DebugTree())
} else {
    Timber.plant(object : Timber.Tree() {
        override fun log(priority: Int, tag: String?, message: String, t: Throwable?) {
            if (priority >= Log.ERROR) crashReporter.log(message) // no verbose/debug/info
        }
    })
}
```

This beats scattering `if (BuildConfig.DEBUG)` at call sites, because it cannot be forgotten at one of them. Pair it with the `-assumenosideeffects` rule in §15.8, and verify by inspecting the artifact.

Clipboard

MASWE-0030. Both platforms have a flag for this and almost nobody sets it.

```
val clip = ClipData.newPlainText("", sensitiveValue).apply {
    description.extras = PersistableBundle().apply {
        putBoolean(ClipDescription.EXTRA_IS_SENSITIVE, true) // API 33+
    }
}
clipboard.setPrimaryClip(clip)
```

```
UIPasteboard.general.setItems(
    [[UTType.plainText.identifier: sensitiveValue]],
    options: [
        .localOnly: true, // do not sync to other devices
        .expirationDate: Date().addingTimeInterval(60) // clears itself
    ]
)
```

The iOS `localOnly` option matters more than it looks: without it, a copied value can travel to the user's other devices through Universal Clipboard.

Session timeout

MASWE-0024 . Time the background period, not just user inactivity, because a phone in a pocket is not an active session:

```
class SessionTimeoutManager(  
    private val timeoutMinutes: Int,  
    private val onTimeout: () -> Unit  
) : DefaultLifecycleObserver {  
  
    private var backgroundedAt: Long? = null  
  
    override fun onStop(owner: LifecycleOwner) {  
        backgroundedAt = System.currentTimeMillis()  
    }  
  
    override fun onStart(owner: LifecycleOwner) {  
        val since = backgroundedAt ?: return  
        if (System.currentTimeMillis() - since > timeoutMinutes * 60_000L) onTimeout()  
        backgroundedAt = null  
    }  
}
```

Reasonable starting points (*reasoned*): 15 minutes for banking and payments, 30 for health and enterprise, hours or none for content and social. And "timeout" must mean invalidating the session **server-side** and clearing local data, per Chapter 0.3 — not just showing a login screen over cached content.

Encrypting the database

Following §6.7, the Android implementation with Room:

```
// build.gradle.kts  
// implementation("net.zetetic:sqlcipher-android:4.x")  
// implementation("androidx.sqlite:sqlite-ktx:2.x")  
  
val passphrase = SQLiteDatabase.getBytes(getOrCreateDatabaseKey()) // from Keystore, §6.6  
val factory = SupportOpenHelperFactory(passphrase)  
  
val db = Room.databaseBuilder(context, AppDatabase::class.java, "app.db")  
    .openHelperFactory(factory)  
    .build()
```

`getOrCreateDatabaseKey()` must come from a Keystore-backed key. A hardcoded passphrase turns database encryption into an obfuscation exercise, and `strings` will find it.

Part 6: The build and release pipeline

Chapter 15: Why your pipeline is now the target

In 2026 your CI is a more attractive target than your app, and the reasoning is simple. It holds your signing keys, your store credentials and your cloud access. It runs code on every commit. And an attacker who owns it does not need to reverse engineer anything — they ship malicious code to your users **under your own signature**, through your own release channel.

Treat workflow files as production code with production privileges. That framing decides everything below.

15.1 What actually happened

These are not hypotheticals, and each one maps to a control.

Shai-Hulud (late 2025) was a self-replicating npm campaign (*reported*). Compromised machines were registered as self-hosted runners named `SHAIHULUD`, and stolen credentials were used to create public repositories that served as exfiltration buckets. It prompted GitHub's revamped npm security roadmap and accelerated the roll-out of npm trusted publishing.

Lesson: monitor runner registrations and new-repository creation.

tj-actions/changed-files and **aquasecurity/trivy-action** (the latter on 19 March 2026) were compromised by **force-pushing tags** to point at malicious code, which then exfiltrated secrets from CI runners. *Lesson: tags are mutable. Pin to commit SHAs.*

Mini Shai-Hulud / TanStack (May 2026) is the one to study. An attacker poisoned the pnpm store cache through a `pull_request_target` workflow. **Eight hours later**, the legitimate release workflow consumed the poisoned cache, extracted OIDC tokens from runner memory, and published 84 malicious versions across 42 packages (*reported*). *Lesson: caches are a trust boundary. And see §15.5, because this one has a sting in it.*

GhostAction (September 2025) hijacked 327 GitHub accounts and stole 3,325 secrets (*reported*) using malicious workflows disguised as "GitHub Actions Security" repositories. *Lesson: workflow changes need review by someone who reads them.*

Megalodon (May 2026) pushed 5,718 malicious commits to 5,561 repositories in a single six-hour window (*reported*) using throwaway bot accounts. *Lesson: automated attacks outrun manual review.*

GitHub's own 2026 Actions security roadmap describes the pattern plainly: attackers abuse untrusted code execution, mutable dependencies, over-permissioned tokens, secrets, and weak runner observability. Those five categories are the agenda for this part.

- <https://github.blog/security/supply-chain-security/securing-the-open-source-supply-chain-across-github/>

15.2 Secrets: classify, then protect

Not every string is a secret, and treating them all identically makes people careless with the ones that matter.

Not secret at all: public API base URLs, feature flags, public keys. Source control is fine. Encrypting these teaches your team that the secret store is bureaucracy.

Client identifiers: OAuth client IDs, Firebase configuration. These necessarily ship in the app. Protect them **server-side** by restricting use to your package name and signing certificate — the identifier is public, the authorisation is not.

Restricted: any third-party key with quota or cost attached, including model provider keys. **Proxy through your backend.** Never in the app. If it is in the app, it is public, and now metered.

Never in the client under any circumstances: signing keys, service account credentials, database credentials, store API keys. CI secret store only, injected at build time.

The operational rules follow from Chapter 2's finding that 64% of 2022's leaked secrets are still valid:

When a secret leaks, rotate first, then clean history. Not the other way round. Deleting the commit does not un-leak the credential; assume harvest within minutes of the push. Then purge history, add scanning, and check access logs for use of the old credential.

Scan every commit, blocking: GitGuardian, `trufflehog`, or GitHub Push Protection.

Prefer short-lived credentials to stored ones wherever the platform allows — see §15.3.

Audit your outputs, not just your inputs. Run `strings` on your release artifact. This is the same experiment as Chapter 1 and it belongs in your release gate.

And two 2026-specific realities: with AI-assisted commits leaking at roughly double the human baseline, commit-time scanning has become load-bearing rather than optional for any team running agentic workflows; and if your tooling uses MCP servers, audit those configuration files specifically, because 24,008 secrets leaked through them and most scanning setups were not built with them in mind.

15.3 Hardening the workflow

Six controls, in the order I would apply them.

Pin third-party actions to full-length commit SHAs. Tags are mutable; SHAs are not. Keep the version in a trailing comment so the file stays readable:

```
- uses: actions/checkout@08c6903cd8c0fde910a37f88322edcfb5dd907a8 # v5.0.1
```

Automate with `pinact` or StepSecurity's `secure-repo`. Keep SHAs current with Dependabot grouped updates, and add a **cooldown window**, seven days being a common choice, so you do not adopt a compromised release within an hour of publication. Then enforce it in repository settings, which can require actions to be pinned to a full-length SHA.

Deny token permissions by default. Set `permissions: {}` at workflow level and grant the minimum per job.

Repositories created before February 2023 default to read-write — check yours rather than assuming.

```
permissions: {}           # workflow level: deny everything
jobs:
  build:
    permissions:
      contents: read      # job level: only what this job needs
```

Replace static cloud credentials with OIDC. A stored key is valid indefinitely and portable the moment it leaks. An OIDC token is minted per workflow run, scoped to that job, and expires in minutes — so a compromised workflow yields a credential that is nearly worthless by the time it is exfiltrated. AWS, Azure and GCP all support it natively.

For package publishing, use **trusted publishing** and scope it to the **exact workflow filename and branch** rather than the whole repository, paired with branch protection requiring pull requests and blocking force pushes.

Treat all external input as hostile. PR titles, issue bodies, branch names and commit messages are attacker-controlled and get interpolated straight into shell steps. That is script injection. Pass untrusted values through `env:` and quote them; never inline `${{ github.event.* }}` into a `run:` block.

Be especially careful with `pull_request_target`, which runs with repository secrets in scope against code from a fork. It is the TanStack vector. If you need it, never check out or execute fork code in the same job that holds secrets.

Put workflow files under review. Add a `CODEOWNERS` entry pointing `.github/workflows/` at someone who understands the implications, and require `CODEOWNERS` approval through branch protection. Without it, anyone with write access can change what runs with your secrets. Be suspicious of external pull requests that modify pinned versions.

Harden and watch the runners. Prefer ephemeral runners so nothing persists between jobs. Never expose self-hosted runners to untrusted pull requests. Alert on **new runner registrations** and **new public repository creation** in your organisation — both would have surfaced Shai-Hulud early.

15.4 Dependencies and build inputs

Your build consumes more than your own code. Each input below is something an attacker can influence if you leave it unpinned.

Lock your dependencies. Commit `gradle.lockfile` via Gradle dependency locking, plus `Gemfile.lock` and `Package.resolved` for tooling. An unlocked build silently resolves a different version a month later and nobody notices.

Enable Gradle dependency verification with checksums, so a swapped artifact fails the build rather than shipping.

Generate an SBOM per build and scan it. The OWASP Dependency-Check Gradle plugin is free and flags known CVEs before release. MASTG-TEST-0274 and 0275 test exactly this, on both platforms.

Audit what your dependencies request. Check permissions in the *merged* manifest, not just the ones you declared. A UI animation library requesting `INTERNET` and storage access deserves a question.

Treat caches as inputs. A poisoned cache is a supply-chain attack with no dependency change to review. Scope cache keys tightly, and never let a fork-triggered workflow write to a cache your release workflow reads.

Pin your toolchain, not just your libraries: Gradle wrapper with checksum verification, a fixed JDK, `.ruby-version`, and fastlane pinned in `Gemfile.lock` so CI and every laptop run identical versions.

15.5 Provenance, and the uncomfortable thing about it

Build provenance — a signed statement about how an artifact was produced, expressed in frameworks such as SLSA — is genuinely valuable. It is also routinely oversold, and the TanStack incident is the proof.

The malicious packages published in that attack carried valid, signed SLSA Build Level 3 provenance. Verification tools reported them clean.

The reason is precise and worth internalising: **provenance attests to the process, not to the cleanliness of the inputs**. The build genuinely ran in the declared environment, from the declared repository, through the declared workflow. It just consumed a poisoned cache. Everything the attestation claimed was true, and the artifact was still malicious.

So when someone in a design review says "we have provenance, our supply chain is verified," the correct response is that provenance closes the *substitution* attack, meaning someone publishing an artifact that did not come from your build. It does nothing about a compromised input. You still need §15.4's controls on what enters the build.

15.6 Signing keys and release integrity

This is the part that decides whether an attacker can ship code as you.

Android: use Play App Signing with a split key model. Google holds the app signing key; you hold an **upload key** used only to upload. The property that matters: if your upload key is stolen, the attacker **cannot re-sign your app** — Google re-signs with the key they hold, and you register a new upload key and move on. Without this, a stolen signing key means an attacker ships code under your identity, which is close to unrecoverable. This is the strongest single argument for the split key model and it is why the recommendation is unambiguous.

Keystore handling in CI. Store it base64-encoded in the CI secret store, or GPG-encrypted in the repository with the passphrase in the secret store. Decode at build time into a path you delete afterwards. Never commit `key.properties` or a `gradle.properties` containing passwords — a leaked `gradle.properties` with a signing password has the same blast radius as a leaked server key.

And **fail the build when release signing is missing**, rather than silently producing a debug-signed artifact that the store rejects on upload. That check catches an entire class of embarrassing release incidents.

iOS: use `fastlane match`. Certificates and profiles live encrypted in a private repository, get pulled into a temporary keychain for the build, and are cleaned up automatically. Authenticate to the store with an **App Store Connect API key** rather than an Apple ID: no interactive 2FA, and it can be scoped and revoked.

Both platforms. Give store service accounts the **minimum role** — a key that can publish to production when it only needs the internal track is standing risk for no benefit. Restrict who can trigger release workflows and require manual approval for the production track. And keep releases **auditable**: tag the commit, record which workflow run produced the artifact, retain the build log. When someone asks "what exactly is in production," you want an answer rather than an investigation.

15.7 Verify what actually shipped

A perfect pipeline can still produce a bad artifact. Check the output, in an automated release-gate job rather than when someone remembers:

Run `strings` and a decompiler on the **release** build. Confirm `debuggable` is false, logging is stripped, and no debug network configuration survived. Confirm the artifact is signed with the expected key. Diff the dependency tree against the previous release and ask about anything new.

- <https://corgea.com/learn/github-actions-security-checklist>
- <https://www.buildmvpfast.com/blog/github-actions-supply-chain-security-hardening-guide-2026>
- <https://www.aikido.dev/blog/checklist-github-actions>

15.8 Build configuration, in code

The pipeline chapter covers secrets and supply chain. This is the build configuration itself, which is where several cheap protections live.

Android: R8 rules with security in mind

```
#### Verify it
```

```
Build configuration is the area where the source and the artifact most often disagree, so check the artifact.
```

What actually shipped?

```
unzip -p app-release.apk classes.dex | strings | grep -iE "api[_-]?key|secret|password|bearer" aapt dump badging app-release.apk | grep -E "debuggable|application-label" apksigner verify --print-certs app-release.apk
```

****Pass:**** no credentials in the output, no ``debuggable`` flag, and a signing certificate that matches your expected fingerprint. ****Fail on any of the three**** is a release-blocking finding, not a backlog item.

For iOS:

```
strings MyApp.app/MyApp | grep -iE "api[_-]?key|secret|https://.*internal" otool -l MyApp.app/MyApp | grep -A3 LC_CODE_SIGNATURE
```

Then confirm logging is gone by running the release build and watching ``adb logcat`` or Console during a login flow. A build flag that says logging is disabled is not evidence; an empty log is.

Put all of this in a release-gate job (§15.7). A check that runs when someone remembers is not a control.

```
# proguard-rules.pro

# Keep serialization models – reflection needs the names
-keepclassmembers class * {
    @kotlinx.serialization.Serializable *;
}

# Keep Tink, which uses reflection internally
-keep class com.google.crypto.tink.** { *; }

# DO NOT add keep rules for your security detection classes.
# Letting R8 obfuscate them is the point: an attacker grepping the
# decompiled output for "RootDetector" should find nothing.

# Strip logging from release builds
-assumenosideeffects class android.util.Log {
    public static int v(...);
    public static int d(...);
    public static int i(...);
}
```

That middle comment is the part worth internalising (*reasoned*), and it is the opposite of most keep-rule advice. Every keep rule you add is a name you have handed to an attacker. Keep what reflection genuinely requires, and nothing else — especially not the classes whose job is to be hard to find.

Android: separate debug and release properly

```
android {
    buildTypes {
        debug {
            isDebuggable = true
            buildConfigField("Boolean", "ENABLE_PINNING_BYPASS", "true")
            buildConfigField("Boolean", "ENABLE_SECURITY_CHECKS", "false")
        }
        release {
            isDebuggable = false
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
            buildConfigField("Boolean", "ENABLE_PINNING_BYPASS", "false")
            buildConfigField("Boolean", "ENABLE_SECURITY_CHECKS", "true")
        }
    }
}
```

The `BuildConfig` flag pattern is how you get a debuggable app without shipping a debuggable app. But note the risk it introduces: **a flag that disables pinning is a flag an attacker would love to flip**. Gate it on `BuildConfig.DEBUG` as well, so the bypass path is not even compiled into release, and verify by inspecting the release artifact (Chapter 15.7) rather than trusting the configuration.

iOS: strip the release build

Set these in Build Settings for the Release configuration:

Setting	Value
STRIP_INSTALLED_PRODUCT	YES
STRIP_STYLE	all
DEPLOYMENT_POSTPROCESSING	YES
GCC_GENERATE_DEBUGGING_SYMBOLS	NO
SWIFT_OPTIMIZATION_LEVEL	-O

Keep the generated dSYM files somewhere you can retrieve them — you need them to symbolicate crash reports, and stripping the binary is precisely what makes them necessary.

Injecting secrets at build time

Android, reading from a file CI writes and `.gitignore` excludes:

```
// build.gradle.kts
val localProps = java.util.Properties().apply {
    val f = rootProject.file("local.properties")
    if (f.exists()) f.inputStream().use { load(it) }
}

android {
    defaultConfig {
        buildConfigField("String", "API_BASE_URL",
            "\"${localProps["API_BASE_URL"] ?: System.getenv("API_BASE_URL")}\"")
    }
}
```

```
# .github/workflows/release.yml - CI writes the file, then builds
- name: Write local.properties
  run: |
    echo "API_BASE_URL=${{ secrets.API_BASE_URL }}" >> local.properties
    echo "MAPS_API_KEY=${{ secrets.MAPS_API_KEY }}" >> local.properties
- run: ./gradlew assembleRelease
```

iOS, using an `.xcconfig` that CI generates and source control ignores:

```
// Secrets.xcconfig (gitignored; CI writes it)
API_BASE_URL = api.example.com
```

Reference it from `Info.plist` as `$(API_BASE_URL)` and read it via `Bundle.main.infoDictionary`.

Remember what this does and does not achieve. Build-time injection keeps secrets out of source control — which is Chapter 15.2's actual goal. It does **not** keep them out of the app: anything in `BuildConfig` or `Info.plist` is in the binary and recoverable with `strings`. Only the classification in Chapter 15.2 decides what may ship at all, and anything genuinely sensitive belongs behind the BFF (Chapter 12.3).

Part 7: Platform surfaces and new frontiers

This part covers the components where most real findings live: WebViews, IPC and deep links, permissions. Plus secure coding, and the two newest surfaces: AI features and Kotlin Multiplatform.

Chapter 16: WebViews — the most under-secured component in mobile

A **WebView** is a browser engine embedded in your app. It's convenient: you reuse web content, ship changes without a release, and support both platforms with one implementation.

It is also, consistently, the weakest part of most mobile apps. The reason is structural rather than accidental: a WebView takes untrusted content from the internet and runs it *inside your app's security context*. Everything your app can do, that content is one mistake away from doing.

MASVS-PLATFORM-2 covers WebViews, and eight MASWE weaknesses relate to them. That ratio tells you something.

16.1 Why a WebView is different from a browser

When Chrome loads a hostile page, the page is confined by the browser's sandbox and the same-origin policy. It can't read your files or call your code.

When *your WebView* loads a hostile page, the page sits inside an app that has your permissions, your storage, your session tokens and, if you added one, a bridge into your native code. The confinement you're relying on is whatever you configured, and the defaults are convenient rather than safe.

16.2 The JavaScript bridge, and why it's the sharpest edge

A **JavaScript bridge** lets web content call native functions. On Android that's `addJavascriptInterface`; on iOS it's `WKScriptMessageHandler`.

Consider what you've built when you add one. Any JavaScript running in that WebView can now call your native method. If the WebView ever loads content you don't fully control — a third-party page, a URL from a deep link, an ad, a page that includes a compromised script — that content calls your native code.

MASWE-0033 is "sensitive native functionality exposed in WebViews," and MASTG-TEST-0033 and 0334 test for it on Android, MASTG-TEST-0078 and 0376 on iOS.

How to do it safely, if you must:

Expose the **narrowest possible surface**. Not a general `invoke(methodName, args)` gateway — one function that does one thing, validating its inputs. Every parameter arriving from JavaScript is untrusted input, exactly like a network response.

Never expose anything that reads files, returns credentials, or performs a privileged action without independent authorisation. "The web page will only call this correctly" is not a security property.

On iOS, prefer `WKScriptMessageHandlerWithReply` for returning data to JavaScript rather than calling `evaluateJavaScript` — MASTG-BEST-0062. And use `WKContentWorld` **isolation** when you inject scripts for DOM inspection, so page JavaScript can't see or interfere with your script (MASTG-BEST-0061, tested by MASTG-TEST-0379).

Prefer **origin-scoped messaging** over legacy bridges (MASTG-BEST-0035).

16.3 File access: the setting nobody remembers changing

WebViews can be configured to read local files, and on Android they historically could read content providers too. Combine that with a page that can be influenced by an attacker and you have local file exfiltration from inside your own app.

The relevant settings on Android are `setAllowFileAccess`, `setAllowFileAccessFromFileURLs`, `setAllowUniversalAccessFromFileURLs` and `setAllowContentAccess`. Turn off what you don't need. MASWE-0034 covers this; MASTG-TEST-0250 through 0253 test both the static configuration and the runtime behaviour.

On iOS, watch for relaxed file origin policies (MASTG-TEST-0335 and 0336) and overly broad file read access, MASTG-TEST-0333.

If you need to load local content, do it properly: MASTG-BEST-0011 and 0033 describe securely loading file content in a WebView. On Android that generally means a content-URI-based loader rather than `file://` with universal access.

16.4 Loading untrusted content

MASWE-0035 is "WebViews loading untrusted content," and the failure usually arrives through a chain: a deep link carries a URL parameter, your code passes it to `loadUrl`, and now an attacker chooses what runs in your WebView.

Validate any URL before loading it, against an allowlist of hosts you control — not a blocklist, and not a `contains("mydomain.com")` check, which `https://evil.com/?x=mydomain.com` passes.

Handle navigation. Intercept it (`shouldOverrideUrlLoading` on Android, `WKNavigationDelegate` on iOS) and decide whether each navigation is permitted, rather than letting the page take your WebView anywhere. MASTG-TEST-0398 and 0400 cover this.

Never ignore TLS errors. `onReceivedSslError` calling `proceed()` disables certificate validation for that WebView entirely. It gets added to silence a development warning and it survives to production because nothing visibly breaks. MASTG-TEST-0284, MASTG-DEMO-0056. On iOS the equivalent is a `WKNavigationDelegate` that accepts any server certificate — MASTG-TEST-0397, MASTG-DEMO-0155. If you take one thing from this chapter, take this one.

Disable JavaScript when you don't need it (MASTG-BEST-0012), and content provider access on Android (MASTG-BEST-0013).

16.5 Sensitive UI inside a WebView

If you render a password field in a WebView, the value lives in the page DOM, where any injected script can read it, and where your app's own `evaluateJavaScript` calls might write it (MASTG-TEST-0378, 0380).

The guidance is direct: **render sensitive UI as native views over the WebView** (MASTG-BEST-0059) and **use native views for sensitive text entry** (MASTG-BEST-0060). This is also, not coincidentally, why RFC 8252 tells you to run OAuth in the system browser rather than a WebView.

16.6 Cleanup and hygiene

WebViews keep cookies, local storage, caches and session data. On logout, clear them — MASTG-BEST-0028, MASTG-TEST-0320, MASTG-DEMO-0082. This is part of the MASWE-0024 problem from Chapter 0.3.

Disable WebView debugging in release builds (MASTG-BEST-0008). A debuggable WebView lets anyone with the device inspect your page and call your bridge from a console.

On iOS, migrate off `UIWebView` if you still have it. It's deprecated and less secure than `WKWebView`, which runs content out-of-process. MASTG-BEST-0032, MASTG-TEST-0331.

16.7 A short checklist you can hold in your head

Does this WebView load only content I control? Does it have a bridge, and if so is the surface minimal and validated? Is file access off? Is JavaScript needed? Do I intercept navigation? Do I ever proceed past a TLS error? Is sensitive input native rather than in the page? Do I clear state on logout? Is debugging off in release?

If you can answer those nine questions about every WebView in your app, you're ahead of most teams.

16.8 How to implement it

Android: a WebView configured defensively

```
webView.settings.apply {
    javaScriptEnabled = false           // turn on only if the page needs it
    allowFileAccess = false             // §16.3
    allowContentAccess = false
    @Suppress("DEPRECATION")
    allowFileAccessFromFileURLs = false
    @Suppress("DEPRECATION")
    allowUniversalAccessFromFileURLs = false
    domStorageEnabled = false           // on only if needed
}

if (!BuildConfig.DEBUG) {
    WebView.setWebContentsDebuggingEnabled(false) // MASTG-BEST-0008
}
```

Allowlist navigation, and never proceed past a TLS error:

```
webView.webViewClient = object : WebViewClient() {

    private val allowedHosts = setOf("app.example.com", "help.example.com")

    override fun shouldOverrideUrlLoading(
        view: WebView, request: WebResourceRequest
    ): Boolean {
        val host = request.url.host
        // Exact host match. NOT `host.contains("example.com")`, which
        // https://evil.com/?x=example.com would pass.
        return if (host != null && host in allowedHosts) {
            false // we handle it: let the WebView load it
        } else {
            true // blocked; optionally open in the system browser
        }
    }

    override fun onReceivedSslError(
        view: WebView, handler: SslErrorHandler, error: SslError
    ) {
        handler.cancel() // NEVER handler.proceed(). See §16.4.
    }
}
```

If you need a bridge, make the surface narrow and validate every argument:

```
class NarrowBridge(private val onRedeem: (String) -> Unit) {
    @JavascriptInterface
    fun redeemVoucher(code: String) {
        // Untrusted input, exactly like a network response.
        if (!code.matches(Regex("[A-Z0-9]{8}$"))) return
        onRedeem(code) // server authorises; the bridge does not
    }
}
webView.addJavascriptInterface(NarrowBridge(::redeem), "AppBridge")
```

The wrong version:

```
// DON'T: a general gateway into native code. Any script in the WebView owns your app.
@JavascriptInterface
fun invoke(method: String, args: String): String = reflectivelyCall(method, args)
```

iOS: WKWebView with reply-based messaging

```
let config = WKWebViewConfiguration()
config.defaultWebpagePreferences.allowsContentJavaScript = false // enable only if needed

// Reply-based handler, per MASTG-BEST-0062 – avoids evaluateJavaScript round-trips
config.userContentController.addScriptMessageHandler(
    self, contentWorld: .defaultClient, name: "appBridge" // isolated world
)

let webView = WKWebView(frame: .zero, configuration: config)
webView.navigationDelegate = self
```

```
extension MyController: WKNavigationDelegate {
    func webView(_ webView: WKWebView,
        didReceive challenge: URLAuthenticationChallenge,
        completionHandler: @escaping (URLSession.AuthChallengeDisposition,
            URLCredential?) -> Void) {
        // Do not accept arbitrary server trust here – MASTG-TEST-0397.
        completionHandler(.performDefaultHandling, nil)
    }

    func webView(_ webView: WKWebView,
        decidePolicyFor navigationAction: WKNavigationAction,
        decisionHandler: @escaping (WKNavigationActionPolicy) -> Void) {
        let allowed = ["app.example.com", "help.example.com"]
        guard let host = navigationAction.request.url?.host, allowed.contains(host) else {
            decisionHandler(.cancel)
            return
        }
        decisionHandler(.allow)
    }
}
```

And on logout, clear the state (§16.6):

```
let store = WKWebsiteDataStore.default()
store.removeData(ofTypes: WKWebsiteDataStore.allWebsiteDataTypes(),
    modifiedSince: .distantPast) { }
```

Verify it

```
# Is WebView debugging still enabled in release?
adb shell am start -a android.intent.action.VIEW -d "chrome://inspect"
```

With the release build running, `chrome://inspect` on a connected machine should show **no inspectable WebView**. If it does, `setWebContentsDebuggingEnabled(false)` is not taking effect and anyone with the device can call your bridge from a console.

Test the allowlist with a hostile URL. Send the app a deep link carrying `https://evil.example.com/?x=app.example.com` and confirm the WebView refuses it. A `contains()` check passes this and an exact-host check rejects it — which is the difference §16.4 is about.

Test the TLS error path. Point the WebView at a host with an invalid certificate. **Pass:** the load fails. **Fail:** the page renders, meaning something calls `proceed()`.

Confirm bridge scope. In a debug build, from the WebView console: `Object.keys(AppBridge)`. What comes back is your entire native attack surface. If it lists more than you intended, narrow it.

Relevant tests: `MASTG-TEST-0284`, `0031`, `0033`, `0334`.

Chapter 17: IPC, deep links and app components

IPC, or inter-process communication, is how apps talk to each other and to the system. It's genuinely useful and it's a door into your app, which means every door needs a lock and a check on who's knocking.

`MASVS-PLATFORM-1` covers this, and twelve MASWE weaknesses live here.

17.1 Android app components, and the word "exported"

Android apps are built from four component types: **activities** (screens), **services** (background work), **broadcast receivers** (event listeners) and **content providers** (data interfaces).

Each is either **exported**, meaning other apps can start or query it, or not. This single attribute decides whether a component is internal plumbing or public API.

The rule: **export nothing you don't need to**. Declare `android:exported="false"` explicitly. Since Android 12 you must declare it explicitly when you have an intent filter, which was a welcome change because the old implicit default caught many people out.

When you *do* export something, protect it: require a permission, verify the caller, and validate every input as untrusted.

`MASTG-TEST-0364`, `0365` and `0366` test for exported and unprotected activities, services and receivers respectively; `MASTG-BEST-0052` describes restricting access.

The failure looks like this: an activity that displays account details, exported because it needed a deep link, launched directly by a malicious app, skipping your login screen. Or an exported service that performs a transfer, invoked by any app on the device. `MASWE-0018` — "lack of authentication or authorization on app components."

17.2 Intents: explicit versus implicit

An **explicit intent** names its destination component. An **implicit intent** describes an action and lets the system choose who handles it.

Implicit intents are the risk. For internal communication, always use explicit intents (`MASTG-BEST-0056`), because an implicit intent for internal work can be intercepted by another app that registered for the same action — `MASTG-TEST-0372`, `MASTG-DEMO-0136`, `MASTG-DEMO-0140`.

Two related failures. **Sensitive data in implicit intent extras** (`MASTG-TEST-0374`, `MASTG-DEMO-0138`): whatever you attached is now readable by whichever app received it. And **not validating what comes back** (`MASTG-TEST-0375`): if you fire an implicit intent to pick a file and a malicious app returns a crafted path, you now read a file of the attacker's choosing (`MASTG-DEMO-0139`, `MASTG-DEMO-0141`).

`MASTG-BEST-0057`, sanitize data coming from external components, is the general fix. Treat every intent extra like a query parameter from the internet.

17.3 PendingIntent: the one that looks harmless

A `PendingIntent` wraps an intent and hands another app the right to fire it **as you**, with your identity and permissions.

If you create a mutable `PendingIntent` with an implicit base intent, the receiving app can fill in the blanks, changing the destination, the action or the data, and the resulting call executes with your app's privileges.

Always use immutable `PendingIntent`s with explicit intents. `FLAG_IMMUTABLE` exists for this, `MASTG-BEST-0063` says it, and

MASTG-TEST-0030 and 0381 test for it.

17.4 Content providers

A **content provider** exposes structured data through a URI interface, usually backed by a database. Two failure modes.

SQL injection. If you build a query by concatenating a caller-supplied selection string, a caller controls your SQL. Use parameterised queries — MASTG-BEST-0039, MASTG-TEST-0339, MASTG-DEMO-0102.

Path traversal via FileProvider. FileProvider shares files by granting URI permissions. Configure the shared paths too broadly and a caller can request a URI outside the intended directory, exfiltrating private files. MASTG-DEMO-0122 and 0123 demonstrate both the oversharing configuration and the exfiltration. Scope your `file_paths.xml` to the narrowest directory that works, and validate requested filenames.

17.5 Deep links, and the verification that makes them safe

A **deep link** opens your app at a specific screen from a URL. There are two kinds and the difference is the whole security story.

A **custom URI scheme** (`myapp://profile/123`) can be registered by **any** app. Multiple apps can claim the same scheme, and which one wins is not defined. So a malicious app can register your scheme and intercept links intended for you — which, as Chapter 0.3 noted, is the code interception attack against OAuth.

A **verified https link**, called **App Links** on Android and **Universal Links** on iOS, is cryptographically tied to a domain you control. Android uses `android:autoVerify="true"` plus a Digital Asset Links file hosted on your domain; iOS uses an `apple-app-site-association` file. The OS fetches that file and confirms your app is authorised. No other app can claim your verified link.

Use verified https links. MASTG-BEST-0070 covers `autoVerify` and Digital Asset Links; MASTG-TEST-0393 tests for unverified App Links; MASWE-0029 is "insecure deep links."

Then validate the contents anyway. A verified link proves the *link* came to the right app; it says nothing about the *parameters*. Every deep link parameter is attacker-supplied — someone can send your user a crafted link. So:

Validate every parameter (MASTG-BEST-0071, 0072; MASTG-TEST-0394, 0395, 0370).

Never let a deep link parameter become a URL you load in a WebView without allowlisting (Chapter 16.4).

Never let a deep link bypass authentication. If `myapp://transfer?to=X&amount=Y` works while logged in, an attacker sends that link and the user taps it. Sensitive actions require confirmation and step-up authentication regardless of how the screen was reached.

On iOS, also validate the **source application** where the API allows (MASTG-BEST-0055, MASTG-TEST-0371).

17.6 The other channels people forget

The clipboard. Shared across apps. On iOS the general pasteboard has historically synced across a user's devices. Don't copy secrets to it; if you must, restrict to the local device, set an expiry, and clear after use. MASWE-0030, MASTG-TEST-0276 through 0280.

App extensions and App Groups (iOS). Extensions run in separate processes and share data with your app through App Groups. Data in a shared container is readable by every member. MASWE-0031, MASTG-TEST-0388, MASTG-BEST-0068.

Accessibility services. A malicious accessibility service can read your screen content and inject events. You can't fully prevent this, but you can avoid putting secrets in accessible text where it isn't needed. MASWE-0040.

Overlay attacks. Another app draws over yours, so the user thinks they're tapping your button and they're tapping something else — tapjacking. Protect sensitive screens with `setHideOverlayWindows` or `filterTouchesWhenObscured`. MASWE-0039, MASTG-BEST-0040, MASTG-TEST-0035, MASTG-DEMO-0103.

Notifications. Lock-screen notifications are visible without unlocking. A notification containing a one-time code defeats the code. MASWE-0037, MASTG-BEST-0027.

Custom keyboards (iOS). A third-party keyboard with full access sees what users type. For sensitive fields, keep input on the system keyboard (MASTG-BEST-0069) or restrict custom keyboards app-wide (MASTG-TEST-0389).

17.7 The pattern behind all of it

Every item in this chapter is the same idea in a different costume: **a channel exists, another app can use it, and your code trusted what came through.**

So the question to ask of any component, link or share point is: *if a hostile app on this device used this, what could it do?* Answer that for each door and you've covered the category.

17.8 How to implement it

Components: closed by default

```
<!-- AndroidManifest.xml -->
<activity android:name=".InternalActivity" android:exported="false" />

<!-- Exported because it handles a deep link – so it must verify state itself -->
<activity android:name=".TransferActivity" android:exported="true">
    <intent-filter android:autoVerify="true"> <!-- App Links, §17.5 -->
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="https" android:host="app.example.com" />
    </intent-filter>
</activity>
```

The Digital Asset Links file that makes `autoVerify` real, served at `https://app.example.com/.well-known/assetlinks.json`:

```
[{
  "relation": ["delegate_permission/common.handle_all_urls"],
  "target": {
    "namespace": "android_app",
    "package_name": "com.example.app",
    "sha256_cert_fingerprints": ["AA:BB:CC:..."]
  }
}]
```

Then treat the link's contents as hostile anyway:

```
// TransferActivity.onCreate
val amount = intent.data?.getQueryParameter("amount")?.toLongOrNull()
val payee = intent.data?.getQueryParameter("to")

// A verified link proves it reached the right app. It proves nothing about who sent it.
if (amount == null || amount <= 0 || payee == null || !isKnownPayee(payee)) {
    finish(); return
}
if (!session.isAuthenticated) { routeToLogin(); return }

// Never auto-execute. Confirm, then step up (Chapter 11).
showConfirmation(amount, payee, onConfirm = { requireBiometricThenTransfer(amount, payee) })
```

PendingIntent: immutable, explicit

```
val intent = Intent(context, ResultReceiverActivity::class.java) // explicit
val pendingIntent = PendingIntent.getActivity(
    context, 0, intent,
    PendingIntent.FLAG_IMMUTABLE or PendingIntent.FLAG_UPDATE_CURRENT // §17.3
)
```

`FLAG_MUTABLE` with an implicit base intent is the combination to avoid — it lets the recipient redirect the call and have it execute with your identity.

Content provider: parameterised, always

```
// Safe: the selection is a fixed string; values are bound separately.
override fun query(
    uri: Uri, projection: Array<String>?, selection: String?,
    selectionArgs: Array<String>?, sortOrder: String?
): Cursor = db.query(
    "orders",
    projection,
    "user_id = ?", // our own selection, not the caller's
    arrayOf(currentUserId), // bound value
    null, null, sortOrder
)
```

The wrong version:

```
// DON'T: the caller controls your SQL. MASTG-TEST-0339.
db.rawQuery("SELECT * FROM orders WHERE $selection", null)
```

Verify it

Exported components are testable from the shell, which is exactly how an attacker enumerates them.

```
# What is exported?
aapt dump xmltree app-release.apk AndroidManifest.xml | grep -B4 "exported.*true"

# Can a component be launched directly, skipping your login flow?
adb shell am start -n com.example.app/.TransferActivity \
-a android.intent.action.VIEW -d "https://app.example.com/transfer?to=x&amount=1"
```

Pass: the activity either refuses to start or lands on your login screen. **Fail:** you reach the transfer screen with an authenticated session absent, which is MASWE-0018.

Verify App Links are actually verified, not merely declared:

```
adb shell pm get-app-links com.example.app
```

Pass: the domain shows `verified`. **Fail:** `legacy_failure` or `none`, which means your `assetlinks.json` is missing, malformed, or served from the wrong path — and your links can be claimed by another app.

Probe the content provider:

```
adb shell content query --uri content://com.example.app.provider/orders \
--where "1=1 OR id=1"
```

Pass: permission denied, or only the calling user's rows. **Fail:** other users' data, which is MASTG-TEST-0339.

Chapter 18: Permissions and privacy

Privacy joined MASVS as a full category in v2.1.0, with four controls and thirteen weaknesses. It arrived because regulation arrived, and because "we collect this because it was easy" stopped being acceptable.

This chapter is short and practical. Most of it is about asking for less.

18.1 Permissions: ask for less, later, and explain why

Five habits. The theme running through all of them is asking for less.

Request only what the feature needs (MASVS-PRIVACY-1, MASWE-0066). Every permission is attack surface, a privacy commitment, and a conversion cost — users decline, and some uninstall.

Check the merged manifest, not just yours. Third-party SDKs declare their own permissions, and those merge into your app. An analytics library that quietly adds location access has made a privacy declaration on your behalf. MASTG-TEST-0254 looks for dangerous permissions; 0255 for permissions that aren't minimised.

Request at the point of use, with a rationale. Asking for camera access when the user taps "take photo" gets granted far more often than asking at first launch, and it's honest. MASTG-TEST-0256 tests for a missing rationale.

Handle denial gracefully. A feature that breaks or nags when a permission is declined is a bug. Users may also revoke later, and Android auto-resets unused permissions (MASTG-TEST-0257).

On iOS, minimise entitlements too (MASTG-BEST-0051, MASTG-TEST-0362) and write accurate purpose strings — the text users see when you ask. A vague or wrong purpose string is both an App Review problem and a MASTG-TEST-0360 finding.

18.2 Identifiers and tracking

MASWE-0068 is "incorrect use of identifiers for user tracking," and the substance is: **use the right identifier for the job, and the most privacy-preserving one that works.**

Use a **resettable, purpose-scoped** identifier where you can — Android's advertising ID or app set ID, iOS's `identifierForVendor`. Don't use hardware identifiers to build a persistent profile the user can't reset, and don't attempt to fingerprint a device to defeat the reset the platform gave them. Both platforms treat that as a policy violation, not just a privacy concern.

Where you don't need identity, **anonymise or pseudonymise** (MASWE-0067). Aggregate counts rather than per-user events. Truncate what you don't need at full precision — coarse location rather than exact coordinates, if coarse answers your question.

18.3 Transparency and control

MASVS-PRIVACY-3 asks you to be transparent about collection and use; MASVS-PRIVACY-4 asks you to give users control.

Practically: your **store data declaration** must match what your app actually does (MASWE-0073), including what your SDKs do. Your **tracking domain declarations** must be complete (MASWE-0074). Your privacy policy must be accurate and reachable (MASWE-0072). Consent must be unambiguous and not pre-checked (MASWE-0078, MASWE-0071), and users need a real

way to see and delete their data (MASWE-0076, MASWE-0077).

The engineering consequence people miss: **you can't declare accurately unless you know what leaves the device.** MASTG-TEST-0206 is "undeclared PII in network traffic capture," and MASTG-DEMO-0009 shows how to detect it. Run that on your own app and compare what you see against your declaration. Teams are routinely surprised, usually by an SDK.

18.4 Third-party SDKs are your responsibility

An analytics, ads, crash or attribution SDK runs inside your app with your permissions, and its network calls are your data flows. MASWE-0069 and MASTG-TEST-0318/0319 cover SDK APIs known to handle sensitive user data.

Before adding one, ask: what does it collect, where does it send it, can I configure it down, and does my declaration cover it? After adding one, capture its traffic and confirm the answers. MASTG-DEMO-0081 demonstrates sensitive user data reaching an analytics SDK, caught with Frida.

Catching an SDK in the act

Chapter 18.4 says to verify what your SDKs collect. On Android 11 and later there is a way to have the OS tell you, which is more reliable than reading documentation:

```
// Application.onCreate, debug builds
val appOps = getSystemService(AppOpsManager::class.java)
appOps.setOnOpNotedCallback(mainExecutor, object : AppOpsManager.OnOpNotedCallback() {
    override fun onNoted(op: SyncNotedAppOp) {
        Log.d("DataAudit", "sensitive access: ${op.op} from ${Thread.currentThread().stackTrace.joinToString()}")
    }
    override fun onSelfNoted(op: SyncNotedAppOp) = onNoted(op)
    override fun onAsyncNoted(op: AsyncNotedAppOp) {
        Log.d("DataAudit", "async sensitive access: ${op.op} - ${op.message}")
    }
})
```

Every access to a permission-guarded resource is now logged with a stack trace. Run your app through its flows and read the output: you will find out which SDK reads location, which reads the clipboard, and when. That output is also what makes your store data declaration accurate rather than aspirational (Chapter 18.3).

Pair it with MASTG-TEST-0206 — capture your own network traffic and compare what leaves the device against what you declared.

18.5 The regulatory frame, briefly

You don't need to be a lawyer, but you should know the shape:

GDPR (EU) and similar laws require a lawful basis for processing, data minimisation, purpose limitation, user rights of access and deletion, and breach notification. Fines reach a percentage of global revenue.

Store policies are enforced faster than laws. Both Apple and Google will reject or remove an app whose data declarations don't match its behaviour — and that lands on you in days, not years.

Sector rules — HIPAA for health data in the US, PCI-DSS for payment card data, plus local banking regulation — add specific technical requirements. If you work in one of these, someone in your organisation owns those requirements; find them before you design, not after.

The practical takeaway: collect less, declare accurately, verify what actually leaves the device, and talk to whoever owns compliance *before* you ship a feature that changes any of it.

Chapter 19: Input validation and unsafe handling

MASVS-CODE-4 asks you to validate and sanitize all untrusted inputs. That sentence is easy to nod at and hard to apply, because the difficult part is recognising what counts as untrusted.

19.1 What "untrusted input" actually includes

Everything on this list is attacker-controllable, and each one is a real finding class:

Network responses — **including from your own backend**, which could be compromised or proxied. Deep link and URL parameters. Intent extras and IPC payloads. Content read through a content provider. Files the user picks or another app hands you. QR codes and NFC tags. Clipboard contents. WebView messages. Push notification payloads. Model output from an LLM (Chapter 20). And user-typed text, which is the one everybody remembers.

If you only validate the last item, you've validated the least likely attack.

19.2 Validate positively, at the boundary

Four habits that turn validation from a chore into something that actually holds.

Allowlist, don't blocklist. Define what's acceptable and reject everything else. Blocklists fail because you have to think of every bad case and the attacker only has to find one you missed.

Validate at the trust boundary, as soon as data enters — not deep inside business logic where three call sites bypass the check.

Validate type, range, length, format and meaning. A quantity should be a positive integer within a sane bound. A file path should resolve inside the directory you expect. A URL should have a host on your allowlist.

Fail closed. When validation fails, reject; don't proceed with a "sensible default."

19.3 Injection: the shape of the bug

Injection has a reputation for being many different bugs. It is really one bug wearing different clothes, which is good news, because one fix generalises.

Injection happens when data gets interpreted as code or as a command. Same shape everywhere: you built a string, and part of the string came from an attacker.

SQL injection — build a query with concatenation and the input changes the query. Fix: parameterised queries, always. Room's `@Query` with bound parameters is safe; string concatenation in a raw query is not. `MASTG-TEST-0339`.

Path traversal — a filename containing `../` escapes the directory you meant. Fix: canonicalise the path and confirm it's still inside your intended directory before opening it.

Command injection — passing input into a shell. Fix: don't invoke shells with user data; if you must, use argument arrays rather than a command string.

Cross-site scripting in WebViews — input rendered into HTML executes as JavaScript. Fix: escape on output, and see Chapter 16.

Log injection — input containing newlines forges log entries. Fix: sanitise before logging, and don't log user content you don't need.

The generalisation worth internalising: **separate code from data**. Every injection fix is a version of that. Parameterised queries separate SQL from values. Argument arrays separate the command from its arguments. Escaping separates markup from text.

19.4 Deserialization

This one is less famous than injection and can be more severe, because it can hand an attacker code execution rather than data.

Deserialization turns bytes back into objects. If those bytes are attacker-controlled, deserialization can construct objects you never intended and, with the right classes present, execute code.

`MASWE-0050` covers unsafe handling of untrusted data; `MASTG-TEST-0337` and `0386` test for untrusted deserialization on Android and iOS; `MASTG-BEST-0064` says use safe APIs.

Practically: prefer explicit, schema-driven formats such as JSON parsed into declared data classes, or Protobuf, over Java `Serializable` or `NSKeyedUnarchiver` on arbitrary input. On iOS, use `NSSecureCoding` with an explicit allowed-class list. Never deserialize data that arrived from an intent, a deep link or a file picker without constraining the types.

19.5 Dynamic code loading

`MASWE-0049` — unsafe dynamic code loading. Loading code at runtime from anywhere other than your signed app package means your app now runs code the OS never verified, and your signature covers nothing.

Both stores restrict this, and the security reasoning is straightforward: it defeats code signing, which is one of the platform guarantees from Chapter 0.4. If you're loading a DEX file, a dynamic library or a script from the network, that's a design to revisit.

19.6 Free compiler protections, and platform currency

`MASWE-0045` is "compiler-provided security features not used," and the fix is usually flipping a build setting. **Position-independent code** and **stack canaries** for native code (`MASTG-TEST-0222`, `0223`, `0228`, `0229`), **ARC** on iOS (`MASTG-TEST-0230`). These cost nothing and blunt whole classes of memory-corruption exploitation. You may see the older `MASTG-TEST-0044` and `MASTG-TEST-0087` ("Make Sure That Free Security Features Are Activated") cited for this — **both are deprecated v1 tests**, superseded by the atomic v2 tests above. The title is a hint about how often these protections aren't switched on.

Related, and easy to defer forever: **keep your platform floor current**. MASWE-0041 and 0042 cover running on and targeting recent platform versions, and MASTG-BEST-0010 covers `minSdkVersion`. Old API levels reopen attack classes the platform already closed — and an old `minSdkVersion` means your security code has to handle devices where the modern primitive doesn't exist.

And keep the **update path** working: MASWE-0043 and MASVS-CODE-2 ask for a mechanism to enforce updates. When you ship a security fix, you need users to actually receive it — which, as Chapter 8 noted, is the one real advantage mobile has over the web.

19.7 Dependencies

MASVS-CODE-3 says use only components without known vulnerabilities. MASWE-0044 names the failure. Chapter 15 covers the pipeline side, including locking, SBOM generation and scanning, and MASTG-TEST-0272 through 0275 test it on both platforms.

The mobile-specific point: your dependency tree is larger than you think, most of it is transitive, and any one library can bring permissions, network calls and privacy obligations with it. Read the merged manifest occasionally.

19.8 How to implement it

Path traversal: canonicalise, then confirm containment

```
fun openInAppStorage(base: File, userSuppliedName: String): File {
    val target = File(base, userSuppliedName).canonicalFile // resolves "../"
    // The check that matters: is it still inside where we meant?
    require(target.path.startsWith(base.canonicalFile.path + File.separator)) {
        "path escapes base directory"
    }
    return target
}
```

iOS deserialization: allow-list the classes

```
// Safe: only the types you expect can be constructed.
let unarchiver = try NSKeyedUnarchiver(forReadingFrom: data)
unarchiver.requiresSecureCoding = true
let order = unarchiver.decodeObject(of: [Order.self, NSString.self], forKey: "order")
```

```
// DON'T: constructs whatever the data asks for. MASTG-TEST-0386.
let order = NSKeyedUnarchiver.unarchiveObject(with: data)
```

Server-side is where validation counts

A reminder from Chapter 12, because client validation is a usability feature and server validation is the control:

```
// Client: reject obvious nonsense early, for the user's benefit.
if (amount <= 0 || amount > dailyLimitShownInUi) return showError()

// Server: re-derive everything. The client's numbers are a request, not a fact.
// - is `payee` actually linked to this account?
// - is `amount` within THIS user's real limit, read from our own database?
// - does the balance cover it, checked transactionally?
// - has an identical request arrived in the last 30 seconds? (replay)
```

Verify it

Validation is best tested with input you would not think to write.

Path traversal. Feed your file handler `../../../../data/data/com.example.app/shared_prefs/prefs.xml` and confirm it is rejected rather than resolved. A canonicalisation check passes this; string matching on `..` does not, because encodings differ.

Injection. Feed your content provider and any raw query path `' OR '1'='1` and a value containing a semicolon. **Pass:** parameterised queries treat them as literal data.

Deserialization. Hand your unarchiver a payload declaring a class you never expected. **Pass:** rejection, because `requiresSecureCoding` and an explicit allowed-class list are set.

And check the server, not the client. Take a valid request, change the price or quantity in a proxy, and replay it.

Pass: the server rejects it. **Fail:** you have found the most consequential bug class in this book, and no client-side validation would have prevented it.

Chapter 20: Securing AI features in mobile apps

Almost every mobile team is now shipping an AI feature, and almost none are threat modelling it. Published guidance is thin, which makes this both a genuine risk and, if you are looking for a place to build expertise, an unusually open field.

20.1 If the user can influence the prompt, what can the prompt influence?

This is the question that matters most, and it has a precise shape.

A language model with **tool access** is a new execution path into your system. Content the model reads becomes, in effect, instructions it may act on. That is **indirect prompt injection**: the attack arrives not in the user's own message but in something the model consumes — a scanned document, a shared file, a web page in a WebView, a message from another user, an email body.

The correct mental model is one you already have: **treat model output exactly as you treat WebView content**. Untrusted. Validated before it reaches anything that mutates state. Never interpolated into a privileged call without checks.

This is recognised at platform level now. Apple's WWDC26 material covers threat modelling agentic app features and applying deterministic mitigations for indirect prompt injection in Foundation Models and App Intents, and the MASTG has added a knowledge entry on **App Intents and AI agent exposure** (MASTG-KNOW-0129). The surface is being formalised as you read this.

Note also why this is specifically a *mobile* problem rather than only a backend one: the mobile client is where untrusted content enters — the camera, the share sheet, the clipboard, the WebView, the file picker. Your existing input validation was designed for form fields, not for content a model will interpret.

20.2 What must never leave the device?

Once user data is in a request body to a third-party model provider, your data-residency and processing story has changed — regardless of what your privacy policy says.

If you have signed commitments with a regulator, or hold obligations under GDPR, HIPAA or a banking supervisor, whoever owns those commitments needs to know **before** you ship, not after. This is a five-minute conversation that occasionally prevents a nine-month remediation.

Practically: classify what the feature needs, send the minimum, and consider whether an on-device model gets you enough. Redact before transmission where you can. And decide deliberately whether provider-side retention and training settings match what you have promised your users.

20.3 Where does the model key live?

If it is in the app, it is public. Chapter 15's problem with a new cost dimension: leaked model keys are billed to you by the token, and the bill arrives faster than your monitoring.

Proxy through your backend, always. There is no configuration of client-side key storage that makes an in-app model provider key safe, because Chapter 1.

20.4 Can you distinguish a real client from a script?

An unattested endpoint behind a metered model is a billing incident waiting to happen.

This is the clearest business case for attestation you will ever get to write, and it is worth using: **an attacker does not need to steal any data to hurt you here. They only need to spend your inference budget**. That reframing lands with finance stakeholders in a way that "defence in depth" does not.

Rate limit per account and per device, require assertions or integrity tokens on the inference path, and alert on volume anomalies rather than discovering them on the invoice.

20.5 What happens when the model is wrong?

Not *if*. Define the fallback path, the timeout, and the user-visible behaviour when the model fails, times out, or returns something unusable. A feature with no fallback is an outage with a friendlier name.

For anything consequential, keep a human in the loop, and make the model's role legible to the user so they can apply their own judgement.

20.6 What are you logging?

Prompts and completions frequently contain user data. If your observability pipeline captures them, and by default it probably does, that pipeline is now in scope for every privacy commitment you hold, and your log retention policy just became a data retention policy.

20.7 The eval question, which is a security question

One more, less obvious. Without an **eval set**, you cannot tell whether a change to your prompt, your retrieval, or your model version broke a safety property you were relying on. Evaluation is usually framed as a quality practice. It is also how you detect regressions in behaviour you are treating as a control.

Chapter 21: Kotlin Multiplatform — what can be shared

KMP adoption grew from roughly 7% to 18–23% in a single year, and this question comes up on every project that adopts it. Published security guidance for it is close to nonexistent, which makes this chapter unusually useful and also a good thing to write about publicly.

21.1 The line, and the reasoning behind it

The rule is: **share the logic and the contracts; keep the platform trust primitives native.**

The reasoning is what matters, because it lets you classify anything not on the list. **Anything whose security derives from platform hardware or a vendor's attestation service cannot be abstracted without losing the property that made it valuable.** A Keystore key is protected because a specific trusted application in a specific TEE enforces its authorizations. There is no cross-platform abstraction of that; there are two different mechanisms with a similar shape.

What you *can* share is everything around it — and that turns out to be most of the code.

Shareable: token storage *interfaces* via `expect / actual`; encryption utilities behind a shared interface with Tink on Android and CryptoKit on iOS; API security header construction, nonce generation and encoding; input validation patterns; risk-signal data models.

Must stay native: Keystore and Keychain access; `BiometricPrompt` and `LocalAuthentication`; Play Integrity and App Attest; network security configuration (Android XML, iOS `Info.plist` and `URLSession` delegate).

21.2 The mistake that will bite you

Define your `expect` declarations around **intent**, not mechanism.

`expect fun getKeystoreKey(alias: String): Key` leaks Android's model into shared code and will not map cleanly onto the Secure Enclave — which, remember from Chapter 4, supports elliptic curves only and does not do direct encryption.

`expect suspend fun storeToken(token: String, requiringUserAuth: Boolean)` describes what you want and implements cleanly on both platforms, because it leaves each `actual` free to use the right primitive.

This sounds like an API design point. It is a security point: an abstraction that forces one platform's mechanism onto the other produces implementations that quietly weaken to fit the interface.

21.3 Two practical cautions

Your `actual` implementations need equivalent review. Shared code gets read by everyone; platform-specific code often gets read by one person. The iOS `actual` that stores a token with `kSecAttrAccessibleAlways` because it fixed a background bug is exactly the kind of thing that survives in a KMP codebase, because the Android reviewers never opened that file.

Budget for CI cost. Multiplatform builds need macOS runners for the iOS targets, which is a real line item rather than an afterthought — and, per Chapter 15, another runner to harden and monitor.

Part 8: Practice

You learn this by doing it. Build a lab, run an assessment on your own app, write it up — and know what to do on the day something goes wrong.

Chapter 22: Build your test lab

You can't learn this from reading. This chapter gets you to a working setup where you can attack your own app safely and legally.

One rule before you start: only test apps you own or have written permission to test. Testing someone else's app without authorisation is illegal in most jurisdictions, regardless of intent. Your own app, your employer's app with sign-off, and OWASP's deliberately vulnerable practice apps are all fine.

22.1 What you need

You can get useful results with free tools and an emulator. Here is the shopping list, in order of how much you will use each thing.

A rooted Android device or emulator. An emulator is easiest and free: create an Android Virtual Device with a **Google APIs** image rather than a Play Store image, because Play Store images are locked and you can't get root. Then `adb root` works. A physical rooted device is closer to reality and worth having eventually — a cheap second device is a reasonable investment. Note the trade-off: rooting changes the environment you're measuring, which matters when you're testing detection.

For iOS, this is harder. A jailbroken device is the full experience and increasingly difficult on current hardware. Without one you can still do a great deal — static analysis of the binary, checking entitlements and `Info.plist`, inspecting a simulator's file system, and reviewing your own source. Start there rather than not starting.

Tools, in rough order of usefulness:

`adb` — the Android Debug Bridge, from the platform tools. Your shell into the device.

JADX — decompiles an APK into readable Java. Use `jadx-gui` and browse your own app for twenty minutes; it's an education.

Frida — runtime instrumentation. Install the Python package on your machine and push the matching `frida-server` binary to the device. Version match matters and is the most common setup problem.

objection — Frida-powered, with commands for the common tasks so you don't write scripts on day one. `objection -g <package> explore` then `android sslpinning disable` is your first real experiment.

mitmproxy — a proxy you can read. Install its CA certificate on the device, point the device's proxy at your machine, and watch your own traffic. On modern Android you'll also need to allow user-added CAs in a *debug* network security configuration, which is itself a useful lesson in why that setting matters.

semgrep — pattern-based static analysis. Many MASTG demos use it, and it's how you find a class of issue across a whole codebase rather than one instance.

apktool for repackaging, **radare2** or `rabin2` for iOS binaries, and `strings` — which you already have.

22.2 Practice targets before your own app

Learn the tools on something designed for it, so you're debugging one thing at a time.

The **OWASP MAS Test Apps** for Android and iOS embed code samples so every MASTG demo is reproducible on a real device. Start here — each demo tells you what to run and what you should see.

The **MAS Crackmes** are deliberately protected apps for reverse-engineering practice: <https://mas.owasp.org/crackmes/>

22.3 Your first five experiments

Do these in order on your own app. Each takes minutes and each will teach you something you didn't expect.

One: `strings` on your release APK. Not the debug build. Read what comes out. Write down anything sensitive.

Two: open it in **JADX**. Find your API base URL. Find one security decision — a root check, a feature flag, a validation. Ask yourself how long it would take to change it.

Three: read your own app's data directory. `adb shell` then `run-as <package>` (or root) and look in `/data/data/<package>/`.

Open the shared preferences files and any SQLite database. Look for tokens and personal data in plaintext.

Four: proxy your traffic. Get mitmproxy in front of the app and read a request. If pinning is enabled, note that it fails — then use `objection` to disable pinning and note that it now succeeds. That contrast is the single most clarifying experience in mobile security.

Five: hook something. With Frida, hook your root-detection function and make it return `false`. Then hook `Cipher.doFinal` and log the arguments — `MASTG-DEMO-0106` shows you how. Watching your own plaintext scroll past changes how you design.

22.4 What to do when a step fails

Each of these will happen to you. None of them means you have done something wrong.

Frida won't connect. Version mismatch between the client and `frida-server`, or the server isn't running, or you're not root. Check versions first.

Proxy shows no traffic. The app may be ignoring the system proxy (some HTTP clients do), or using a non-HTTP protocol, or pinning is rejecting the connection before you see it. Check the app's logs.

Certificate errors everywhere. The device doesn't trust your proxy CA, or the app doesn't trust user-added CAs — which on modern Android is the default and correct behaviour. Use a debug network security configuration.

Nothing in the data directory. You might be looking at the wrong user profile, or the app genuinely stores little locally — which would be good news.

Each of these failures is itself information about your app's posture. Note it rather than fighting it.

Chapter 23: Running an assessment and writing it up

Now you have a lab. This chapter is the method, and then how to communicate what you found — which is the part that determines whether anything changes.

23.1 Scope it first

Write down, before you start: which app and build, which platforms, which device and OS versions, which flows are in scope, and what you're *not* looking at. Ten minutes of scoping saves you from a report that's impossible to interpret later.

Pick a **testing profile** to aim at (Chapter 3.2): **MAS-L1** for a basic level on an app handling sensitive data, **MAS-L2** for highly sensitive data, **MAS-R** if resilience against reverse engineering matters. This sets your expectations so you're not writing up a missing anti-debugging control on a content app.

23.2 Work the standard, not your instincts

Take the MASWE catalogue in Chapter 27 — seventy-eight weaknesses — and go down it, marking each **yes**, **no**, or **not applicable** for your app. For each *yes*, find the matching MASTG test and run it.

This is slower than poking around and dramatically better, for two reasons. You cover the things you'd never have thought of, and your output maps to a standard someone else recognises.

Each MASTG test page tells you what to look for, which tools to use, how to reproduce it on each platform, and what a passing implementation looks like. Use the URL pattern from the ID:

<https://mas.owasp.org/MASTG/tests/android/MASVS-STORAGE/MASTG-TEST-0001/>

23.3 Rate what you find

Severity is likelihood combined with impact, and both need thinking about.

For **impact**, ask what an attacker gains. Reading their own cached data is not the same as reading another user's account, which is not the same as moving money.

For **likelihood**, ask what the attack requires. Something exploitable remotely with no special access is far more likely than something requiring a rooted device and physical possession. "Requires root" is a genuine mitigating factor and stating it honestly builds trust — as does not using it to dismiss something that matters.

Be consistent, and be prepared to defend each rating. A report where everything is critical gets ignored entirely.

23.4 Write the report

Structure it so a reader can act:

Scope and date. What you tested, which build, which devices and OS versions. Without this the report ages into uselessness.

Summary. Three or four sentences: what you assessed, what the overall posture looks like, and the two or three things that matter most. Assume this is all a senior stakeholder reads, and write it accordingly.

Findings. For each one: a clear title, the MASVS control and MASWE weakness, severity, reproduction steps precise enough that someone else gets the same result, evidence, and **impact in business terms**.

That last part is what gets work funded. Compare:

Token stored in plaintext in SharedPreferences.

with:

An attacker with brief physical access to an unlocked phone, or malware on a rooted device, can extract a session token that remains valid for 90 days and is not bound to the device — allowing full account access from the attacker's own machine, with no further interaction from the user.

Both describe one bug. Only one of them gets prioritised.

What held. List the controls you tested that worked. This is counterintuitive and it's what makes your report credible. "Pinning held against objection's standard bypass and required a custom Frida script" tells a reader you understand the difference between a tool's output and an assessment.

Remediations. Specific, with before and after where you can. Link the relevant `MASTG-BEST` practice so the developer has the pattern, not just the problem.

Residual risk. What you're accepting and why. Senior reviewers read this section first, because it shows whether you understand trade-offs or merely found things.

23.5 Getting it fixed

A report nobody acts on was a waste of a week. Two things help.

Turn findings into tickets yourself, with owners, rather than handing over a document and hoping. Include the reproduction steps in the ticket so the developer doesn't have to open the report.

Retest and record it. A finding isn't closed because someone pushed a commit; it's closed because you ran the test again and it passed. Note the date and the build. That retest history is what turns an assessment into a security programme.

23.6 Make it a habit

An annual assessment finds a year of accumulated problems at the worst possible moment. Instead:

Pick **one MASWE weakness a week**. Find the matching test. Run it against your app. Write down the result. Fifty-two weeks of that and you have covered the catalogue, built the skill, and produced a documented history — and you become the person the team asks.

Chapter 24: When something goes wrong

Every other chapter is about prevention. This one is about the day prevention failed, because that day arrives and mobile has specific constraints that make it different from a server incident.

24.1 The constraint that shapes everything

Mobile incident response differs from server incident response for one reason, and everything else follows from it.

You cannot patch a mobile app quickly. A server fix deploys in minutes. A mobile fix needs a build, a store review, and then *users choosing to update* — and a meaningful share of your install base won't update for weeks, or ever.

So mobile incident response leans on things you can change *without* a release:

Server-side mitigation. Reject the vulnerable request pattern, tighten validation, revoke credentials, disable an endpoint. Almost always your fastest lever, and often sufficient.

Feature flags and remote configuration. If you can turn the affected feature off remotely, you can stop the bleeding in minutes. This is why remote kill switches are a security capability and not just a product one — and why Chapter 8 asks for one on pinning specifically.

Forced update. MASVS-CODE-2 and MASWE-0043 exist for this moment. If you have a mechanism to require a minimum version, you can compel the fix. If you don't, build one before you need it — during an incident is a bad time to discover you can't.

24.2 Prepare before you need it

Four things, none of which take long, all of which are painful to arrange under pressure:

Know who to call. Who decides to disable a feature? Who talks to the regulator? Who approves an emergency release? Write down names, not roles.

Know what your logs contain. During an incident you need to answer "who was affected and when," and you can only answer it from data you were already collecting. Log security decisions with enough context to investigate — and, per Chapter 12, never the secrets themselves.

Have a rotation runbook. Which credentials exist, where they live, and how to rotate each one. Chapter 15's discipline pays off here.

Know your notification obligations. GDPR requires breach notification within 72 hours. That clock starts before you finish understanding the problem, which is exactly why you find out the requirement now rather than then.

24.3 The sequence

Five steps, in this order. The ordering matters more than the detail.

Contain. Stop it getting worse. Revoke the leaked credential, disable the endpoint, turn off the feature. Resist the urge to investigate first — containment is cheap and reversible; a widening breach is not.

Assess. What was accessed, by whom, how many users, over what window. Be careful about early numbers; the first estimate is usually wrong and it's the one that ends up in the notification.

Notify. Follow your legal obligations and tell your users the truth. Vagueness reads as concealment and costs more trust than the incident did.

Fix. Server-side first because it's fast. Then the client fix, with a forced update if warranted.

Learn. A blameless post-mortem: what happened, why the control that should have caught it didn't, and what changes. The output is a change to your process, not a person to blame — teams that blame individuals stop reporting problems early, which is the one thing you cannot afford.

24.4 The specific mobile cases

Five scenarios you are most likely to meet, and what each one actually demands of you.

A signing key is compromised. With Play App Signing, your upload key is recoverable — register a new one (Chapter 15.6). Without it, an attacker can ship code as you, and you're into store escalation and possibly a new listing. This is the strongest argument for the split key model, and it's much better made before the incident.

A secret leaked in your app package. Rotate immediately and assume it's public. Then ask why it was in the package, because that's the actual fix — the leak is a symptom of Chapter 15.2's classification not being applied.

Certificate pinning broke. Users can't connect. Use your kill switch. If you don't have one, you're shipping an emergency release and waiting for review — which is precisely the outage risk that Chapter 8's critics were talking about.

A vulnerable dependency lands in a shipped version. Mitigate server-side if you can, ship the update, and consider a forced update if the exploit is remote and unauthenticated.

A cloned version of your app appears. Store takedown, and check whether attestation would have prevented the clone from reaching your API. Often it would (Chapter 9).

24.5 The uncomfortable value of an incident

An incident gives you organisational attention you cannot otherwise buy. The security work that was deprioritised for two quarters becomes fundable in a week.

Use it well and use it honestly: bring the plan you already wrote, name the controls that would have prevented this specific event, and don't overreach into unrelated wishes. Credibility spent well here lasts for years — and credibility spent badly, on an inflated ask, does not come back.

Chapter 25: Reference — testing method at a glance

Building controls is the easy half. Proving they hold is what separates an engineer who has read about security from one who can be trusted with it. This chapter is a method, not a checklist — the difference being that you are meant to think while doing it.

25.1 Where the material is

Work the **MASTG** test cases rather than the theory. Since the v2 refactor, every test is individually referenceable with structured metadata and reproducible demos, so you can go directly to what you need instead of reading a book-length guide front to back.

Each MASVS control links to specific tests. `MASVS-STORAGE-1`, for instance, maps to tests covering `SharedPreferences` analysis, Keychain inspection, SQLite checks and log review. A test case tells you what to look for, which tools to use, how to reproduce on both platforms, and what a passing implementation looks like. The mapping table is the thing to bookmark.

Two practice environments exist so that you are not learning tooling and auditing production simultaneously. The **MAS Test Apps** for Android and iOS are purpose-built skeleton apps with code samples embedded, so every demo is reproducible on a real device. The **MAS Crackmes** are deliberately protected apps for reverse-engineering practice.

25.2 Your toolkit

Everything here is free. Chapter 22 covers installing and using them; this is what each one is for.

JADX decompiles an APK so you read your own logic as an attacker sees it. **radare2** and `rabin2` do the equivalent for iOS binaries, and appear throughout the MASTG's iOS demos. **Frida** attaches to a running process for hooking and instrumentation. **objection** is Frida-powered and gives you storage inspection and pinning bypass without writing scripts. **adb** and `sqlite3` let you inspect `/data/data/<package>/`. **mitmproxy** or **Burp** intercept traffic to confirm whether pinning actually holds. **semgrep** appears across the MASTG demos for static pattern matching, which is how you find a class of issue across a codebase rather than one instance. **strings** is the thirty-second test that finds embedded secrets. **trufflehog** and **GitGuardian** scan history and commits.

25.3 A first assessment, in order

Do these in sequence on your own app. Each step informs the next.

One. Build a release artifact. Run `strings`. Write down everything sensitive that appears. Do not skip this because you are sure it is clean.

Two. Decompile with **JADX**. Find your API endpoints and any security-relevant logic. Ask honestly how long it would take you to change one of those decisions and repackage.

Three. Install on a rooted device or emulator. Inspect app storage for plaintext tokens, cached personal data, and log files.

Four. Put a proxy in front of it. Does traffic intercept? If pinning is enabled, does it hold?

Five. Attach Frida. Bypass your own root detection. Bypass your own pinning. Hook your own biometric callback. Note how long each took.

Six. Replay a valid authenticated request from a different device. What stops you? (Chapter 12.3.)

Seven. Write it up.

25.4 Writing findings properly

The findings document is the artifact that changes how people see you, so write it as a document rather than as a dump.

Open with **scope and date**: what you tested, which build, which device and OS version. Without this the document ages badly and nobody can tell whether it still applies.

For each **finding**, give the MASVS control and MASWE weakness it maps to, a severity, reproduction steps precise enough that someone else gets the same result, evidence, and, most importantly, **impact stated in business terms**. "Token readable from app storage on a rooted device" is a fact. "An attacker with brief physical access to an unlocked phone can extract a credential that remains valid for 90 days and is not bound to the device" is a finding someone will fund.

Include **what held**. This is counterintuitive and it is what makes the document credible. A report that only lists failures reads like a tool output; one that says "pinning held against objection's standard bypass and required a custom Frida script" reads like it was written by someone who understands the difference.

Then **remediations** with before and after, and **residual risk** — what you accepted, and why. That last section is the one senior reviewers read first, because it tells them whether you understand trade-offs or just found things.

25.5 The claim this earns you

There is a specific sentence that gets people hired into mobile security, and it is not "I know which controls to specify."

It is: **"I attacked my own app, here is what I found, here is what I changed, and here is what I decided to accept."**

Those are different claims and interviewers can tell them apart within two questions.

Part 9: The catalogues

The previous parts explain mechanisms. This part is the standard's own structure, so you can audit against it and speak its vocabulary. Everything here was read from mas.owasp.org on 10 September 2026. IDs and weakness titles are as published; where I add a one-line explanation of a control, that summary is mine, and the normative text is at the linked page.

Chapter 26: The 24 MASVS controls

MASVS-STORAGE — secure storage of sensitive data at rest

MASVS-STORAGE-1 — the app securely stores sensitive data. This applies wherever the data lands, app-private internal storage or public locations such as Downloads, and covers data originating from the user, the backend, system services and other apps.

MASVS-STORAGE-2 — the app prevents leakage of sensitive data to places you did not choose: logs, backups, screenshots, the keyboard cache, notifications and IPC. Chapter 6.4 is this control in practice.

<https://mas.owasp.org/MASVS/05-MASVS-STORAGE/>

MASVS-CRYPTO — cryptography

MASVS-CRYPTO-1 — the app uses strong, current cryptographic primitives with parameters following industry standards, typically defined externally in NIST SP 800-175B and SP 800-57.

MASVS-CRYPTO-2 — the app manages keys properly across the whole lifecycle: generation, storage, use, rotation and destruction. Chapters 4 and 5.

<https://mas.owasp.org/MASVS/06-MASVS-CRYPTO/>

MASVS-AUTH — authentication and authorization

MASVS-AUTH-1 — the app uses secure authentication and authorization protocols and follows the relevant best practices.

MASVS-AUTH-2 — the app performs local authentication securely, meaning it cannot be bypassed by hooking a boolean. Chapter 11.

MASVS-AUTH-3 — the app secures sensitive operations with additional authentication. Step-up, per Chapter 11.5.

<https://mas.owasp.org/MASVS/07-MASVS-AUTH/>

MASVS-NETWORK — network communication

MASVS-NETWORK-1 — the app secures all network traffic according to current best practice.

MASVS-NETWORK-2 — the app performs identity pinning for all remote endpoints under the developer's control. Read Chapter 8 before acting on this one, and note the scoping phrase.

<https://mas.owasp.org/MASVS/08-MASVS-NETWORK/>

MASVS-PLATFORM — platform interaction

MASVS-PLATFORM-1 — the app uses IPC mechanisms securely: intents, content providers, deep links, app extensions.

MASVS-PLATFORM-2 — the app uses WebViews securely: no unnecessary native bridges, no untrusted content, no local file access.

MASVS-PLATFORM-3 — the app uses the user interface securely: no sensitive data in screenshots, notifications, the clipboard or behind overlays.

<https://mas.owasp.org/MASVS/09-MASVS-PLATFORM/>

MASVS-CODE — code quality

MASVS-CODE-1 — the app requires an up-to-date platform version. **MASVS-CODE-2** — the app has a mechanism to enforce updates. **MASVS-CODE-3** — the app only uses software components without known vulnerabilities. **MASVS-CODE-4** — the app validates and sanitizes all untrusted inputs.

<https://mas.owasp.org/MASVS/10-MASVS-CODE/>

MASVS-RESILIENCE — resilience against reverse engineering and tampering

MASVS-RESILIENCE-1 — the app validates the integrity of the platform. Root and jailbreak detection, attestation.

MASVS-RESILIENCE-2 — the app implements anti-tampering mechanisms. **MASVS-RESILIENCE-3** — the app implements anti-static-analysis mechanisms. Obfuscation. **MASVS-RESILIENCE-4** — the app implements anti-dynamic-analysis mechanisms. Debugger and hook detection.

Read Chapter 14 alongside these, because this category is the one where a naive reading produces controls that report success while providing little.

<https://mas.owasp.org/MASVS/11-MASVS-RESILIENCE/>

MASVS-PRIVACY — privacy, added in v2.1.0

MASVS-PRIVACY-1 — the app minimizes access to sensitive data and resources. **MASVS-PRIVACY-2** — the app prevents identification of the user. **MASVS-PRIVACY-3** — the app is transparent about data collection and use. **MASVS-PRIVACY-4** — the app offers the user control over their data.

<https://mas.owasp.org/MASVS/12-MASVS-PRIVACY/>

Chapter 27: The 78 MASWE weaknesses

This is the most directly useful list in the standard, because these are the things that actually go wrong. Read it as an audit: for each line, ask whether your app does this.

Storage — 0001 sensitive data stored unencrypted in private storage · 0002 sensitive data stored unencrypted outside private storage · 0003 cryptographic keys stored outside the platform keystore · 0004 sensitive data hardcoded in the app package · 0005 insertion of sensitive data into logs · 0006 sensitive data not excluded from backup.

Cryptography — 0007 improper encryption · 0008 improper hashing · 0009 improper use of MAC · 0010 improper generation of cryptographic signatures · 0011 improper verification of cryptographic signature · 0012 improper random number generation · 0013 improper cryptographic key generation · 0014 improper cryptographic key derivation · 0015 key rotation not implemented · 0016 key access not restricted · 0017 device secure lock not enforced.

Authentication — 0018 lack of authentication or authorization on app components · 0019 lack of auto-fill support for credential providers · 0020 local authentication can be bypassed · 0021 fallback to non-biometric credentials allowed for sensitive transactions · 0022 crypto keys not invalidated on new biometric enrollment · 0023 step-up authentication not implemented for sensitive actions · 0024 sensitive data accessible after session termination · 0025 lack of non-repudiation for critical actions.

Network — 0026 network traffic not encrypted · 0027 insecure certificate validation · 0028 insecure identity pinning.

Platform — 0029 insecure deep links · 0030 improper use of the clipboard · 0031 allowing untrusted app extensions · 0032 insecure intents · 0033 sensitive native functionality exposed in WebViews · 0034 WebViews allow access to local resources with untrusted content · 0035 WebViews loading untrusted content · 0036 unnecessary exposure of sensitive data via the UI · 0037 unnecessary exposure via notifications · 0038 insufficient protection from screenshots or screen recordings · 0039 app vulnerable to overlay attacks · 0040 sensitive data leaked via accessibility services.

Code — 0041 running on a recent platform version not ensured · 0042 latest platform version not targeted · 0043 enforced updating not implemented · 0044 dependencies with known vulnerabilities · 0045 compiler-provided security features not used · 0046 use of deprecated APIs · 0047 non-standard APIs for security-critical functionality · 0048 malicious code included in the app · 0049 unsafe dynamic code loading · 0050 unsafe handling of untrusted data.

Resilience — 0051 root/jailbreak detection not implemented · 0052 app virtualization environment detection not implemented · 0053 emulated or virtual device detection not implemented · 0054 device attestation not implemented · 0055 malware detection not implemented · 0056 app attestation not implemented · 0057 app resources integrity not verified · 0058 runtime code integrity not verified · 0059 code obfuscation not implemented · 0060 resource obfuscation not implemented · 0061 debug artifacts not removed · 0062 no application-level payload encryption · 0063 debug mechanisms not disabled · 0064 debugger detection not implemented · 0065 dynamic analysis tools detection not implemented.

Privacy — 0066 inadequate permission management · 0067 lack of anonymization or pseudonymisation · 0068 incorrect use of identifiers for user tracking · 0069 usage of non-privacy-preserving functionality · 0070 inadequate awareness for privacy-relevant actions · 0071 inadequate defaults for privacy-relevant actions · 0072 inadequate privacy policy · 0073 inadequate data collection declarations · 0074 inadequate tracking domains declarations · 0075 non-reproducible builds · 0076 lack of proper data management controls · 0077 inadequate data visibility controls · 0078 inadequate or ambiguous

user consent mechanisms.

<https://mas.owasp.org/MASWE/>

Three that deserve special attention, because engineers routinely miss them:

MASWE-0022 — keys not invalidated on new biometric enrollment. Chapter 11.3 is the scenario. This is the highest-impact, lowest-awareness item in the catalogue.

MASWE-0024 — sensitive data accessible after session termination. Logout that clears the token but leaves cached responses, database rows and log entries on disk is a finding, and it is extremely common because "logout" is usually implemented as an auth concern rather than a data concern.

MASWE-0075 — non-reproducible builds. Filed under Privacy, which surprises everyone. It matters because nobody can verify what shipped if the build cannot be reproduced, which connects it directly to Part 6.

Chapter 28: The tests, techniques and practices you will use

The tests worth knowing by number

Android storage: TEST-0001 local storage · 0003, 0203, 0231 logs statically and at runtime · 0009, 0216 backups · 0011 memory · 0287 unencrypted data via `SharedPreferences` at runtime · 0305 unencrypted via `DataStore`.

Android crypto: TEST-0212 hardcoded keys in code · 0221, 0232, 0350 broken algorithms and modes · 0309, 0310 reused initialization vectors · 0208 insufficient key sizes.

Android auth: TEST-0018 biometric authentication · 0326 fallback to non-biometric · 0328 enrollment-change detection · 0330 keys with extended validity duration.

Android network: TEST-0020, 0217, 0218 TLS settings and insecure protocols · 0022, 0242, 0243, 0244 pinning presence, expiry and in live traffic · 0235, 0236 cleartext in config and on the wire · 0285, 0286 trust in user-added CAs · 0284 SSL error handling in `WebViews`.

Android platform: TEST-0031, 0033, 0334 JavaScript, Java objects and native code exposed through `WebViews` · 0028, 0393 deep links and unverified App Links · 0030, 0381 vulnerable and insecure `PendingIntent` · 0364, 0365, 0366 exported unprotected activities, services and receivers · 0289, 0291 screenshot exposure.

Android code and resilience: TEST-0272, 0274 vulnerable dependencies including via SBOM · 0339 SQL injection in content providers · 0038, 0224, 0225 signing correctness, scheme and key size · 0039, 0226, 0227 debuggable app, manifest and `WebView` debugging · 0045, 0324, 0325 root detection · 0341 hook detection · 0368, 0369 insufficient obfuscation of Java/Kotlin and native code.

Android privacy: TEST-0206 undeclared PII in captured traffic · 0254, 0255 dangerous and non-minimized permissions.

iOS storage: TEST-0052 local data storage · 0299 data protection classes in private storage · 0215, 0298 backup exclusion · 0055, 0313 keyboard cache · 0388 shared App Group containers.

iOS crypto and auth: TEST-0062 key management · 0213, 0214 hardcoded keys in code and files · 0064 biometric authentication · 0266, 0267 event-bound biometric auth · 0270, 0271 enrollment-change detection.

iOS network: TEST-0066, 0342, 0343 TLS settings, weak ATS exceptions, `URLSession` configuration · 0068, 0385 pinning and pinning in ATS · 0396, 0397 `URLSessionDelegate` and `WKNavigationDelegate` bypassing certificate validation.

iOS platform: TEST-0076, 0078 `WebViews` and exposed native methods · 0331, 0333 deprecated `WebView` APIs and broad file read access · 0070, 0075, 0370, 0371 universal links and custom URL schemes with missing input and source validation · 0276–0280 pasteboard use, contents, clearing, expiry and device scope · 0389, 0390 custom keyboard restrictions and full-access requests.

iOS code and resilience: TEST-0273, 0275 vulnerable dependencies · 0081, 0220 signing and outdated signature format · 0088, 0240, 0241 jailbreak detection · 0354 hook detection · 0391 insufficient native obfuscation.

<https://mas.owasp.org/MASTG/tests/>

The techniques, in learning order

Generic: MASTG-TECH-0047 reverse engineering · 0048, 0049 static and dynamic analysis · 0050 binary analysis · 0051 tampering and runtime instrumentation · 0071 retrieving strings · 0119 intercepting HTTP by hooking network APIs at the application layer · 0120, 0121 intercepting HTTP and non-HTTP traffic with a proxy · 0122 passive eavesdropping · 0123, 0124 achieving MITM via ARP spoofing or a rogue access point.

Android: MASTG-TECH-0001, 0002 device shell and host-device transfer · 0003 obtaining and extracting apps · 0004 repackaging · 0007, 0008 exploring the package and app data directories · 0009 monitoring system logs · 0011 setting up

an interception proxy · **0012 bypassing certificate pinning** · 0013–0018 reverse engineering, static and dynamic analysis, smali disassembly, Java decompilation, native disassembly.

<https://mas.owasp.org/MASTG/techniques/>

The best practices worth putting in a PR template

Logging: BEST-0002 remove logging code, 0022 disable verbose and debug logging in production. Backups: 0004, 0023 exclude sensitive data. Crypto: 0005, 0009 secure encryption modes and algorithms, 0001 and 0025 secure random number generators. Signing and build: 0006 up-to-date APK signing schemes, 0007 debuggable disabled, 0008 WebView debugging disabled, 0010 up-to-date `minSdkVersion`.

Screens and input: 0014–0018 preventing screenshots and recording via `SECURE_FLAG`, `setRecentsScreenshotEnabled`, `setSecure` for SurfaceViews and `SecureFlagPolicy.SecureOn` for Compose · 0019, 0026, 0044 non-caching input types, keyboard-cache prevention and masking · 0027 notifications · 0069 keep sensitive input on the system keyboard.

Biometrics: **0031 enforce strong biometrics for sensitive operations** · **0036 use cryptographic binding** · **0037 invalidate keys on enrollment changes** · **0038 require explicit user confirmation**.

Resilience: 0029 implement resilience and RASP signals · 0030 root detection · 0041 harden against runtime hooking · 0046, 0053 harden against emulation and virtual devices · 0047, 0074 continuous anti-debugging.

Network: 0042, 0043 strong TLS in ATS and where ATS does not apply · 0073 properly validate server trust in `URLSessionDelegate` and `WKNavigationDelegate`.

WebViews: 0011, 0033 securely load file content · 0012, 0013 disable JavaScript and content provider access where possible · 0035 prefer origin-scoped messaging over legacy bridges · 0058–0062 restrict native functionality via bridges, render sensitive UI natively over the WebView, use `WKContentWorld` isolation, and `WKScriptMessageHandlerWithReply` for returning data.

IPC and components: 0039 prevent SQL injection in content providers · 0040 prevent overlay attacks · 0049, 0052 restrict and validate access to exported components · 0056, 0057 explicit intents for internal IPC and sanitize external data · 0063 immutable `PendingIntent`s · 0064 safe deserialization APIs.

Integrity and links: 0065, 0066, 0067 storage and source-code integrity checks · 0070–0072 verify App Links with `autoVerify` and Digital Asset Links, validate deep link and universal link parameters.

Platform hygiene: 0051 minimize iOS permissions and entitlements · 0020 update the GMS security provider · 0021 proper error and exception handling · 0003 comply with privacy regulations.

<https://mas.owasp.org/MASTG/best-practices/>

Part 10: Questions and answers

These are the questions that come up in design reviews and interviews, and the ones where people's understanding tends to be thin. Answers are written out properly. If you cannot answer one without looking, that is your next reading.

Foundations

Why not just obfuscate everything and prevent reverse engineering? Because you cannot. Any app decompiles with free tools and any method can be hooked at runtime. Obfuscation raises cost and buys time against opportunistic attackers and automated tooling; it does nothing against a determined one. Design so that whatever is extracted is worthless without server-side validation. See Chapters 1 and 14.

What does "the client is not the trust boundary" mean in practice? Every decision that matters happens on your server. The client gathers signals and proposes actions; the server decides. If your app decides locally whether a transfer is permitted, an attacker patches that decision out in minutes.

A root detection library flags a device. What should the app do? Report it to the backend as a risk signal and let the session continue under elevated scrutiny. The backend then decides whether to require step-up authentication, cap limits, or flag the account. Blocking locally is trivially removed by an attacker who is already hooking your code, generates false positives against developers and custom-ROM users, and destroys the intelligence you would otherwise have gained. MASTG-DEMO-0108 demonstrates the bypass.

What is the difference between the Mobile Top 10 and MASVS? The Top 10 is an awareness list from a separate working group, updated in late 2024 after eight years. MASVS is the verification standard: eight categories, twenty-four controls in v2.1.0. Top 10 builds awareness; MASVS provides assurance. Different groups, different jobs.

What is MASVS L2? A trap. Since MASVS v2.0.0 the verification levels moved out of MASVS and became MASTG testing profiles: MAS-L1, MAS-L2 and MAS-R. Say "the MAS-L2 profile." If a source still says MASVS L2, or MSTG rather than MASTG, treat its technical detail as dated too.

Can you get an app OWASP-certified? No. OWASP states it cannot certify mobile applications or accredit third parties to do so. What you can publish is a certification statement — your own claim about which controls you meet and how you verified them, which anyone can check with the MASTG. That is more useful than a badge because it is falsifiable.

Who am I actually defending against? Four different adversaries with different economics: the opportunist running automated scans, the cloner repackaging your binary, the fraudster abusing your business logic at scale, and the targeted adversary with time and money. Naming which one a control addresses stops you from arguing about obfuscation when your real problem is that your API trusts a client-supplied user ID. Chapter 1.3.

Storage and keys

Where do you store an auth token on Android in 2026? Not `EncryptedSharedPreferences` — Google deprecated the whole Jetpack Security Crypto library in April 2025 at 1.1.0-alpha07 with no further releases. Use `DataStore` for persistence, `Tink` for encryption, and `Android Keystore` via `KeyGenerator` for key protection. For long-lived tokens add `setUserAuthenticationRequired`. Better still, keep short-lived access tokens in memory and persist only the refresh token.

Why was `EncryptedSharedPreferences` deprecated? Google was quiet about it, but the reasons are visible: it had to paper over Keystore inconsistencies across OEM devices and Android versions, it performed synchronous crypto on the calling thread producing `StrictMode` violations, and keyset corruption exceptions plagued specific OEM devices. Maintaining it alongside the modern `DataStore` path was not sustainable.

Is plain `SharedPreferences` insecure? Mostly no, and the existence of `EncryptedSharedPreferences` misled a generation of developers about this. Since Android 10, file-based encryption is enforced on device and the app sandbox isolates your data; reading another app's preferences generally requires physical access or an already-compromised device. Encrypt because your threat model includes device compromise, not out of reflex.

What actually happens when I encrypt with a Keystore key? Your process sends the data and a key handle to the `keystore2` daemon. The daemon holds your key as an encrypted keyblob it can store but cannot use or reveal. It passes the request to the `KeyMint` trusted application running in secure hardware, which decrypts the keyblob internally, validates the key's authorizations, performs the operation and returns only the result. The key never enters your process or the Android OS.

TEE versus StrongBox — what is the real difference? The TEE is a secure area of the *main* processor, isolated by hardware; if Android is fully compromised an attacker might *use* your keys on that device but cannot *extract* them. StrongBox is a dedicated, physically separate secure processor: an embedded or integrated Secure Element, such as

Titan M in Pixel devices. It offers stronger isolation and tamper resistance. StrongBox arrived in Android 9, is not universal, is slower, and supports a deliberately reduced subset of algorithms and key sizes.

Why does StrongBox availability change your design? Because you must decide the fallback explicitly. Check `KeyInfo.getSecurityLevel()`, which returns `SOFTWARE`, `TRUSTED_ENVIRONMENT` or `STRONGBOX`, and either fall back to TEE and record that you did, or require server-side step-up for that flow on that device. Silently falling back to a software-backed key for a payment flow is a finding, not a fallback.

How does the iOS Keychain protect an item? It is one SQLite database for all apps, managed by securityd, which decides your access from your entitlements. Each item uses two AES-256-GCM keys: a metadata key encrypting all attributes except the secret, protected by the Secure Enclave but cached in the Application Processor so searches stay fast; and a per-row secret key encrypting `kSecValueData`, which always requires a round trip through the Secure Enclave.

Which Keychain accessibility class for a token, and why? `WhenUnlockedThisDeviceOnly` for most tokens.

"WhenUnlocked" requires the device to be unlocked; "ThisDeviceOnly" keeps the item out of backups and off other devices. If background refresh needs access, `AfterFirstUnlock` is the considered answer — Apple names that exact use case. `kSecAttrAccessibleAlways` maps to no protection and should not appear in new code.

Background refresh is failing because my Keychain item is WhenUnlocked. What do I do? Restructure when the work happens, or move to `AfterFirstUnlock` deliberately. Do not loosen to `Always` — that makes the item readable on a locked device and puts it in the backup. This is the single most common way iOS token storage gets quietly weakened.

Can the Secure Enclave encrypt my data directly? No, and this catches people. It supports elliptic-curve keys only, with no RSA, and it is for signing and key agreement rather than direct encryption and decryption. To get Secure Enclave protection for encrypted data you do ECDH key agreement, or encrypt a symmetric key. Your `SecKey` in Swift is a handle; the key itself never enters RAM.

What is the risk of adding an app to my keychain-access-group? You are trusting that app completely. Anything in the group can call `SecItemCopyMatching` on your items, including a compromised app under your own Team ID. Enumerate what is in your groups.

Where do secrets leak that has nothing to do with encryption? Logs, backups, auto-generated screenshots when the app backgrounds, the keyboard cache, lock-screen notifications, the clipboard (on iOS the general pasteboard is shared and has historically synced across devices), and process memory. MASVS-STORAGE-2 exists for exactly this, and it is where most real leaks happen.

Key attestation

What is key attestation, and why would my backend believe the app? Your server issues a nonce; the app generates a key in the Keystore with that nonce as the attestation challenge; Android produces an X.509 chain rooted in a Google attestation root, with a `KeyDescription` extension carrying the security level, boot state and your nonce; your server verifies the chain and reads the properties. It is credible because the authorization list is collected or generated by code inside the secure hardware and is not controlled by the platform — sourced from the bootloader or a secure channel that does not require trusting Android.

What is going to break my attestation verification in 2026? Google activated a new Remote Key Provisioning root certificate on 1 February 2026, and all RKP-enabled devices must use it by 10 April 2026. Applications that verify key attestation and do not trust the new root will start failing. Also: under online provisioning the chain is longer than it used to be and is subject to change, and the root is moving from RSA to ECDSA. Do not hardcode the root, the chain length, or the algorithm.

Why is RKP better for privacy than factory-provisioned keys? Each application receives a different attestation key, keys rotate regularly, and Google's backend is segmented so the server verifying a device's public key does not see the attached attestation keys — so attestation keys cannot be correlated back to a device.

Can hardware key attestation be defeated? Yes, and you should know it. Tooling exists that serves selected apps from a software KeyMint running inside the real keystore daemon while other keys stay on real hardware, embedding AOSP's own reference trusted application and signing with a supplied keybox — producing certificates generated the same way real hardware generates them, and therefore internally consistent. Attestation raises the bar substantially; treat a single result as a strong signal rather than proof.

Network and pinning

Should you pin certificates? Know both sides. OWASP's Pinning Cheat Sheet says for most apps probably never, because outage risk outweighs benefit now that Certificate Transparency, short lifetimes and automated issuance exist; Google's Android docs caution against it for the same operational reason. But MASTG recommends it for MAS-L2 apps and MASVS-NETWORK-2 asks for it on endpoints you control, and MASTG notes that Google's warning is often misread

— the real advice is to pin *with* backup pins and a rotation plan. My position: pin if you control both ends, can update the pinset, and have an owned rotation runbook. Otherwise CT plus HSTS.

What does pinning actually protect against, given TLS already validates? Standard validation trusts any root in the device trust store — roughly a hundred organisations plus anything an administrator or user has added. Pinning narrows that to a specific identity you nominated. The threat is a rogue or compromised CA, or an administrator-installed certificate.

Why pin SPKI rather than the certificate? The SPKI is the public key info. If a certificate is renewed with the same key pair, the SPKI is unchanged and an SPKI pin still validates; a full-certificate pin breaks on every renewal. Given lifetimes dropped to 200 days in March 2026 and reach 47 days by 2029, that difference decides whether pinning is operable at all.

Why is an intermediate CA pin more rotation-resilient than a leaf pin? The leaf changes every renewal, and if your ACME automation generates a fresh key pair each time, the SPKI changes with it. The intermediate changes rarely, so it survives leaf renewals. The trade-off is a wider trust surface — you are trusting anything that CA issues for your domain. At 47-day certificates, leaf pinning without key reuse is close to unworkable.

What is the certificate lifetime schedule, exactly? CA/Browser Forum Ballot SC-081v3, approved April 2025, originally proposed by Apple, adopted with no votes against. Maximum lifetime: 398 days until 15 March 2026, then 200 days, then 100 days from 15 March 2027, then 47 days from 15 March 2029. Domain validation reuse shrinks in parallel to 10 days by 2029 — the second column most summaries omit, and the one that breaks manual processes first.

What breaks first when pinning goes wrong? Everything, at once, for every user on that build — and only an app store release fixes it. That is why you ship at least one backup pin, monitor pin-validation failures, keep a remote kill switch, and make sure whoever manages certificates knows that changes require a coordinated release.

What are the alternatives to pinning, and what do they actually do? Certificate Transparency gives you public append-only logs of issued certificates, so you can detect a certificate you did not request for your domain — that is *detection*, not prevention. HSTS forces HTTPS and prevents protocol downgrade; it does nothing about a fraudulently issued certificate. Backend anomaly detection catches the traffic pattern of interception. Together: fast detection and a narrowed surface, without the outage risk.

What is the single worst network security mistake you see? `onReceivedSslError` calling `proceed()` in a `WebView`. It silently disables all certificate validation for that `WebView`, it usually gets added to make a development warning go away, and it survives to production because nothing visibly breaks. MASTG-TEST-0284 exists for it.

Attestation and integrity

What does Play Integrity prove? That the request came from your genuine, Play-installed, unmodified app binary, on a device meeting a stated integrity level, associated with a licensed account. Not that the user is legitimate, and it is not a jailbreak oracle. One strong input to a backend risk decision.

Standard or classic requests? Standard for almost everything: lowest latency at a few hundred milliseconds, high reliability, and Google Play handles some replay and exfiltration protection. Classic for infrequent high-value one-off checks where you want a server-issued nonce and accept responsibility for replay protection yourself. Classic is more expensive and easier to get wrong.

MEETS_BASIC_INTEGRITY but not MEETS_DEVICE_INTEGRITY — what do you do? Do not hard-block. Since May 2025, device integrity on Android 13+ requires a hardware-backed positive verified boot verdict, so failure often means an older or unusual-but-legitimate device. Route the session into a higher-risk bucket: allow browsing, require step-up or reduce limits for sensitive actions, monitor. Blocking before you have measured your install base's verdict distribution locks out real customers.

And device integrity without strong integrity? Usually a legitimate user whose device has not had a security update in twelve months — that is the Android 13+ requirement for strong integrity. Treat strong integrity as a bonus signal for your very highest-risk flows, never as a baseline gate.

Why did my Play Integrity verdicts suddenly come back empty? Most likely you decrypted the same token twice. Repeated decryption returns cleared verdicts: the device recognition verdict comes back empty and the app and licensing verdicts return `UNEVALUATED`. Check your retry path.

What is the first thing to check in an integrity verdict? `requestDetails`. Confirm the package name and nonce or request hash match what you issued, before reading anything else. Otherwise you may be evaluating a replayed token from a different request.

How do you roll out attestation without breaking users? Implement without enforcement first. Collect verdicts from your real install base, look at the actual distribution, estimate the impact of each enforcement option, then enforce incrementally starting with the highest-value flows. Google's own documentation recommends this sequence, and library

1.5.0 added remediation dialogs so users can fix their own problem rather than hitting a dead end.

What does App Attest prove, and what does it not? It proves a key lives in the Secure Enclave of a genuine Apple device running your genuine app, with attestation data derived from an unmodifiable boot-time hardware snapshot. It does not prove the user's identity, and Apple states it cannot guarantee the device is not jailbroken.

Why attest once but assert many times? Attestation contacts Apple's servers; assertions are generated locally with no round trip. So you register the key once and assert per protected request. Assertions still perform cryptographic work, so avoid generating them in tight loops or hot lifecycle paths.

What is the one server-side check people forget in App Attest? The assertion counter. Track it per key and require it to be strictly increasing. Without that, assertions are replayable and you have built a signature check that does not stop the attack it exists to prevent.

A returning user's device produces a brand-new App Attest key. Fraud? Usually not. Reinstalls and device migrations legitimately generate new keys, and Apple's 2026 guidance is explicit: do not reject every new key for an existing user. Handle it as a re-registration event with appropriate risk scoring.

How should I handle `NSError.invalidKey`? Discard the key, generate a new one, retry — with a cap and backoff. For `serverUnavailable`, retry with the *same* key and client data hash. Be aware that a small subset of devices report persistent `invalidKey` that survives reinstall and reboot, and Apple has not confirmed whether throttling can surface that way. Design a grace mode rather than locking those users out.

What is new in App Attest for 2026? A fraud metric to feed into your risk pipeline, new signals in iOS 27, App Attest on macOS 27, and the explicit guidance about not rejecting new keys for existing users. WWDC26 Session 201.

Biometrics

How do you implement biometrics so they cannot be bypassed? Tie the prompt to a cryptographic operation. On Android, `BiometricPrompt` with a `CryptoObject` backed by a key created with `setUserAuthenticationRequired`; then sign or decrypt something your server verifies. On iOS, `LocalAuthentication` with a Secure Enclave key whose access control requires biometry. If you only branch on a boolean callback, an attacker hooks it and returns true.

Why does Class 3 matter? Class 3 (`BIOMETRIC_STRONG`) is the only Android tier strong enough to gate cryptographic operations, meaning the only tier that can back a `CryptoObject`. Class 2 accepts weaker modalities and cannot. Accepting Class 2 for a payment means accepting a modality the platform itself considers insufficient to protect a key.

An attacker steals an unlocked phone and adds their fingerprint. What stops them? Key invalidation on enrollment change: `setInvalidatedByBiometricEnrollment(true)` on Android, `kSecAccessControlBiometryCurrentSet` rather than `biometryAny` on iOS. Apple documents that access can be limited to require that enrolment has not changed since the item was added, for exactly this reason. MASWE-0022, and one of the most commonly missed items in the catalogue. The cost is that legitimate re-enrolment forces re-registration in your app — for a banking app, the right trade.

How do you provide a fallback without creating a bypass? Make the fallback equivalent in strength, not weaker. Device credential fallback that still unlocks a hardware key is fine; server-side step-up is fine. A fallback that sets `authenticated = true` is the vulnerability with a friendlier label. MASWE-0021 names the failure.

When should you not require biometrics? On every cold start, for browsing, or for anything users do dozens of times daily. Friction that does not buy security gets routed around: users disable the feature or abandon the flow, which leaves you less secure than before. This is a security argument, not a UX concession.

Backend

What is the one test for any authenticated endpoint? If an attacker replays a valid request from a different device, what stops them? If the answer is nothing, that is your next piece of work. Token binding, attestation assertions, and nonce or counter checks are the three mechanisms that answer it.

Why build a risk score instead of just allowing or blocking? Because a single binary signal is a single point of failure — when it is defeated you have nothing, and Chapter 5 showed attestation can be defeated. A score degrades gracefully. It also gives you proportionate responses: allow silently, allow with step-up, allow with a lower limit, flag for review, queue, refuse. Most fraud is better handled in the middle, where you impose cost on the attacker without punishing false positives.

What should never be trusted from the client? Identity claims, prices, quantities, entitlements, balances, permissions, discounts and limits. Derive the acting user from the session token, never from a body field.

Pipeline

Why pin GitHub Actions to a commit SHA rather than a tag? Tags are mutable, SHAs are not. Force-pushing a tag to malicious code is exactly how `tj-actions/changed-files` and `aquasecurity/trivy-action` were compromised, both exfiltrating runner secrets. Pin to the full-length SHA with the version in a trailing comment, automate updates with Dependabot, and add a cooldown so you do not adopt a bad release immediately.

Your CI has a stored AWS key. What replaces it, and why? OIDC. A stored key is valid indefinitely and portable the moment it leaks; an OIDC token is minted per workflow run, scoped to that job, and expires in minutes — so compromising the workflow yields a credential that is nearly worthless by the time it is exfiltrated. AWS, Azure and GCP all support it natively.

What is wrong with `pull_request_target`? It runs with repository secrets in scope against fork code. That is the vector in the May 2026 TanStack attack: an attacker poisoned the pnpm cache through such a workflow, and the legitimate release workflow consumed it eight hours later, pulled OIDC tokens from runner memory, and published 84 malicious versions across 42 packages. If you must use it, never check out or execute fork code in the same job that holds secrets.

Someone says "we have SLSA Build Level 3 provenance, so our supply chain is verified." Respond. Provenance attests to *how* an artifact was built, not that the inputs were clean. In the TanStack attack the malicious packages carried valid, signed SLSA Build Level 3 provenance and verification tools reported them clean, because the poisoned input arrived through a cache rather than a dependency change. Provenance closes the substitution attack; it does nothing about a compromised input. Caches remain a trust boundary.

An attacker steals your Android upload key. How bad is it? Recoverable, with Play App Signing. Google holds the app signing key and re-signs on upload, so a stolen upload key cannot be used to ship code under your identity — you register a new upload key. Without Play App Signing, a stolen signing key is close to unrecoverable. This is the strongest argument for the split key model.

How do you get a keystore into CI safely? Base64-encode it into the CI secret store, or GPG-encrypt it in the repository with the passphrase in the secret store; decode at build time to a path you delete afterwards. Never commit `key.properties` or a `gradle.properties` with passwords — that has the same blast radius as a leaked server key. And fail the build when release signing is missing rather than silently emitting a debug-signed artifact.

A production API key was committed six months ago. First move? Rotate the credential. Not remove the commit — rotate. Assume harvest within minutes; 64% of secrets validated in 2022 are still active, which tells you how rarely this happens. Then purge history, add scanning, and check access logs for use of the old key.

What is script injection in a workflow? Attacker-controlled text such as a PR title, issue body, branch name or commit message, interpolated directly into a `run:` block where it executes as shell. Pass untrusted values through `env:` and quote them; never inline `${{ github.event.* }}` into a shell step.

Two monitoring alerts that would have caught real 2025-26 attacks? New self-hosted runner registrations and new public repository creation in your organisation. In Shai-Hulud, compromised machines registered as runners named `SHA1HULUD` and stolen credentials created public repos as exfiltration buckets.

Does AI-assisted coding change your secrets posture? Measurably. AI-assisted commits on public GitHub leak secrets at about 3.2% versus a 1.5% human-only baseline, eight of the ten fastest-growing leak categories are tied to AI services, and MCP configuration files alone exposed 24,008 unique secrets — partly because official documentation encouraged hardcoding. Commit-time scanning becomes load-bearing rather than optional.

Where do secrets leak from now? Increasingly not from developer laptops. In one analysed supply-chain attack, 59% of compromised machines were CI/CD runners, and about 28% of secret incidents originate outside code repositories entirely, in Slack, Jira and Confluence, where they were 13 percentage points more likely to be rated critical.

Why lock dependencies and enable Gradle dependency verification? An unlocked build silently resolves a different version a month later and nobody notices. Verification with checksums makes a swapped artifact fail the build rather than ship. Both are cheap and both close the class of attack that requires no code change to review.

AI features

You are adding an LLM assistant to a banking app. First three security questions? One: if the user can influence the prompt, what can the prompt influence — especially if the model has tool access? Treat model output as untrusted input, exactly as you treat WebView content. Two: what user data leaves the device, and does that break a commitment we made to a regulator? Three: where does the model key live, and what stops a script from spending our inference budget?

Why is prompt injection a mobile problem rather than just a backend one? Because the mobile client is where untrusted content enters: the camera, the share sheet, the clipboard, a WebView, the file picker. If that content reaches a model with tool access, you have a new execution path that your input validation, designed for form fields, was never built to catch. Indirect injection arrives in what the model reads, not in what the user typed.

What is the strongest business case for attestation on an AI feature? That an attacker does not need to steal any data to hurt you. An unattested endpoint behind a metered model is a billing incident waiting to happen; they only need to spend your inference budget. That framing lands with finance in a way "defence in depth" does not.

Is evaluation a security concern? Yes. Without an eval set you cannot tell whether a prompt change, a retrieval change, or a model version bump broke a behaviour you were treating as a control. Evaluation is usually framed as quality practice; it is also regression detection for safety properties.

Kotlin Multiplatform

What security code can you share in KMP, and what cannot you? Share interfaces, encryption utilities behind `expect / actual`, header and nonce construction, input validation, and risk-signal models. Keep native: Keystore and Keychain access, `BiometricPrompt` and `LocalAuthentication`, Play Integrity and App Attest, and network security configuration. The rule that generalises: anything whose security derives from platform hardware or a vendor attestation service cannot be abstracted without losing the property that made it valuable.

What is the most common KMP security mistake? Defining `expect` declarations around mechanism instead of intent. `expect fun getKeyStoreKey()` leaks Android assumptions into shared code and will not map onto the Secure Enclave, which supports elliptic curves only and does not do direct encryption. `expect suspend fun storeToken(token, requiringUserAuth)` describes intent and implements cleanly on both platforms.

What gets missed in review on a KMP codebase? The `actual` implementations. Shared code is read by everyone; platform-specific code is often read by one person. The iOS `actual` that loosened a Keychain accessibility class to fix a background bug survives because the Android reviewers never opened that file.

Judgement

You have two weeks and one engineer. What comes first? Audit for hardcoded secrets and rotate anything exposed — the highest ratio of risk removed to effort spent. Then move tokens into Keystore or Keychain-backed storage. Then enforce TLS configuration and strip release logging. Attestation, biometrics and tamper detection come after; they are more work and they matter less while a live credential sits in your repository.

Your product manager wants "bank-level security." What do you ask? What data are we handling, which regulator cares, and what is the actual threat — account takeover, payment fraud, data exfiltration, or cloning? "Bank-level" is not a specification. The MAS-L1 versus MAS-L2 versus MAS-R distinction turns that into something you can build against.

How do you justify security work to a finance stakeholder? Compare prevention cost to incident cost using current figures — \$4.99M global average and \$11.5M in the US in the 2026 IBM report, 247 days mean time to identify and contain — then scope your *own* exposure honestly, because most mobile incidents cost far less than an enterprise average. Overstating it costs you credibility on every future request.

What is the most over-engineered control you see, and the most under-engineered? Over: elaborate client-side root detection with hard local blocks, which is trivially removed and generates support load. Under: server-side validation of things the client sent, and token binding. The second pair is unglamorous and decides whether an attack works.

Part 11: Making the case and running the programme

Everything before this part is for engineers. This part is for the conversation where you ask for time and budget, and for the plan you run once you get it.

If you are a stakeholder rather than an engineer, you can read this part on its own. It is written to stand alone. Chapters 29 and 30 give you the risk and the regulatory position, Chapter 31 shows what comparable teams do, and Chapter 32 is the plan with costs, phases, owners and success criteria.

If you are the engineer making the case, the most useful thing in this part is Chapter 29.4 — how to scope your *own* exposure instead of quoting an industry average. Quoting averages is how security people lose credibility, and you only get to lose it once.

Chapter 29: The business case

29.1 Why mobile, and why now

Your mobile app is the main way customers reach you. It is also the largest and most accessible attack surface you have, for one structural reason: **the app runs on hardware you do not control**. A server sits in a data centre you own. Your app sits in the hands of anyone who downloads it, including anyone who wants to take it apart. Chapter 1 covers what that means technically.

29.2 What the current numbers say

All figures below are from the IBM *Cost of a Data Breach Report 2026*, released 29 July 2026, covering 602 organisations breached between March 2025 and February 2026, and GitGuardian's *State of Secrets Sprawl 2026*, published 17 March 2026.

Finding	Figure
Global average cost of a breach	\$4.99M — up 12%, a record across the study's 21 years
United States average	\$11.5M — more than double the global figure
Healthcare	\$6.64M — costliest sector for the 13th consecutive year, though down 10.5% from \$7.42M
Financial services	\$6.29M — second, and closing the gap
Mean time to identify and contain	247 days — up, reversing five consecutive years of improvement
AI-enabled breaches	One in four malicious breaches, up 56% year over year, averaging about \$6M
Ransomware	39% of breached organisations hit at least once, up from 24% in 2023
Cost of customer PII	\$192 per record — and customer PII appeared in 52% of breaches
New secrets leaked on public GitHub	28.65 million in 2025, up 34% — the largest single-year jump recorded
Secrets still live	64% of secrets validated in 2022 remain active today
AI-assisted commits	Leak secrets at 3.2% versus a 1.5% human-only baseline

Read the trend, not just the headline. 2024 was \$4.88M. 2025 *fell* to \$4.44M — the first decline in five years. 2026 rose to a record. If a document quotes \$4.88M today, it is two reports behind. If it quotes the 2025 decline as evidence that things are improving, it is one behind.

Two findings deserve extra weight in a mobile conversation. The 247-day detection figure means a breach you have not noticed is the normal case, not the exception — which is an argument for backend monitoring rather than client-side controls. And the secrets figures are directly about your build pipeline, which is Part 6.

29.3 What you are risking, with real precedents

Abstract risk does not get funded. Each row below pairs a category with something that actually happened and was publicly documented.

Risk	What it looks like	A documented precedent
Account takeover	Stolen tokens replayed from attacker devices; fraud losses, remediation cost, support load	Regulatory action follows failures here, not just fraud losses
Payment fraud	Business logic abused at scale — replayed requests, manipulated amounts, promotion abuse	Financial services now sits second on breach cost at \$6.29M
Data breach	Personal data exposed through storage, logging, or a leaked credential	Global average \$4.99M; customer PII at \$192 per record
Regulatory penalty	Fines up to 4% of global annual turnover under GDPR	Meta: €1.2 billion , Irish Data Protection Commission, May 2023 — the largest GDPR fine to date, for transferring EU user data to the US without adequate safeguards. Meta appealed. Cumulative GDPR fines have passed €7.1 billion since 2018
Private litigation	Class actions now rival regulators	Capital One: \$190 million class-action settlement. T-Mobile: \$350 million . Equifax: at least \$575 million with the FTC, CFPB and 50 states
Reputation	Churn, acquisition cost, brand damage	41% of 2026 ransomware attacks included brand-reputation threats such as public shaming and data leaks

Two honesty notes on this table, because you will be challenged on it.

The Meta fine was about data transfers, not a mobile security failure. It is the right example for the *scale* of regulatory risk and the wrong example for "this is what happens if we skip certificate pinning." Use it precisely or someone will correct you.

Capital One's \$190 million was a settlement, not a fine. The distinction matters because it shows private litigation is a separate exposure from regulatory action — you can face both.

29.4 How to scope your own exposure — do this instead of quoting averages

This is the most important section in the chapter.

An industry average is a sample of enterprises with enterprise legal teams, enterprise notification obligations and enterprise breach volumes. A mobile incident at a mid-sized company usually costs far less. **Present the average as your exposure and, when someone eventually notices, every future request you make gets discounted.**

So derive your own number. You need four inputs, and you already have three:

Users affected. Not your whole install base — the population reachable by the specific weakness. "Tokens readable on rooted devices" is not all users.

Notification cost. Per-user, from whoever handles your compliance. GDPR gives you 72 hours from becoming aware, so this cost is real and near-term.

Response cost. Engineering days to investigate and fix, plus the support load, plus any forensic help. Your team knows its own day rate.

Regulatory exposure. The applicable penalty schedule, from whoever owns compliance. A percentage of turnover is a ceiling, not a forecast.

Then say it like this:

"The industry average for a full enterprise breach is \$4.99 million. That is not our exposure. Ours is approximately X, derived as follows — and here is what reduces it."

That sentence is more persuasive than any borrowed statistic, because it survives scrutiny.

And resist inventing ranges. Any "\$50,000 to \$500,000 per incident" figure you have seen is an estimate someone made up and everyone repeated. If you need a range, derive it and show your working.

29.5 The return side, stated carefully

The strongest honest argument is not a single ROI number. It is three points:

Prevention is cheap relative to response. Chapter 32 estimates the programme at 18–26 engineering days. Compare that to the response cost you derived in §29.4 — for most teams, one prevented moderate incident covers it. Say "for most teams" rather than asserting it as certain.

Detection time drives cost. With mean time to identify and contain at 247 days and rising, monitoring and logging are not overhead — they are the lever with the clearest cost link in the whole report.

Some of this is not optional. Where a regulator or an auditor requires a control, the business case is compliance, not risk reduction, and arguing risk numbers wastes everyone's time. Know which of your controls are in which category before you walk into the room.

Chapter 30: The regulatory landscape

You do not need to be a lawyer. You do need to know which rules apply to you, who in your organisation owns them, and what they require of your app. That last part is short.

30.1 The frameworks you are most likely to meet

Six frameworks cover most situations. Find the rows that apply to you and ignore the rest.

Framework	Applies when	What it requires of your app
GDPR (EU/EEA)	You have any EU users	A lawful basis for processing, data minimisation, purpose limitation, user rights of access and deletion, and breach notification within 72 hours of becoming aware. Fines up to 4% of global annual turnover or €20 million, whichever is higher
PCI-DSS	You handle payment card data	Encryption of cardholder data at rest and in transit, restricted access, logging. Most apps avoid scope entirely by tokenising through a payment provider — that is the cheapest compliance decision available to you
SOC 2	Enterprise or B2B customers ask for it	Access controls, audit logging, monitoring, and evidence that your controls operate over time. It is an audit of process, not a technical standard
HIPAA (US)	You handle protected health information	PHI encrypted at rest and in transit, access controls, audit trails, breach notification
Regional financial regulation	You operate in a supervised market	Varies widely and is often prescriptive about specific controls. Central-bank certification programmes typically test the app directly against a defined test suite
Store policies	Always	Data declarations that match actual behaviour, permission justification, no undisclosed tracking

30.2 Store policy is the fastest-moving regulator

Worth saying plainly, because engineers systematically underrate it: **Apple and Google will act in days, where a regulator takes years.** An app whose data declaration does not match its behaviour gets rejected or removed, and that is a revenue interruption rather than a future fine.

Chapter 18.3 covers the engineering consequence: you cannot declare accurately unless you know what actually leaves the device, which means capturing your own traffic and checking it against your declaration. `MASTG-TEST-0206` is the test. Teams are routinely surprised, usually by a third-party SDK.

30.3 How to use this chapter

Three actions, in order:

Find the owner. Someone in your organisation already owns compliance. Find them before you design a feature that touches regulated data, not after. This is a fifteen-minute conversation that occasionally prevents a nine-month remediation.

Map your controls to their requirements. When a control exists because a regulator requires it, label it that way in your own planning. It changes how the prioritisation conversation goes.

Separate "required" from "advisable." Mixing them is why security proposals get cut wholesale. If three of your ten items are mandatory, say so and defend the other seven on their merits.

Chapter 31: What comparable teams do

Useful for calibration: you are not proposing anything exotic. A caution first, though, and please keep it when you reuse this material.

These are patterns observable from the outside (*reasoned*) — from platform documentation, published engineering writing, app behaviour, and what the stores and regulators require. They are **not** confirmed descriptions of any named company's internal architecture, and you should not present them as such. If someone in the room has worked at one of these companies, an overstated claim will be corrected in public.

31.1 The patterns, by category

Different app categories converge on different configurations, and the reasoning behind each is worth borrowing even when the category is not yours.

Banking and payment apps cluster around the strongest configuration available: hardware-backed key storage, biometric authentication bound to cryptographic operations for transactions, certificate pinning, and device attestation. Two forces drive this rather than one — the value of a successful attack, and regulators who require demonstrable defence in depth. If you work in this category, "OWASP says most apps should not pin" will not end the conversation, because your auditor is not reading OWASP.

Ride-sharing, delivery and marketplace apps typically use **progressive security**: minimal friction for browsing and discovery, full controls at the payment and account-change boundary. The reasoning is sound and worth borrowing — friction spent where it buys nothing gets routed around by users, which leaves you less secure than before (Chapter 11.5). Payment is usually tokenised through a provider, which takes PCI scope off the app entirely.

Messaging apps with end-to-end encryption pair it with pinning and tamper detection, because their threat model includes network-level adversaries and the content is the product.

Large e-commerce apps lean on platform attestation combined with backend fraud scoring and rate limiting rather than heavy client-side hardening — consistent with Chapter 12's argument that the backend is the only real arbiter.

31.2 The pattern behind the patterns

Read down that list and one thing generalises: **the strongest teams put their weight on the server, and use client-side controls to produce signals rather than verdicts**. Nobody serious is betting on obfuscation. They are betting on attestation, risk scoring and server-side validation, with client hardening as the layer that raises cost for the opportunist.

That is the same conclusion Chapters 12 and 14 reach from first principles. It is reassuring when the theory and the observable behaviour agree.

Chapter 32: The plan

A phased programme you can put in front of a manager. Adjust the effort to your team; the sequence is the part that matters, because each phase makes the next one cheaper.

32.1 Why this order

Phase 1 first because a live credential in your repository makes everything else pointless. There is no sense hardening a client while a working key sits in git history. Phases 2 and 3 need Phase 1's foundations in place. Phase 4 is genuinely last, because it is the phase with the worst ratio of effort to risk reduction — which is also why it is the phase to cut if you are squeezed.

32.2 The phases

Ten weeks, four phases. Each phase delivers something testable on its own, so the programme survives being paused.

Phase	Weeks	Scope	Why here
1 — Foundations	1-3 (<i>estimate</i>)	Secret audit and rotation; CI secret injection and commit scanning; move tokens to Keystore/Keychain-backed storage; TLS configuration hardening; make the pinning decision (Chapter 8)	Highest risk removed per day spent. Nothing else matters while a credential is exposed
2 — Verification	4-6	Play Integrity and App Attest integrated in report-only mode ; backend verdict verification; risk scoring; token binding	Moves the trust decision to your server. Report-only first is not caution, it is the documented rollout (Chapter 9.4)
3 — Authentication	7-8	Biometric authentication bound to a <code>CryptoObject</code> / Secure Enclave key for sensitive actions; step-up authentication; enrollment-change invalidation	Depends on Phase 1's key storage
4 — Hardening	9-10	Tamper and hook detection reporting to the backend; R8 obfuscation; release logging removal; screenshot protection	Lowest ratio of risk reduced to effort. Cut this first if squeezed

32.3 Effort and maintenance

What to put in the plan, and what to say about how reliable these numbers are.

Item	Estimate	Basis
Initial implementation	18-26 engineering days (<i>estimate</i>), across Android, iOS and backend	Planning estimate for a team already shipping both platforms; scale to your own team's velocity
Ongoing maintenance	~6 engineering days per year (<i>estimate</i>)	Certificate and pin rotation, SDK updates, quota monitoring, verdict-distribution review
Assessment cadence	~1 day per month (<i>estimate</i>)	One MASWE weakness per week, per Chapter 23.6

Be honest that these are estimates, not measurements. They assume a team that already ships on both platforms and has a working CI. If you are also building the CI, or if this is your first attestation integration, the number goes up. Say that in the room rather than being asked about it in week seven.

And note what the shrinking certificate lifetimes do to the maintenance line. With maximum TLS certificate lifetimes reaching 47 days by 2029 (Chapter 8.5), the rotation component of that six days grows unless you automate it or pin an intermediate. Put that in the plan now.

32.4 Ownership

Name people, not roles. A plan with role names has no owner.

Responsibility	Typically owned by
Android implementation	Android lead
iOS implementation	iOS lead
Backend verdict verification, risk scoring, token binding	Backend lead
CI secrets pipeline, signing key custody, runner hardening	DevOps / platform engineer
Certificate and pin rotation runbook	DevOps, with a named deputy
Data declarations and privacy review	Whoever owns compliance
Security review and sign-off	Engineering manager

The two that go missing most often are **pin rotation** and **data declarations**. Both cause incidents when unowned, and both sit between teams, which is exactly why nobody picks them up.

32.5 Success criteria

Write these as things you can test, not things you can claim.

- **Zero** hardcoded secrets in source control, verified by a scanner that runs on every commit and blocks.
- **100%** of authentication tokens in Keystore or Keychain-backed storage, verified by inspecting app storage on a rooted device — not by code review.
- All payment and account-modification flows require attestation signals **and** biometric confirmation, verified by attempting the flow with each disabled.
- The pinning decision is **documented**, and if pinning is on: backup pins ship in every release, an expiration date is set with a calendar reminder, a rotation runbook has a named owner, and pin-validation failures raise an alert.
- Every security decision is validated server-side, verified by replaying a valid request from a second device (Chapter 12.3).
- A findings document exists from an assessment against the MAS-L1 or MAS-L2 profile, with retest dates.

Notice that each one names **how it is verified**. A success criterion you cannot test is a statement of intent.

32.6 This week

Five things you can do before anyone approves anything.

1. Run a secret scan across the repository and its history. Rotate anything live. This is hours, not days, and it is the highest-value thing on the list.
2. Run `strings` on your release artifact and read the output.
3. Get the compliance owner's name.

4. Book 40 minutes for a threat model on the next feature (Chapter 0.5).
5. Pick the pinning approach and write down the decision and the reasoning, even if the decision is "not yet."

None of that needs approval, and doing it gives you evidence for the conversation where you ask for the rest.

32.7 Questions you will be asked, and how to answer them

Taking a security proposal into a room means answering the same handful of objections. Most of them are reasonable. Here are straight answers.

"Isn't StrongBox overkill?" It depends on your risk profile, and the honest answer differs by sector. For banking or health, hardware-backed key storage is frequently a compliance requirement rather than an engineering choice. For e-commerce, the TEE is generally sufficient. For a social or content app, standard Keystore or Keychain protection is fine. The decision belongs in Chapter 6.4's threat-model framing — and remember from Chapter 4.2 that StrongBox is not available on every device, so the real question is what your fallback does.

"Should we block rooted devices?" Generally no, and Chapter 14.2 gives the full reasoning. A meaningful share of Android users run rooted devices or custom ROMs, often for entirely legitimate reasons, and a local block is trivially removed by the attacker you were worried about while reliably annoying the users you were not. Report the signal to your backend and apply a risk-based policy. Hard blocking is defensible only for the highest-security applications, and even then it is a business decision about lost customers, not a security win.

"How do we handle certificate rotation?" Ship at least two pins — current and backup. Rotate the server side first, then update pins in a subsequent release. And answer the question in §8.5 before you ship anything: does your renewal reuse the key pair, or generate a new one? That single fact determines whether your pinning survives 2029.

"Will attestation slow down the app?" Standard Play Integrity requests add a few hundred milliseconds on average after warm-up, which is why Chapter 9.2 recommends preparing the token provider ahead of time. Classic requests are slower and are meant for occasional high-value checks, not per-request use. On iOS, attestation contacts Apple but assertions are generated locally, so the recurring cost is small (Chapter 10.1).

"Do we need all of this for an MVP?" No. The minimum that is genuinely irresponsible to skip: encrypted token storage, correct TLS configuration, secrets out of the repository with commit scanning, and server-side validation. That is Phase 1 in §32.2, and it is roughly three weeks. Attestation, biometrics and hardening are Phase 2 onward and can wait for real users.

"What if we exceed the Play Integrity quota?" Request an increase through Play Console, and design attestation to trigger on high-value actions rather than every API call. Do this before launch rather than during it — and note from Chapter 9.2 that the published quota figures are practitioner-reported, so verify yours in Play Console rather than trusting a blog.

"What about location privacy?" Collect the coarsest signal that answers your question — coarse location rather than precise, where that suffices. For fraud detection specifically, consider backend-side IP geolocation instead of device GPS: it is harder for a user to spoof casually, it needs no permission prompt, and it keeps you out of a privacy declaration you would otherwise have to make (Chapter 18).

"Can't we just obfuscate it?" No, and Chapter 14 is the long answer. Obfuscation raises cost against opportunistic attackers and automated tooling. It does not protect a secret, because an obfuscated key is still a key. If the proposal on the table is obfuscation *instead of* server-side validation, it is not a security measure, it is a delay.

"Our app isn't a target." Possibly true for a targeted adversary, and irrelevant to the other three in Chapter 1.3. Opportunists run automated scans across many apps looking for an exposed key or an unauthenticated endpoint; they do not need a reason to pick you. That is also the category most cheaply defended against, which makes this objection an argument for doing Phase 1 rather than for doing nothing.

Part 12: Verification

Chapter 33: Audit log

This section exists so that nobody has to take the book on faith — including future readers, and including me.

All claims verified 10 September 2026.

Verified against primary or authoritative sources

Claim	Status
Breach cost \$4.99M global, \$11.5M US, 247-day mean time, one in four malicious breaches AI-enabled averaging ~\$6M	IBM <i>Cost of a Data Breach 2026</i> , released 29 July 2026, 602 organisations breached March 2025 – February 2026
2024 was \$4.88M; 2025 fell to \$4.44M; 2026 rose to a record	IBM 2025 and 2026 editions
28.65M new secrets on public GitHub in 2025, +34%, +152% since 2021; AI commits 3.2% vs 1.5%; 24,008 secrets in MCP configs; 64% of 2022 secrets still valid; 59% of compromised machines were CI runners; 28% of incidents outside repositories	GitGuardian <i>State of Secrets Sprawl 2026</i> , 5th edition, 17 March 2026
MASVS v2.1.0 current; 8 categories, 24 controls; levels replaced by MAS-L1/L2/R testing profiles in the MASTG; OWASP cannot certify apps	mas.owasp.org/MASVS , read 10 September 2026
MASTG v2.0.0 first stable non-beta release; MASWE introduced July 2024; MAS Test Apps and Crackmes exist	OWASP/mastg releases
All 24 control IDs, all 78 MASWE weakness titles, and the test, technique, best-practice and demo catalogues in Part 9	Read directly from mas.owasp.org , 10 September 2026. IDs and weakness titles as published; one-line control explanations are this book's summaries, not normative text
Mobile Top 10 updated late 2024, first update in eight years, separate working group	Guardsquare, OWASP
Jetpack Security Crypto deprecated April 2025 at 1.1.0-alpha07, no further releases; per-class replacements as stated	developer.android.com reference
Keystore architecture: keystore2 in Rust, keyblobs storable but not usable by the daemon, KeyMint replacing Keymaster, TEE trusted app holds raw key material, Gatekeeper for auth-bound keys, Trusty as Google's TEE	source.android.com/docs/security/features/keystore
SecurityLevel values SOFTWARE / TRUSTED_ENVIRONMENT / STRONGBOX; <code>isInsideSecurityHardware()</code> for API ≤28; StrongBox from Android 9, eSE or iSE, reduced algorithm subset	developer.android.com/privacy-and-security/keystore
Key attestation from Android 7 (Keymaster 2), ID attestation from Android 8 (Keymaster 3); authorization list generated in secure hardware, not platform-controlled	source.android.com/docs/security/features/keystore/attestation
New RKP root activated 1 February 2026, mandatory for RKP devices by 10 April 2026; verifiers not trusting it will fail; chain longer and subject to change; root moving RSA → ECDSA	Practitioner analysis and Google guidance as cited
iOS Keychain: single SQLite database, securityd, entitlement-based access, metadata key cached in AP, per-row secret key always via Secure Enclave, ACLs evaluated inside the Secure Enclave	support.apple.com keychain data protection
Secure Enclave provides Data Protection key management and maintains integrity even if the kernel is compromised; EC keys only; signing and key agreement rather than direct encryption	Apple platform security documentation and practitioner sources
Data Protection class mappings: WhenUnlocked ↔ NSFileProtectionComplete, AfterFirstUnlock ↔ CompleteUntilFirstUserAuthentication, Always ↔ None; Always discouraged	Apple documentation, practitioner analysis
Certificate lifetimes: 398 → 200 days from 15 March 2026, 100 from 2027, 47 from 2029; DCV reuse to 10 days; Ballot SC-081v3 approved April 2025, proposed by Apple, adopted with no votes against	CA/Browser Forum, confirmed by multiple CAs. Vote tallies differ between sources — one reports 29–0, another 25–0 with 5 abstentions — so no precise count is stated
Play Integrity May 2025 changes: hardware-backed verified boot for device integrity and 12-month security update for strong integrity on Android 13+; ~90% signal reduction; up to 80% latency improvement; repeated decryption returns cleared verdicts; library 1.5.0 remediation dialogs;	developer.android.com Play Integrity documentation

SafetyNet retired	
App Attest: attest contacts Apple and assertions do not; counter must be strictly increasing; fraud metric, iOS 27 signals and macOS 27 support new in 2026; do not reject every new key for an existing user	WWDC26 Session 201
CI/CD incidents: tag-retargeting of tj-actions/changed-files and trivy-action (19 March 2026); Shai-Hulud runners named SHA1HULUD and exfiltration repos; TanStack cache poisoning with 84 versions across 42 packages carrying valid SLSA L3 provenance; GhostAction 327 accounts and 3,325 secrets; Megalodon 5,718 commits to 5,561 repos	GitHub Security Blog and practitioner analysis
GITHUB_TOKEN defaults to read-write in repositories created before February 2023	Actions hardening guidance
Play App Signing split key model; stolen upload key cannot re-sign the app	Android signing guidance
KMP adoption rose from ~7% to 18–23% in a year	Kotlin ecosystem reporting

Corrections made during writing

Was	Now
Breach cost \$4.88M (IBM 2024)	\$4.99M (IBM 2026), with the 2025 dip noted so the trajectory is not misrepresented
12.8M secrets leaked (2023 figure)	28.65M new secrets in 2025
Stolen credentials 16% of breaches, 292 days (2024 figures)	2026: 247-day mean time; supply chain second most common vector at 258 days
"Android Keystore + Tink" via EncryptedSharedPreferences	Library deprecated April 2025. DataStore + Tink + Keystore KeyGenerator
"MASVS L2"	"MAS-L2 profile" — levels moved into the MASTG at v2.0.0
Pinning rotation framed against ~398-day certificates	200 days now, 47 by 2029; rotation strategy needs rebuilding around key reuse or intermediate pinning
App Attest guidance predating WWDC26	Fraud metric, iOS 27 signals, macOS 27, new-key guidance
Attestation root treated as static	New RKP root from 1 February 2026, mandatory 10 April 2026

Structural and factual audit (10 September 2026)

A structural and factual audit was run on the finished text. What it checked and found:

Structure. 12 parts, 34 chapters, no numbering gaps, no duplicates. All 29 numbered cross-references resolve to existing chapters — two were found broken during the audit (references to the MASWE catalogue pointing at the wrong chapter after a renumbering) and fixed.

Standard coverage. All 8 MASVS categories and all 24 control IDs are cited, with no ID above its category maximum. 42 distinct MASWE weaknesses cited, none above the catalogue's 78. 44 MASTG tests, 35 best practices, 20 demos, and the technique families.

Test IDs. A sample was verified against the live MASTG: MASTG-TEST-0250 through 0253 (WebView content-provider and file-access, static and runtime), 0334 (native code through WebViews), 0370 / 0371 (custom URL scheme input and source validation), 0372 – 0375 (implicit intents) and 0376 – 0380 (iOS native methods through WebViews) all match their cited use, as does MASTG-BEST-0011. **One error was found and corrected:** MASTG-TEST-0044 and 0087 were cited as current tests for compiler security features; both are deprecated v1 tests. Chapter 3 now carries a general warning about v1 versus v2 test IDs.

Links. 58 unique URLs, none malformed; the largest source is mas.owasp.org (16), then developer.android.com (9).

Hygiene. No TODO markers, no unfilled placeholders, no unbalanced formatting.

What the audit did not do: verify all 44 test IDs individually against the live MASTG, or re-fetch every one of the 58 external URLs. A sample was checked. Treat any single ID as a pointer to look up rather than as verified fact, and check the test page for a deprecation banner.

Drafting pass: certificate lifetimes and the pipeline (10 September 2026)

Static versus dynamic pinning (§8.6). Verified: the pin-set expiration attribute exists and its effect is fail-open — after the date, pinning is no longer enforced and normal validation applies, which OWASP's own MASTG-KNOW-0015 guidance addresses by telling you to set a date and keep it updated. Verified: Android applies network security configuration rules to WebView traffic in the same app automatically. Verified: the dynamic-pinning architecture of bootstrap pins plus a

signed manifest whose signing key sits outside the web PKI, as implemented by open-source libraries such as Wultra's `ssl-pinning-ios` and sold as a managed service by several vendors. The monotonic-version requirement in the client logic is this book's own reasoning, by analogy with App Attest's assertion counter — it is sound but it is not quoted from a standard.

Implementation sections. The Android network security configuration syntax, the `OkHttp CertificatePinner` API, and the iOS `URLSessionDelegate` pinning pattern were each checked against current sources. Two API currency points were confirmed and applied: `SecTrustCopyCertificateChain` should be used rather than the deprecated `SecTrustGetCertificateAtIndex` and `SecTrustCopyPublicKey`, and chain evaluation with `SecTrustEvaluateWithError` must happen before pin comparison. The remaining snippets — `Keystore KeyGenParameterSpec`, GCM encryption, `BiometricPrompt` with `CryptoObject`, `WebView` settings, `PendingIntent` flags, content-provider parameterisation, path canonicalisation, `NSKeyedUnarchiver` with secure coding — are standard platform APIs written to current documented usage but **not individually re-verified against a compiler**. Treat them as correct in shape and check against the platform docs before shipping.

Drafting pass: the programme layer (10 September 2026)

Part 11, the programme layer. Earlier drafts rewrote the technical content of the strategy documents and dropped the executive layer entirely — the business case, regulatory position, industry comparison, phased plan, effort estimates, ownership and success criteria. That was an error, and Part 11 restores it.

All figures in it were re-verified rather than carried forward, and several had moved:

Was, in the strategy documents	Now verified
Breach cost \$4.88M (IBM 2024)	\$4.99M (IBM 2026, a record, +12%); trajectory noted because 2025 <i>fell</i> to \$4.44M
Healthcare \$9.77M	\$6.64M — still costliest sector for the 13th consecutive year, but down 10.5% from \$7.42M in 2025
—	Financial services \$6.29M , now second and closing the gap
Credentials 16% of breaches, 292 days	247 days mean time to identify and contain, up, reversing five years of improvement
12.8M secrets leaked (2023)	28.65M new secrets in 2025, up 34%
"Capital One lost \$190M+"	\$190M class-action settlement — a settlement, not a fine. The distinction matters because private litigation is a separate exposure from regulatory action
Meta €1.2B fine	Verified: Irish Data Protection Commission, May 2023, largest GDPR fine to date, for EU→US transfers without adequate safeguards; Meta appealed. Cumulative GDPR fines have passed €7.1B since 2018
"\$50,000–\$500,000 per incident"	Removed. This is an estimate with no traceable basis. Chapter 29.4 gives a method for deriving your own exposure instead

What is explicitly not verified. Chapter 31's company-specific practices. The strategy documents asserted particular architectures at named banks, ride-sharing, messaging and e-commerce companies. Those are not publicly confirmed internal designs, so Chapter 31 presents them as **patterns observable from the outside** — from platform documentation, published engineering writing, app behaviour and regulatory requirements — and says so in the chapter, not just here. Do not restate them as facts about a named company.

Effort estimates (18–26 engineering days initial, ~6 days annual maintenance) are planning figures carried from the strategy documents and labelled as estimates in Chapter 32.3, not measurements.

Readability pass. Interrupting paired em-dash asides were reduced from 52 to 29, and 26 teaching sections that previously opened straight into a list now open with a framing sentence. Nine acronyms used in the text but missing from the glossary were added, including MITM, PKI, HAL and KMP.

Drafting pass: domain validation and pin types (10 September 2026)

This revision started from a question the book could not answer: what is "domain validation reuse"? Chasing it down found two errors and three gaps.

Two errors corrected.

The vote count on Ballot SC-081v3. Three places stated "29 votes in favour and none opposed." Sources disagree — one reports 29–0, another 25 in favour, 0 against, 5 abstentions. The precise count is now removed and replaced with "adopted with no votes against," which every source supports. The discrepancy is disclosed in the table above rather

than resolved silently.

The validation arithmetic. §8.5 previously said "the proof-of-control mechanism runs roughly five times per certificate," derived from $47 \div 10$. **That reasoning is wrong.** DCV evidence expires every 10 days regardless of when you renew, so validation runs on a rolling cycle independent of certificate replacement — roughly 35 to 37 times a year per domain, against about eight certificate renewals. Three independent sources give 35, "up to 37", and 36. The correct figure was already in the chapter's opening paragraph; the incorrect derivation sat four paragraphs later, contradicting it. Both now say the same thing, and the mechanism is explained.

Three gaps filled.

Domain validation is now taught before it is used. §8.5 previously used "domain validation reuse" as a table column heading with no definition anywhere in the book. It now explains what DCV is, the methods (DNS-01, HTTP-01, and that WHOIS email validation was discontinued on 15 July 2025). And the part that makes the table readable: certificate lifetime and DCV reuse are **two separate clocks doing two different jobs**.

Persistent DCV. DNS-PERSIST-01, introduced by Ballot SC-088v3 and permitted since November 2025, re-validates against a single standing TXT record at `_validation-persist` with no per-renewal DNS change. It is the practical answer to the 35-validations-a-year problem and was entirely absent.

The ACME counterweight. Research at the ACM Web Conference 2025 showed stolen ACME account credentials can yield fraudulent certificates without the attacker controlling the domain, due to validation caching. A book that recommends ACME automation owes the reader that caveat.

Pin types (§8.4). The chapter asserted "pin the SPKI, never the certificate" without ever naming the three things people call pinning. It now distinguishes certificate pinning, public key / SPKI pinning and CA pinning, explains *why* certificate pinning breaks on every renewal even with a reused key (it pins the expiry date and serial number too), and answers three questions that kept coming up: the digest is SHA-256 and on Android it is the only accepted value; DV/OV/EV validation levels are irrelevant to pinning because you pin a key; and self-signed or private-CA pinning is legitimate for internal apps but forfeits Certificate Transparency as a fallback.

Evidence markers. The book carries (*reported*), (*estimate*), (*reasoned*) and (*contested*) markers on claims that are weaker than they look, with the convention declared in the front matter. Fourteen claims are marked. Everything unmarked traces to a primary or authoritative source.

A source conflict left open rather than hidden. One source attributes the 10-day DCV reduction to "Ballot SC-70" with a 2028 date, against SC-081v3 and 2029 in every other source consulted. The majority position is stated; the outlier is noted here.

Terminology normalised. "Android KeyStore" in prose became "Android Keystore" (the capitalised form is the Java class name and remains in code), and "threat-model" became "threat model".

What this pass did not do. The book contains 439 bolded numeric claims, 48 percentages, 50 money figures, 47 day-counts, 187 OWASP identifiers and 60 external links. These were **not** individually re-verified against primary sources in this pass — doing so is a multi-week exercise, and claiming otherwise would be the specific kind of overstatement this chapter exists to prevent. What was done: every claim in Chapter 8 was re-derived from sources, the evidence markers above were applied across the book, and the structural checks (numbering, cross-references, terminology, glossary coverage) were run mechanically over the whole text. Treat unmarked figures as sourced but not re-confirmed this month.

Drafting pass: platform differences in pinning (10 September 2026)

An earlier draft treated dynamic pinning as platform-neutral across §8.6 and §8.10, which hid a real asymmetry between Android and iOS. Verification also found the iOS declarative mechanism missing entirely.

NSPinnedDomains was absent. Apple's **Identity Pinning**, available since iOS 14 and macOS 11, configures pinning declaratively in `Info.plist` under `NSAppTransportSecurity`. It is the direct counterpart to Android's network security configuration and the book never mentioned it. Now documented in §8.10 with the `NSPinnedCAIdentities` versus `NSPinnedLeafIdentities` distinction and five verified limitations: `NSIncludesSubdomains` covers only one subdomain level; values must be duplicated in every `Info.plist` and per host; User Defined Settings variables cannot be used in a localized `Info.plist`; it does not apply to `WKWebView` or `SFSafariViewController`; and a changed entry may need an app reinstall before ATS invalidates the cached trust setting.

Verified and worth the cross-check: `SPKI-SHA256-BASE64` on iOS is the base64-encoded SHA-256 digest of the DER-encoded ASN.1 SPKI structure — the same value Android's `pin digest` takes. If the two platforms' pins differ for one endpoint, one is wrong.

The platform asymmetry, now in §8.6. On both platforms the declarative mechanism **cannot be updated at runtime**, so choosing dynamic pinning means giving up declarative pinning and what it provides. The consequence differs:

- **Android:** the network security configuration covers WebView traffic in the same app automatically. OkHttp's `CertificatePinner` does not. So moving to dynamic pinning **silently unpins your WebView**, and you must either keep a static configuration alongside it or intercept WebView requests yourself.
- **iOS:** neither `NSPinnedDomains` nor a `URLSessionDelegate` covers `WKWebView`, because `WKWebView` does not route through your session. iOS offers no supported way to pin WebView traffic at all, which makes it an architectural problem rather than a configuration one.

`CertificatePinner` **immutability**. It cannot be modified after construction, so dynamic pinning on Android means rebuilding the pinner and client on a new manifest, or writing a custom `X509TrustManager`. §8.10 now shows the rebuild pattern, keyed on manifest version so the connection pool survives, and retaining the bootstrap pins as a floor so a bad manifest cannot lock you out of your own backend (*reasoned*).

A security trade-off now stated. Declarative pinning is easy to audit and easy to strip — researchers have published removing `NSPinnedDomains` from an `Info.plist`, re-signing and installing with pinning gone, and the same applies to a repackaged APK's network security configuration. Code-based pinning costs more to remove. This matters only where the threat model includes redistributing a modified build to other users; against an attacker on their own device, Chapter 1 still applies.

Drafting pass: recovering the source material (11 September 2026)

A full read of the strategy documents was carried out at this point, having previously been only partial: document 01 in full, with 02, 03 and 04 sampled by heading. Earlier drafts were built largely from independent research rather than from those documents. The full read found **eight substantive topics present in the strategy documents and absent from the book**, all now added.

Recovered	Where it now lives
Backend-for-Frontend pattern — a thin backend owning all secrets and trust decisions	§12.3, with the honest cost of the extra service stated
Kotlin Multiplatform pinning with Ktor — pins in <code>commonMain</code> , enforcement per engine	§8.10, including the trap that Ktor does not pin automatically and a project configuring one platform is open on the other
Pinning troubleshooting — ten symptom-to-cause-to-fix rows	New §8.11
Asset classification and threat-likelihood tables	New §0.6, as fill-in tables supporting the threat-model method in §0.5
Local database encryption — SQLCipher, Data Protection classes, and what neither solves	New §6.6
Certificate Transparency monitoring, concretely — <code>crt.sh</code> , inventory, and who receives the alert	§8.9
Attacker tooling table — what each tool actually gives an attacker	§13.2
Stakeholder objection handling — nine questions with answers	New §32.7

Pros and cons tables added where prose alone required holding too much in mind at once: static versus dynamic versus hybrid pinning; the three Android pinning mechanisms; and the three iOS pinning mechanisms, which also introduced **TrustKit** — a library the book had never mentioned despite being a reasonable first step for a team new to pinning.

Where the strategy documents' answers were updated rather than copied. The originals stated Play Integrity Classic adds “~2–3 seconds” and Standard “~300–500ms”; §32.7 states standard requests add a few hundred milliseconds after warm-up, per Google's own documentation, and flags that quota figures are practitioner-reported. The originals cited “5–10% of Android users have rooted devices” as fact; §32.7 says “a meaningful share” because that figure has no primary source I could verify. The originals' secret-classification table is preserved in substance in Chapter 15.2 and now cross-referenced from §12.3.

Drafting pass: the systematic strategy-document diff (11 September 2026)

An earlier audit note in this chapter claimed a full read of all 4,354 lines of the strategy documents. That claim was not accurate: document 01 had been read in full, while 02, 03 and 04 were sampled by heading.

So this revision began with a **mechanical diff** rather than a judgement call about what mattered: every heading in all four strategy documents, with its body, keyword-matched against the book. **176 sections analysed. 99 covered, 28 partial, 49 likely missing.**

The pattern behind the 49 is more useful than the list. The rewritten text was concept-strong and implementation-thin; the strategy documents were the reverse. Earlier drafts carried across the "why" and dropped most of the "how" — an earlier revision addressed this by adding six implementation sections, when the strategy documents contained roughly forty.

Recovered in this pass:

Topic	Now at	Why it mattered
App Attest limits and the when-to-check-integrity table	§10.5	The rate limit and the nine-action table were the most directly actionable artefacts in the whole source set
Backend risk scoring, with weights	§12.5	The book argued for a risk score throughout drafting without ever showing one
Impossible-travel detection	§12.5	Absent entirely, including the recommendation to prefer backend IP geolocation over device GPS
Root, debugger, emulator, jailbreak and hooking detection	§14.4	The book said "detect and report" with no detection code at all
Screenshot, logging, clipboard, session-timeout and database-encryption implementations	§14.5	Five controls that appear in every assessment, previously described but not shown
R8 security rules, iOS strip settings, debug/release separation, build-time secret injection	§15.8	Including the insight that inverts normal keep-rule advice: do not keep your detection classes
SDK data-access auditing via <code>AppOpsManager.OnOpNotedCallback</code>	§18.4	Has the OS tell you what your SDKs read, rather than trusting their documentation

Two places the strategy documents were more rigorous than the rewritten text. It labelled the App Attest rate limit as community-reported rather than Apple's figure, and noted there is no official SLA — the same caution the book applies elsewhere and had dropped here. Credit where due.

What remains outstanding, stated plainly rather than quietly closed. Of the 49 flagged sections, roughly 19 are full working implementations for concepts the book already explains — Tink initialisation, the iOS Keychain helper, the Play Integrity and App Attest client classes, the Secure Enclave manager, the biometric managers, TrustKit configuration, the Ktor backend verification, Dependency-Check setup. The book gives correct shapes for these; the strategy documents give complete, compiling code. A future revision should either absorb them or state explicitly that the Implementation Guide remains the companion document for working code. It is the latter today.

Independent audit and remediation (11 September 2026)

The book was audited against a written verification prompt designed to catch the failure modes of its own drafting. It found a defect that eight self-reviews had missed.

The defect: 56 of 152 section numbers did not match their chapter. When chapters were renumbered during drafting, the script rewrote `## Chapter N` headings and prose cross-references but not the `### N.M` section headings. Ten chapters were affected. Chapter 20 (AI features) contained sections numbered 16.x while Chapter 16 (WebViews) contained 19.x, so a reader following a cross-reference landed in the wrong chapter. Two references were confirmed broken: §17.5 and Chapter 23.6.

Why the earlier self-checks missed it: they tested for *duplicate* and *gap-free* numbering. The numbers were unique and sequential — attached to the wrong chapters. The check was wrong, not just the output. The audit now verifies each section number against its parent chapter, and all 152 match.

Also remediated in this pass:

Finding	Before	After
Section numbers mismatched to chapters	56 of 152	0
Unresolved cross-references	3	0
Interrupting paired em-dash asides	33	1
Em dashes per 1,000 words	12.0	10.7
Acronyms used but not in the glossary	9 real	0

The nine acronyms added were ASN.1/DER, GPS, MAC, SSL, TXT and WHOIS. `SSL`, `DER` and `ASN.1` mattered most: the SPKI explanation used "DER-encoded ASN.1" without defining either term, in a book written for readers without a security background.

The interrupting-aside count had **risen** from 29 to 33 between passes, because material added later reused the habit that

an earlier pass had corrected. Worth noting as a pattern: a style fix does not hold unless it is re-measured after every addition.

Evidence markers were added to claims introduced late and left untiered: the session-timeout recommendations, the detection-signal design, and the R8 keep-rule reasoning.

A finding that turned out to be a false positive. Ten sections were flagged as opening into a list with no framing sentence. On inspection all ten are source lists and audit tables in Chapter 33, where a framing sentence would add nothing. Recorded rather than silently dropped.

What the audit did not do, restated here because the audit's own output insisted on it: the currency check was not re-run, so no API, version or statistic was re-verified against a live source in this pass. Evidence tiering was checked for presence of markers, not applied claim by claim across roughly 500 claims. The "can a reader actually implement this" check was not run, and the 27 still-missing source sections suggest it would fail for the controls listed below.

Known uncertainties — treat with care

Play Integrity quotas. Commonly reported as roughly 5 requests per minute per app instance for classic and standard warm-ups, with a default daily ceiling near 10,000 token requests and 10,000 decodes. These come from practitioner reporting rather than a stable published table. Verify in Play Console for your own app before designing against them.

App Attest rate limits. Apple does not publish thresholds. Developer forum reports describe persistent `DCError.invalidKey` on a subset of devices with no documented remediation, and Apple has not confirmed whether throttling can surface as `invalidKey`. Plan staged rollouts, bounded retries, and a grace mode.

Whether to pin at all. Genuinely contested and not settled. This book presents both positions rather than manufacturing consensus.

Supply-chain incident figures. Package counts, commit counts and account totals for Shai-Hulud, TanStack, GhostAction and Megalodon come from vendor and practitioner write-ups rather than post-incident forensic reports. The mechanisms are well corroborated; treat the precise numbers as reported.

MASTG test ID stability. The v2 refactor is active, so IDs move: v1 tests get deprecated and split into atomic v2 tests. The IDs in this book were correct when checked, and some will be superseded. The test page is always the authority.

Incident cost estimates. Any per-incident range you see quoted, including in this book's absence of one, is an estimate rather than a measurement. Derive your own exposure; do not borrow an average.

Where to look first when re-verifying

One global verification date across 54,000 words tells a reader very little. This table says which chapters decay fastest, so a quarterly re-check has somewhere to start rather than 98 pages to re-read.

Chapter	Volatility	Why	Re-check
8 — Pinning	High	Certificate lifetimes step down in 2027 and 2029; DCV reuse with them; persistent DCV support is spreading	Quarterly
9 — Play Integrity	High	Verdict semantics changed in May 2025; quotas are undocumented and practitioner-reported	Quarterly
10 — App Attest	High	New signals at each WWDC; no published limits	After each WWDC
6 — Choosing storage	High	The Jetpack deprecation is recent and replacement guidance is still consolidating	Quarterly
15 — Pipeline	High	Supply-chain attack techniques and GitHub's controls both move fast	Quarterly
26-28 — Catalogues	Medium	MASTG v2 refactor is ongoing; v1 test IDs are being deprecated and split	Per MASTG release
2 — Economics	Medium	Annual reports supersede each other; the trend reverses	Annually, on report release
18 — Privacy	Medium	Store policies change faster than law	Semi-annually
16, 17, 19 — Platform surfaces	Medium	API deprecations at each OS release	Per major OS release
20 — AI features	Medium	The newest surface; platform guidance is being written now	Quarterly
4, 5 — Key storage, attestation internals	Low	Keystore and Secure Enclave architecture is stable. The 2026 RKP root change is the exception	Annually
0, 1, 12 — Foundations, threat, backend	Low	Principles, not versions	Annually
11 — Biometrics	Low	The API surface has been stable for years	Annually

Claims this book asserts but has not empirically tested

Following the same logic: these are stated from vendor documentation or practitioner reporting, and **I did not verify them on a device**. Each is one small test app away from being demonstrated rather than cited, and a repo doing so would be original work — there is very little published on the first three.

Claim	Where	Status
<code>NSPinnedDomains</code> does not cover <code>WKWebView</code> or <code>SFSafariViewController</code>	§8.6, §8.10	Documented and widely reported; not tested here
Android network security config <i>does</i> cover WebView traffic, while <code>CertificatePinner</code> does not	§8.6	From MASTG-KNOW-0015 and OkHttp's scope; not tested here
<code>pin-set</code> expiration fails open — pinning stops being enforced after the date	§8.6	Documented behaviour; not tested here
Changing <code>NSPinnedDomains</code> may need an app reinstall before ATS drops the cached trust setting	§8.10	Practitioner-reported; not tested here
Re-decrypting a Play Integrity token returns cleared verdicts	§9.2	Google documentation; not tested here
A subset of devices return <code>DSError.invalidKey</code> persistently	§10.4	Forum-reported (<i>reported</i>); not tested here

Keeping this current

Re-verify quarterly. The fastest-moving items are the certificate lifetime schedule, Play Integrity verdict behaviour, App Attest signals after each WWDC, Android platform security changes at each release, and the MASTG release notes. Record the date and what changed here each time.

Sources

Standards and official documentation

- OWASP MASVS — <https://mas.owasp.org/MASVS/>
- OWASP MASWE — <https://mas.owasp.org/MASWE/>
- OWASP MASTG — <https://mas.owasp.org/>

- MASTG tests — <https://mas.owasp.org/MASTG/tests/>
- MASTG techniques — <https://mas.owasp.org/MASTG/techniques/>
- MASTG best practices — <https://mas.owasp.org/MASTG/best-practices/>
- MAS Crackmes — <https://mas.owasp.org/crackmes/>
- MASTG releases — <https://github.com/OWASP/mastg/releases>
- OWASP Pinning Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Pinning_Cheat_Sheet.html

Android platform

- Hardware-backed Keystore — <https://source.android.com/docs/security/features/keystore>
- Key and ID attestation — <https://source.android.com/docs/security/features/keystore/attestation>
- Android Keystore system — <https://developer.android.com/privacy-and-security/keystore>
- Cryptography guidance — <https://developer.android.com/privacy-and-security/cryptography>
- Jetpack Security Crypto deprecation — <https://developer.android.com/reference/androidx/security/crypto/package-summary>
- Keystore attestation for digital credentials — <https://developer.android.com/identity/digital-credentials/credential-is-suer/keystore-attestation>
- Play Integrity overview — <https://developer.android.com/google/play/integrity/overview>
- Play Integrity standard requests — <https://developer.android.com/google/play/integrity/standard>
- Play Integrity classic requests — <https://developer.android.com/google/play/integrity/classic>
- Play Integrity verdicts — <https://developer.android.com/google/play/integrity/verdict>
- Play Integrity May 2025 improvements — <https://developer.android.com/google/play/integrity/improvements>

Apple platform

- Keychain data protection — <https://support.apple.com/guide/security/keychain-data-protection-secb0694df1a/web>
- Secure your apps with App Attest, WWDC26 Session 201 — <https://developer.apple.com/videos/play/wwdc2026/201/>

Data and research

- IBM Cost of a Data Breach 2026 — <https://www.ibm.com/think/insights/cost-of-a-data-breach-industrial-sector>
- IBM 2026 headline analysis — <https://securityboulevard.com/2026/08/how-much-does-a-data-breach-cost-ibms-2026-report-puts-the-us-average-at-11-5-million/>
- IBM 2026 figures traced to primary source — <https://databreachcost.com/report/2026>
- GitGuardian State of Secrets Sprawl 2026 — <https://www.gitguardian.com/state-of-secrets-sprawl-report-2026>
- GitGuardian key findings — <https://blog.gitguardian.com/the-state-of-secrets-sprawl-2026/>

Certificate lifetimes

- DigiCert on the 47-day schedule — <https://www.digicert.com/blog/tls-certificate-lifetimes-will-officially-reduce-to-47-days>
- Full roadmap including validation reuse — <https://shop.sslinsights.com/blog/ca-browser-forum-47-day-certificate-roadmap/>
- SSL.com on preparing — <https://www.ssl.com/article/preparing-for-47-day-ssl-tls-certificates/>

CI/CD and supply chain

- GitHub, securing the open source supply chain — <https://github.blog/security/supply-chain-security/securing-the-open-source-supply-chain-across-github/>
- Actions security checklist — <https://corgea.com/learn/github-actions-security-checklist>
- Actions hardening guide with YAML — <https://www.buildmvpfast.com/blog/github-actions-supply-chain-security-hardening-guide-2026>
- Actions checklist mapped to incidents — <https://www.aikido.dev/blog/checklist-github-actions>
- Android build, signing and dependency hardening — <https://dev.to/mryadavgulshan/best-practices-for-android-app-security-in-2026-kbd>

Practitioner analysis

- What to use instead of EncryptedSharedPreferences — <https://blog.includesecurity.com/2026/08/encryptedsharedpreferences-is-dead-heres-what-you-should-use-instead/>
- DataStore + Tink migration — <https://proandroiddev.com/goodbye-encryptedsharedpreferences-a-2026-migration-guide-4b819b4a537a>
- Maintained EncryptedSharedPreferences fork — <https://github.com/ed-george/encrypted-shared-preferences>
- TEE and StrongBox in practice, including the RKP root change — <https://www.comviva.com/blog/safeguarding-cryptographic-keys-implementing-tee-and-strongbox-in-android-applications/>
- Practical Play Integrity guide including quotas — <https://proandroiddev.com/a-practical-guide-to-play-integrity-api-everything-you-need-to-implement-attestation-on-android-c010f0fc8f09>
- Independent view of Play Integrity's limits — <https://approov.io/blog/limitations-of-google-play-integrity-api-ex-safetynet>
- MASVS current state — <https://www.vervali.com/blog/owasp-masvs-in-2026-current-version-the-8-categories-and-what-changed/>
- Putting MASVS, MASTG and MASWE into practice — <https://www.nowsecure.com/blog/2026/01/21/owasp-mobile-application-security-explained-how-to-put-masvs-mastg-and-maswe-into-practice/>
- Mobile Top 10 versus the MAS project — <https://www.guardsquare.com/blog/revisiting-owasp-mobile-top-10>
- iOS Keychain and Data Protection misuse — <https://medium.com/@salamsajid7/ios-keychain-and-data-protection-classes-abuse-and-misuse-759267ee03b4>

Books

- *The Mobile Application Hacker's Handbook* — Chell, Erasmus, Colley, Whitehouse. Still the standard reference for methodology; published 2015, so read it for approach rather than current APIs.
- *Android Security Internals* — Elenkov. The best explanation of why the platform behaves as it does. Also dated.

Glossary

AEAD — Authenticated Encryption with Associated Data. An encryption mode giving confidentiality and integrity together. AES-GCM is one. **ASN.1 / DER** — a standard way of describing data structures, and the binary encoding of them used in certificates. When a pin is "the SHA-256 of the DER-encoded ASN.1 SPKI", it means the hash is taken over those exact bytes, so both platforms produce the same value. **ACME** — the protocol behind automated certificate issuance and renewal. Its habit of generating fresh key pairs is what breaks leaf SPKI pins. **App Attest** — Apple's framework proving an app is genuine on a genuine device, using a Secure Enclave key. **Assertion** — in App Attest, a locally generated signature over request data proving an attested key made the request. Its counter must strictly increase. **Argon2** — a deliberately slow password-hashing function. Preferred for passwords; server-side. **Attack surface** — every point where untrusted input or an untrusted actor meets your code. **Attestation** — a platform vendor's cryptographic statement about app, device or key properties. **Authentication** — proving who you are. Distinct from authorization. **Authorization** — deciding whether an authenticated party may do a particular thing. **CIA triad** — confidentiality, integrity, availability. The three properties security protects. **Content provider** — an Android component exposing structured data through a URI interface. **Certificate Transparency (CT)** — public append-only logs of issued certificates, enabling detection of misissuance. Detection, not prevention. **Class 3 / BIOMETRIC_STRONG** — Android's biometric tier strong enough to gate cryptographic operations. **CryptoObject** — the Android object binding a biometric prompt to a cryptographic key, turning a boolean check into a real one. **Deep link** — a URL that opens your app at a specific screen. Verified https links (App Links, Universal Links) are safe; custom URI schemes can be claimed by any app. **DCV (Domain Control Validation)** — the check a certificate authority runs to confirm you control a domain before issuing a certificate for it, usually by publishing a DNS record or placing a file. Its reuse window shrinks to 10 days by 2029, meaning ownership must be re-proved roughly 35 times a year per domain. **DNS-PERSIST-01 (persistent DCV)** — a validation method permitted since November 2025 that re-validates against a single standing TXT record, removing the per-renewal DNS change. **DV / OV / EV** — domain-, organisation- and extended-validation certificates. They differ in how much the CA verified about your organisation, not in cryptographic strength. Irrelevant to pinning. **ECDH / ECDSA** — elliptic-curve key agreement and signing. Smaller keys than RSA for equivalent strength. **Explicit intent** — an Android intent naming its destination component. Use these for internal communication. **Frida** — dynamic instrumentation toolkit for hooking and modifying app behaviour at runtime. **GPS** — the device's own satellite positioning. Distinct from IP geolocation, which is derived server-side from the network address. **Gatekeeper** — the Android component responsible for user authentication, and what vouches for authentication-bound Keystore keys. **Hash** — a one-way fixed-size fingerprint of data. Not encryption; there is no key and no reversing it. **HMAC** — a keyed hash proving both integrity and that the sender held the shared key. **HSTS** — HTTP Strict Transport Security. Forces HTTPS, preventing protocol downgrade. **Implicit intent** — an Android intent describing an action, letting the system pick a handler. Interceptable; avoid for internal use. **Injection** — a bug class where data gets interpreted as code. Fixed by

separating code from data. **IPC** — inter-process communication. On Android: intents, services, broadcasts, content providers. **IV / initialization vector** — a per-operation value making identical plaintext encrypt differently. Must never repeat under the same key with GCM. **Indirect prompt injection** — attacker-controlled content steering a model's behaviour, arriving via content the model reads rather than the user's own message. **JADX** — decompiler turning an APK into readable Java-like source. **JWT** — JSON Web Token. Signed, not encrypted; anyone can read the payload. **Keyblob** — encrypted key material that the Android keystore daemon can store but cannot use or reveal. **KeyMint** — the current Android HAL for key operations, replacing Keymaster. Added Curve25519 support. **keystore2** — the modern Android keystore daemon, rewritten in Rust. **MAS-L1 / MAS-L2 / MAS-R** — MASTG testing profiles: basic, higher, and reverse-engineering resilience. **MASTG** — OWASP Mobile Application Security Testing Guide. The tests. Currently v2.0.0. **MASVS** — OWASP Mobile Application Security Verification Standard. The requirements. Currently v2.1.0. **MASWE** — OWASP Mobile App Security Weakness Enumeration. Bridges controls and tests. **MAC (message authentication code)** — a keyed value proving data was not altered and came from someone holding the shared key. HMAC-SHA256 is the usual choice. Unrelated to a network MAC address. **MITM (man-in-the-middle)** — an attack where an adversary secretly sits between two parties and can read or alter what passes. **MSTG** — the old name for the MASTG. A source still using it predates the v2 refactor. **NSPinnedDomains / Identity Pinning** — Apple's declarative pinning, configured in `Info.plist` from iOS 14. Cannot be changed at runtime and does not cover `WKWebView`. **Nonce** — a number used once, included in a request so an old copy cannot be replayed. **objection** — Frida-powered toolkit for runtime mobile exploration without writing scripts. **OIDC (in CI)** — short-lived, workflow-scoped cloud credentials minted per run, replacing stored static keys. **PKI (public key infrastructure)** — the system of certificate authorities and certificates that lets clients validate a server's identity. The "web PKI" is the public one your device trusts by default. **PendingIntent** — a wrapped Android intent another app can fire as you, with your permissions. Use immutable, with an explicit base intent. **PKCE** — Proof Key for Code Exchange. Protects an OAuth authorization code in a public client. Required for native apps by RFC 8252. **Play App Signing** — Google holds your app signing key and re-signs on upload; you hold only an upload key. **Pin manifest** — a signed list of pins fetched at runtime in dynamic pinning. Signed with a key outside the web PKI. **Provenance** — a signed statement about how an artifact was built. Attests to the process, not to the cleanliness of the inputs. **pull_request_target** — a workflow trigger running with repository secrets in scope against fork code. High risk. **Public client** — an OAuth client that cannot hold a secret. Every mobile app is one. **HAL (hardware abstraction layer)** — the interface between Android and a device's hardware implementation. KeyMint is one. **KMP / CMP** — Kotlin Multiplatform and Compose Multiplatform: sharing logic, and sharing UI, across Android and iOS. **Custom ROM** — a third-party build of Android. Common in some regions and a frequent source of false positives in root detection. **RASP** — Runtime Application Self-Protection. In-app detection of tampering and instrumentation, best used to produce signals. **RKP** — Remote Key Provisioning. Android's privacy-preserving replacement for factory-provisioned attestation keys. New root active from 1 February 2026. **Static / dynamic / hybrid pinning** — pins baked in at build time; pins fetched at runtime from a signed manifest; or both, with a static backup under a dynamic primary. **Salt** — a random per-password value added before hashing, defeating precomputed attacks. **Sandbox** — the platform isolation giving each app private storage and its own process. **SSL** — the predecessor to TLS, long obsolete as a protocol but still used loosely in phrases such as "SSL pinning", which in practice always means TLS. The inventory of what went into your build. **Secure Enclave** — Apple's dedicated coprocessor for key storage and cryptographic operations. EC keys only. **securityd** — the iOS daemon mediating all Keychain access based on entitlements. **SHA pinning** — referencing a dependency or CI action by immutable commit hash rather than a mutable tag. **SLSA** — Supply-chain Levels for Software Artifacts. A build-integrity framework expressed in levels. **Step-up authentication** — requiring additional proof for a sensitive action inside an existing session. **STRIDE** — a threat modelling prompt set: spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege. **SPKI** — Subject Public Key Info. The correct thing to pin, because it survives certificate renewal with a reused key. **StrongBox** — a dedicated secure processor (embedded or integrated Secure Element) providing Android's strongest key protection. From Android 9. **TEE** — Trusted Execution Environment. A secure area of the main processor, isolated from the OS. **Trusty** — Google's open-source TEE implementation, provided to OEMs. **WHOIS** — the public registry of domain ownership records. Email validation based on it was discontinued on 15 July 2025. **WebView** — an embedded browser engine running web content inside your app's security context. **TXT record** — a DNS record holding arbitrary text. Used by certificate authorities for domain validation, including the standing record in persistent DCV. **Trust anchor** — the key or certificate a verification ultimately rests on. In dynamic pinning it is your manifest signing key, not a CA. **Trust boundary** — a line where data crosses from something you control to something you do not. Every one needs a check. **Verified boot** — a hardware-backed check that the device booted an unmodified OS. Now required for Play Integrity's device integrity verdict on Android 13+.

Corrections and additions welcome. If you find something stale, update Chapter 19 with the date and what changed — that is what keeps a book like this worth reading.